DATA7201 Data Analytics at Scale - Project Report

Examining Non-Government Aligned Election Spending

Stewart Fletcher
Student Number: s3348393

May 18, 2026

Abstract

This report examines non-government advertisers on the Facebook ad platform around the May 2022 federal election in order to determine the existence of American-style political action committee (PAC) advertisers in Australian elections.

Facebook ad data surrounding the election was ingested and preprocessed using PySpark on YARN. A pandas UDF tagged candidate, party, and government advertising via token matching against the Australian Electoral Commission candidate roll. The remaining ads were clustered with Latent Dirichlet Allocation and hand-labelled by category to exclude non-political content.

The corpus was then analysed in two ways: a deterministic classifier based upon ad spending before and after the election, and a machine learning classifier using hashtags to create a training dataset for a logistic-regression text classifier. Neither metric alone was sufficient to properly classify advertisers.

PAC-style advertisers were defined as those with strong pre-election spend relative to post-election spend combined with election-focused content in the weeks before polling day. Combining the two classifiers as a quadrant filter on these criteria identified a defensible list of Australian PAC equivalents.

A number of advertisers were subsequently found who matched the PAC-style advertisers seen in the United States, including Smart Voting, the Australian Education Union, Thrive By Five, the Australian Christian Lobby, GetUp!, ABC Friends, and It's Not A Race.

Contents

Word Count

The following is a summary of the word count this report, as generated by `texcount`.

Section	**Words**
Introduction	206
Data ingestion and preparation	267
Identifying government advertising	126
Topic clustering via LDA	165
Temporal analysis: persistence and centroid bands	237
Content analysis: hashtag-supervised classifier	272
Discussion and Conclusions	328
Total	**1,601**

Table 1: Word count breakdown.

1 Introduction

Political Action Committees (PACs) are an American campaigning vehicle that raise money for political advertising outside of the existing political structure. Australia does not formally have PACs. Yet independent, third-party advertisers spend millions of dollars on advertising during Australian elections. Identifying these informal Australian "PACs" is important for political-advertising transparency, and for finding the boundary between advertisers who are campaigning on specific issues and advertisers who are campaigning on behalf of political parties with private funding.

The Facebook ad platform is attractive for advertisers as it provides access to a large viewer base with minimal effort. Traditional methods such as door-knocking and physical advertising are not required. The Facebook Ad Library is also a convenient data source for finding Australian PAC equivalents. A dataset was provided covering four years of political advertising in Australia, from 2020 to 2024. Although this corpus covered a range of political events, this report focuses on the 2022 Australian federal election.

At approximately 320 GB of JSON files, the dataset cannot easily be analysed on a conventional workstation. A distributed analytics framework is required. PySpark (Zaharia et al., 2016) was used for its lazy DataFrame API (Armbrust et al., 2015), and Spark MLlib (Meng et al., 2016) was used for both the unsupervised topic modelling (LDA) and the supervised text classifier (logistic regression).

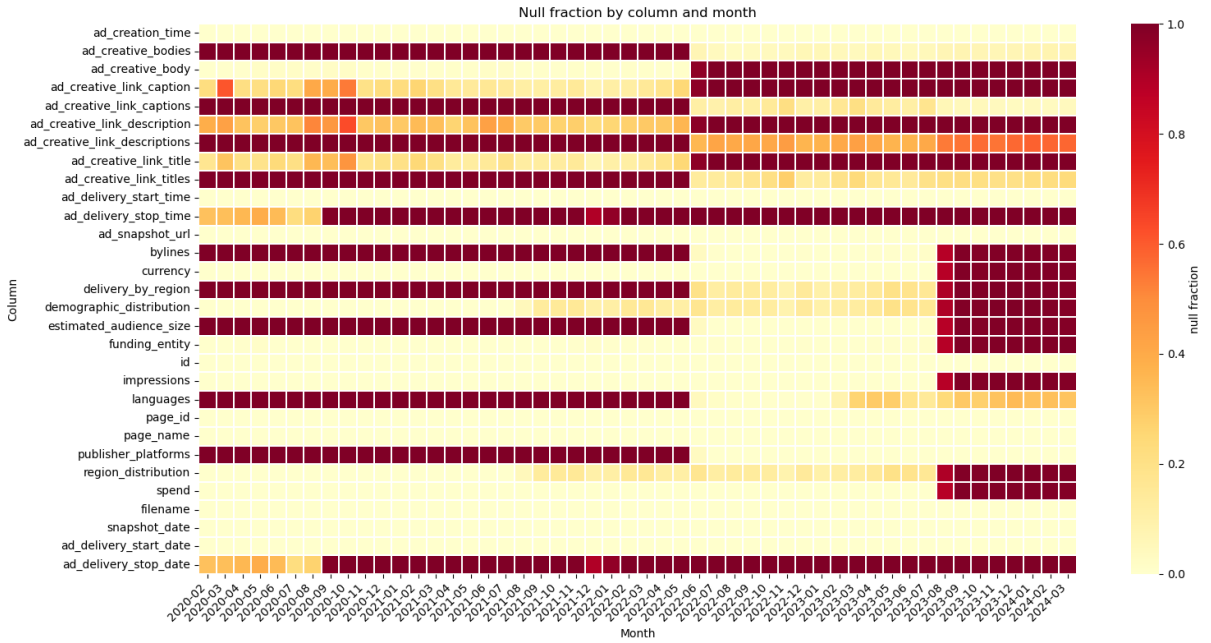
2 Dataset Analytics

2.1 Data ingestion and preparation

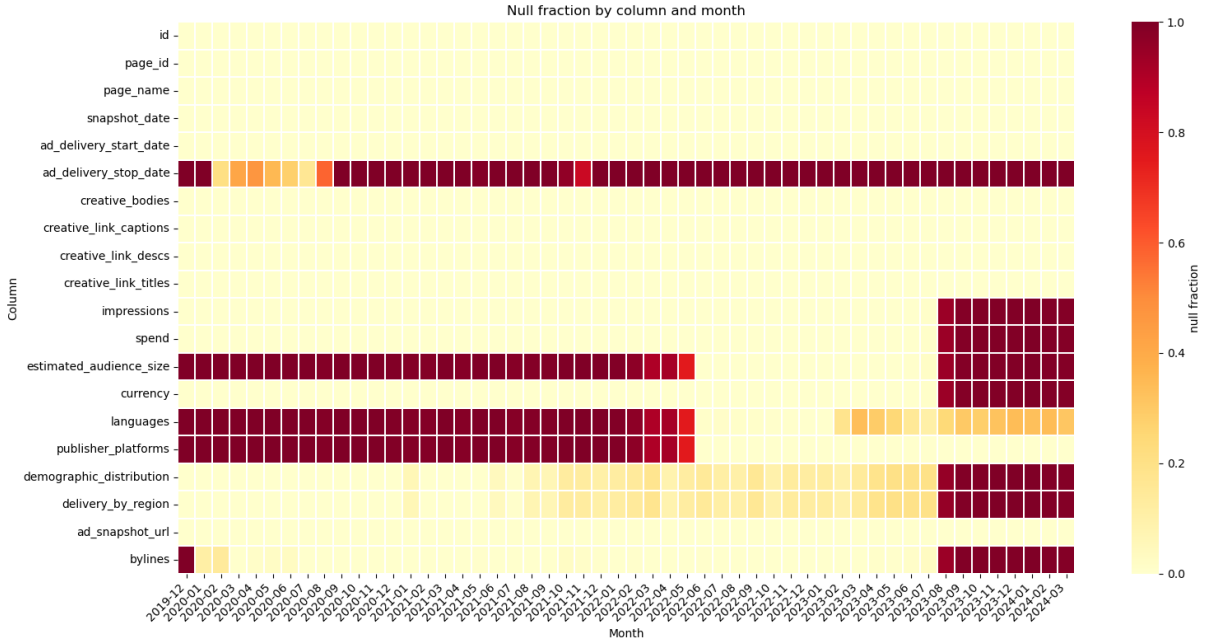
The data consisted of approximately 4,500 JSON files. These files contained around 40 million rows of data covering the period from February 2020 to March 2024. Each file was a snapshot of all Australian political advertisements that were active at the time the snapshot was taken. The same ad could therefore appear in many files. Each row represented a single advertisement with around 25 columns covering the ad purchaser, the Facebook page, the creative text, the spend, the impressions and various other fields. Some schema changes occurred in June 2022 and August 2023, with several key fields becoming sparse thereafter.

A null-fraction analysis was performed in order to visualise the schema drift. Automatic schema inference was used to collect the superset of all fields across the dataset. The per-column null fraction was then computed on a monthly basis and plotted as a heatmap (Figure 1). Columns were then merged to give a consistent schema across the window. Some fields were stored as nested structs of string-typed lower and upper bounds. These were cast to numeric values and replaced with their midpoint.

Deduplication was performed by sorting the snapshots chronologically per ad ID. Each row was then tagged with an `ad_seq_no` using a `row_number()` window operation, where a value of 1 indicates the most recent snapshot. The snapshot filenames used inconsistent date conventions and these required special handling. The cleaned data was then windowed to six months either side of the May 2022 federal election and written to HDFS as Parquet. This reduced the corpus from approximately 40 million rows down to 5.7 million rows.



(a) Before schema correction.



(b) After schema correction.

Figure 1: Null-fraction heatmaps (column × month) before and after the schema coalesce step. The raw view (top) shows the June 2022 singular-to-array transition and the August 2023 breakage clearly. The cleaned view (bottom) has the duplicate fields merged and a single consistent schema across the window. Hot colors show a high percentage of null values for that month/column.

2.2 Identifying government advertising

To focus on third-party organisations the government advertising had to be removed. This included both political-party advertisements and candidate advertisements. The 2022 House of Representatives and Senate candidate rolls were sourced from the Australian Electoral Commission. A straight join was not effective because page names can be misrepresentative and the byline can be paid by an aide. Thus a pandas UDF was used to apply three rules in order with the first match winning, summarised in Table 2.

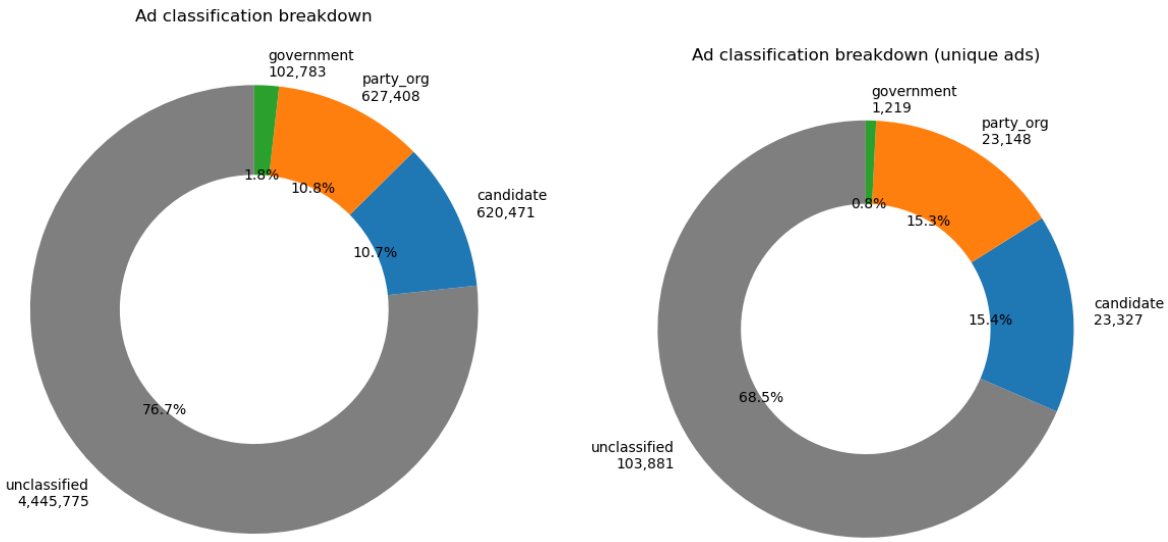
Rule	**Logic and output**
Candidate name	The candidate's tokenised name (surname plus first given name) is a subset of either the page-name tokens or the byline tokens.
Party byline signature	A tokenised party signature from `aec_parties.csv` is a subset of the byline tokens, e.g. {liberal, national} → LNP. Signatures are tested longest-first so LNP matches before LP.
Government byline signature	The byline tokens contain one of {department}, {australian, government}, or {australian, electoral, commission}.

Table 2: Three rules applied by the byline-matching pandas UDF in order, first match wins.

Every ad was thus tagged according to its nature; ads tagged `null` (no match against any of the three rules) formed the third-party corpus of interest. The row counts are shown in Table 3, with the match-type composition visualised in Figure 2 and the weekly spending pattern by match-type in Figure 3.

match_type	**total rows**	**unique ads**
null	4,511,742	105,814
party_org	649,857	23,932
candidate	536,232	20,644
government	98,660	1,188

Table 3: Rows and unique ads after government, party, and candidate classification.



(a) All rows (per snapshot).

(b) Unique ads (`ad_seq_no`=1).

Figure 2: Match-type distribution by total rows (left) and by unique ads after deduplication (right). The `null` bucket dominates by both measures and forms the third-party corpus carried forward.

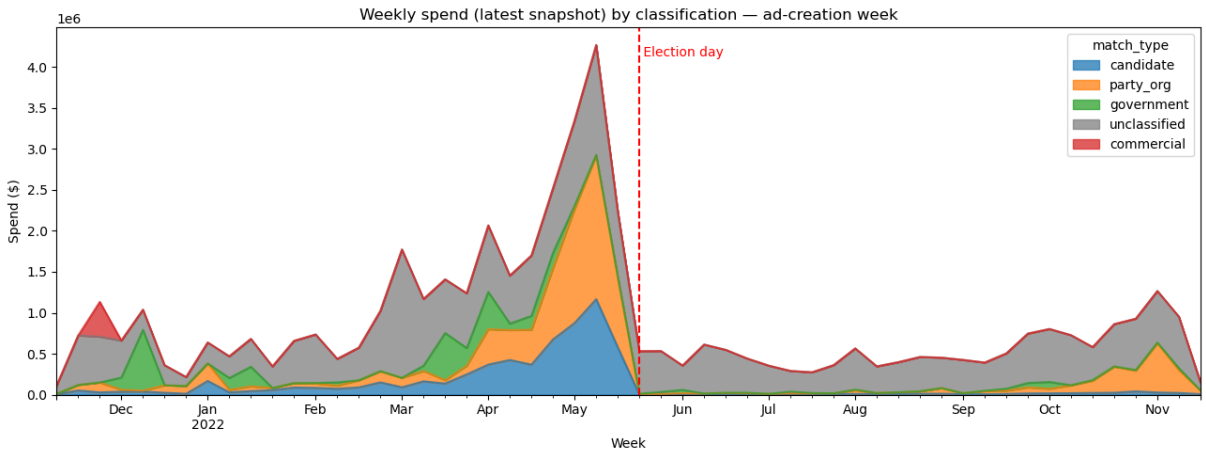


Figure 3: Weekly spend by match-type across the election window. The candidate and party advertising spike around the May 2022 election as expected; the `null` bucket shows a more diffuse pattern, which motivates the topic-clustering step in the next subsection.

2.3 Topic clustering via LDA

The non-government component contained a range of topics, not all of which were relevant to the election. The corpus was too large to label by hand and no pre-existing labels were available for supervised training. Latent Dirichlet Allocation (LDA; Blei et al., 2003) is an unsupervised topic-clustering method which can group documents by latent themes without requiring training data. It was thus well suited to this corpus. After some standard text pre-processing, LDA was fitted with 25 topics. These topics were then hand-labelled with the categories `climate`, `humanitarian_rights`, `political_advocacy`, `cost_of_living` and `mixed`. This reduced the labelling problem from hundreds of thousands of ads down to just 25 topics.

Topics without a single coherent theme were tagged as `mixed` and retained in the corpus, so

downstream analysis could choose whether to include them. The result was a corpus of about 95,000 advocacy ads, written partitioned by category. The per-topic distribution is shown in Figure 4. Notably the largest spending bucket was the non-government `political_advocacy` category.

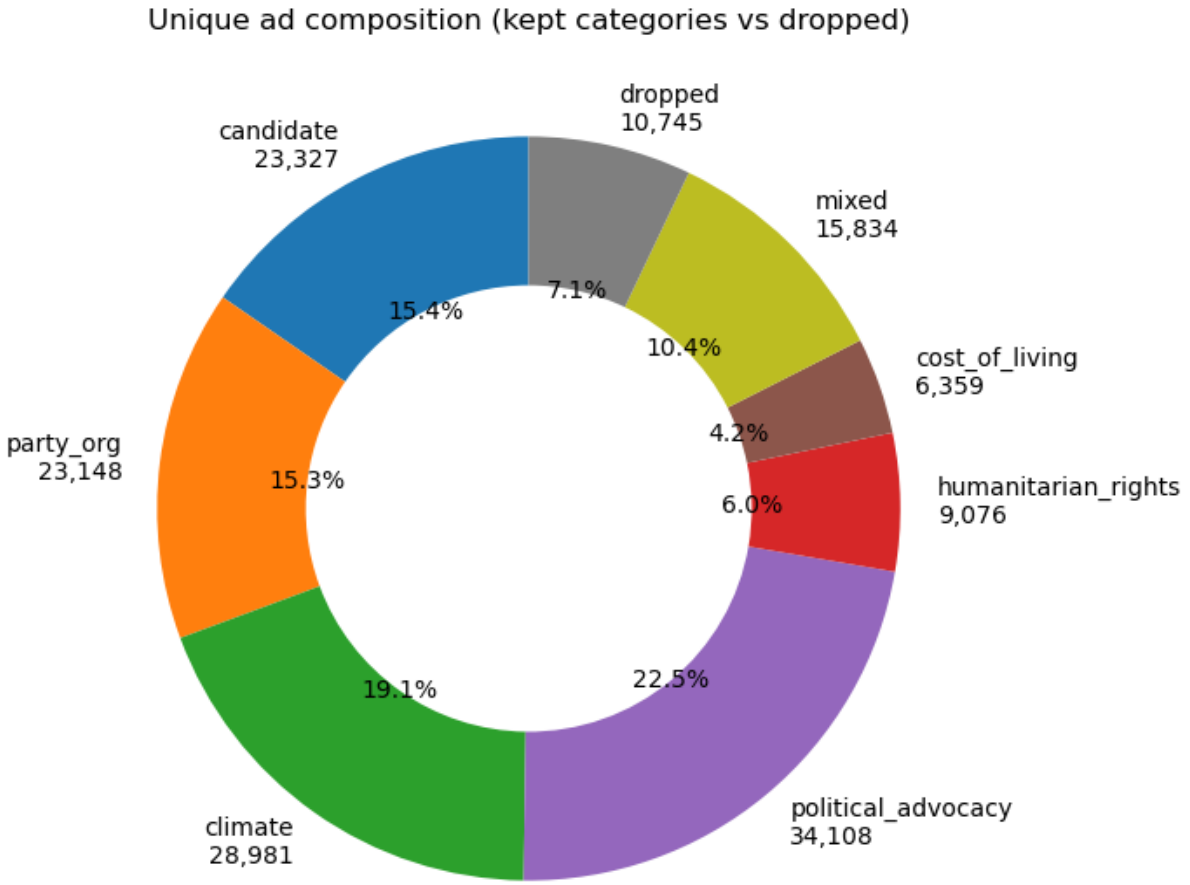


Figure 4: Distribution of ads across the 25 LDA topics with the hand-assigned categories.

2.4 Temporal analysis: persistence and centroid bands

A simple method for identifying "PAC"-style advertising is to look at spending before and after the election. An advertiser who has a high pre-election spending spike compared to the post-election period is more likely to be focused on the election.

In order to compare advertisers, spending was collated into a weekly time series per advertiser. The resulting time series was very bursty so a four-week rolling mean was then applied to smooth it. A hard boundary was also added at the election date in order to prevent bleed-over from the pre-election month into the post-election month. The resulting weekly spending pattern across the non-government corpus is shown in Figure 5.

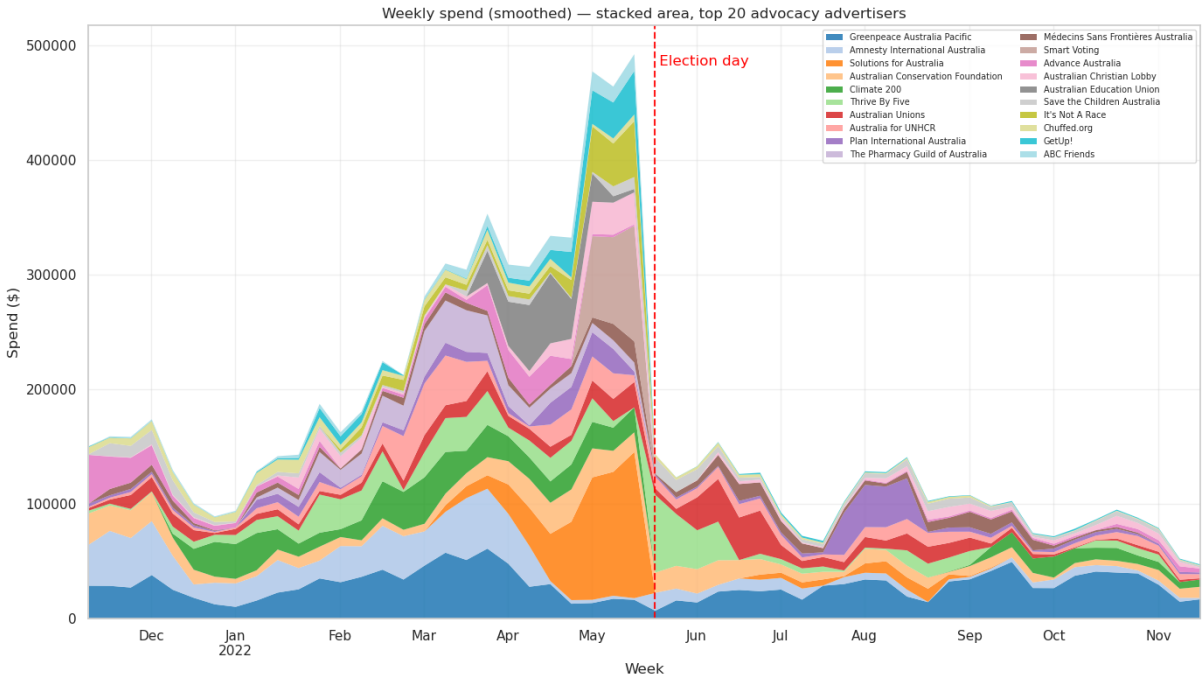


Figure 5: Weekly spend across the non-government (advocacy) corpus. Spending clusters in the weeks leading up to the May 2022 election and drops sharply afterwards.

A ratio of pre-election spend to post-election spend was calculated and saved as "persistence". Investigation of persistence found it was insufficient to identify strong pre-election spenders alone. Early-window spenders received the same weight as near-election spenders, and due to the crossover with the start of the Voice referendum, some political spenders had a late, post-election spend which reduced their ratio.

A second metric was thus developed. "Centroid" was the weighted-mean week from election day, where negative values indicate spending before the election and positive values indicate spending after. Centroid was then combined with persistence to identify advertisers who focused their spend in the near pre-election period.

Advertisers were split into four deterministic bands as shown in Table 4. The bands are also plotted on a quadrant in Figure 6.

Band	**Rule**
`election_only`	persistence < 0.30 and $\lvert\text{centroid}\rvert \leq 16$ weeks
`transitional`	$0.30 \leq$ persistence ≤ 0.80
`ongoing`	persistence > 0.80
`off_campaign`	persistence < 0.30 and $\lvert\text{centroid}\rvert > 16$ weeks

Table 4: Four-band classifier applied to top-20 advertisers.

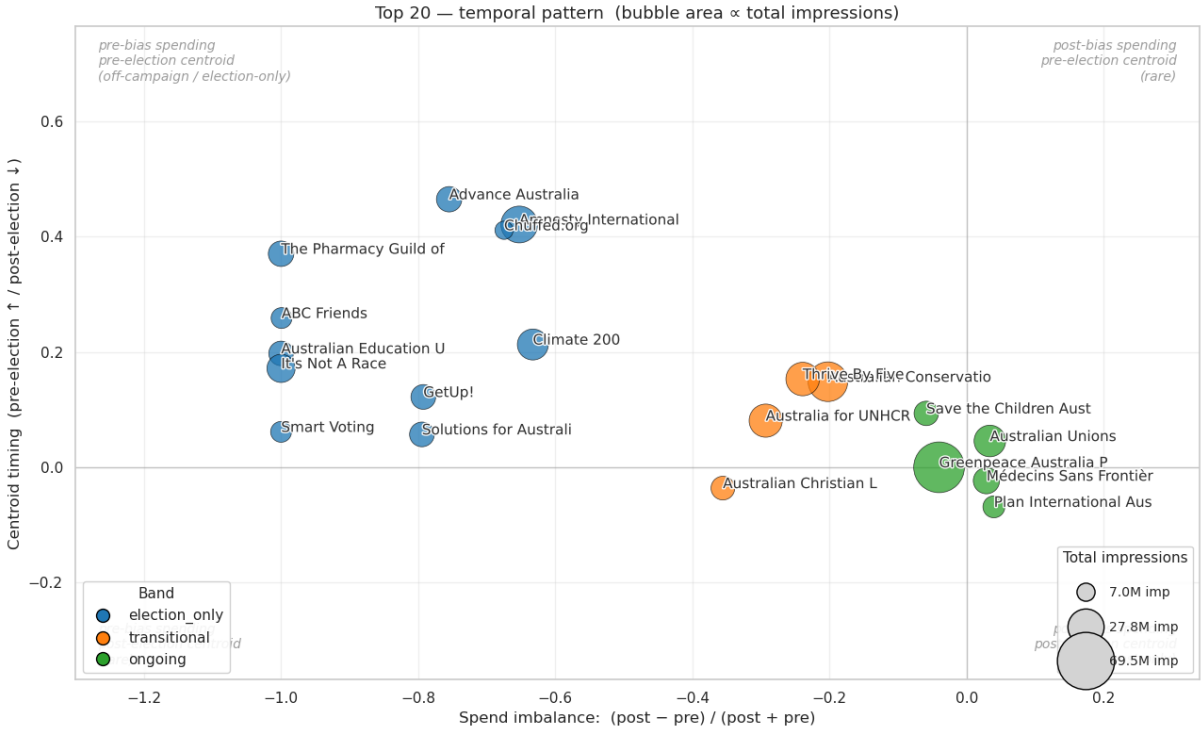


Figure 6: Top 20 non-government advertisers by spend. Low persistance and centroid indicates ongoing spend. Larger persistance and a strong negative centroid indicates an advertisiting spend focused on the pre-election period.

2.5 Content analysis: hashtag-supervised classifier

Detection based purely upon spending patterns has a number of weaknesses. Firstly, it may identify issue-based advertisers whose spend incidentally peaks before the election. Secondly, the temporal classifier reveals nothing about an advertiser's actual content. Two advertisers with identical spend patterns may be running completely different campaigns. A content-based classifier was added to capture the second case.

Supervised classification requires a training corpus, and after some experimentation hashtags proved a reliable way to extract one. About 210 political hashtags (e.g. `#federalelection`, `#morrisonmustgo`) were drawn from the `election_only` category and about 286 non-political hashtags (e.g. `#breakfreefromplastic`) from the `ongoing` category, then hand-curated. Labels were assigned as:

- Positive: at least one political hashtag, no non-political hashtag.
- Negative: at least one non-political hashtag, no political hashtag.
- Ambiguous ads excluded from training but scored at inference.

Hashtags were stripped from the text before training so the model learned from the surrounding prose rather than memorising the labels. A logistic regression classifier, a standard approach for text classification (Manning et al., 2008), was trained on the labelled corpus of around 8,000 ads.

The classifier performed strongly on held-out test data, but weaker when validated against advertisers it had not seen during training. This suggests that more advanced methods may improve coverage. Inspection of high-scoring no-hashtag ads confirmed the model caught political content the hashtag-based labelling had missed, including local-council and state-election candidates running outside the federal-election window.

Per-advertiser scores were aggregated as the mean `election_prob` across each advertiser's ads in the 4-week pre-election window. This `mean_prob_pre` metric was then exported for the combined PAC analysis in Section 3. Figure 7 shows this score across the top-20 spenders.

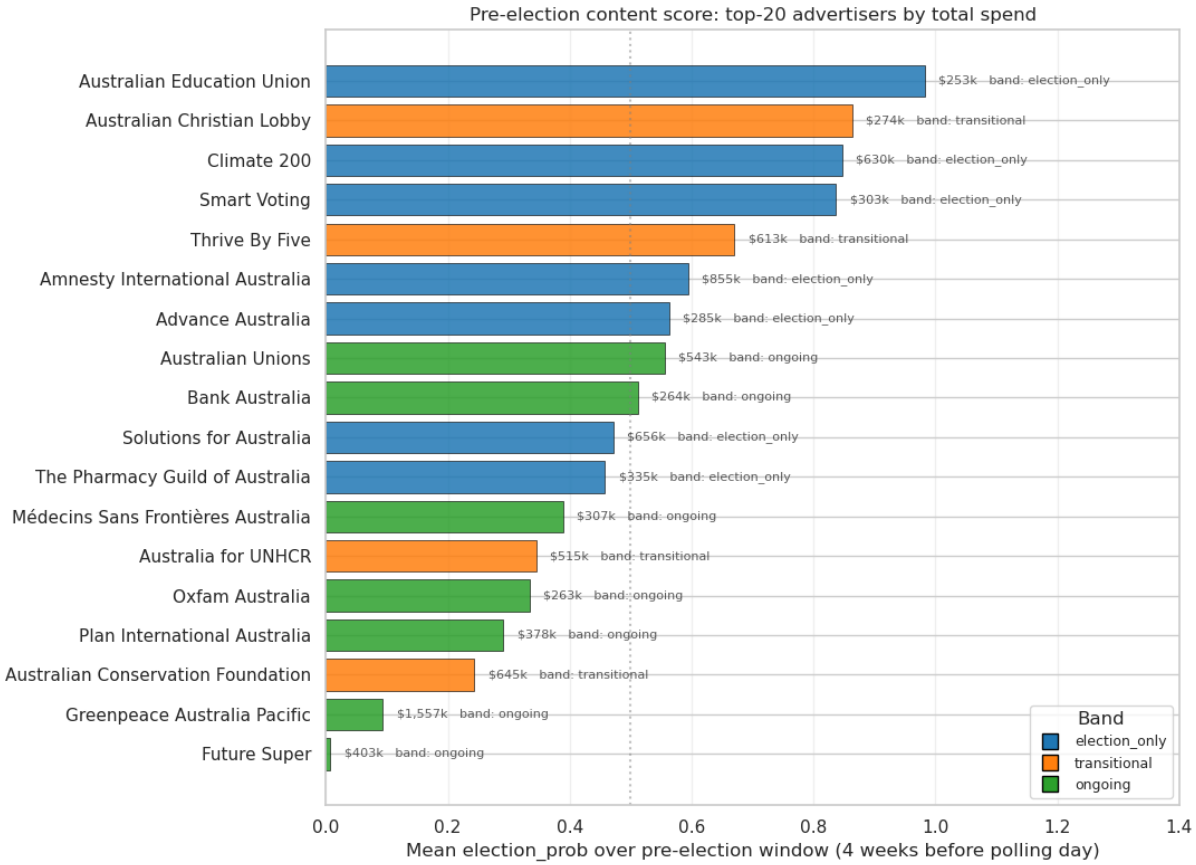


Figure 7: Mean `election_prob` over the 4-week pre-election window for the top-20 advocacy advertisers by spend, coloured by temporal band. Bars longer than 0.5 indicate advertisers whose pre-election content reads as predominantly election-themed by the classifier.

3 Discussion and Conclusions

Classifying PAC-style advertisers proved more challenging than anticipated. The temporal classifier alone tended to confuse ongoing-issue advocates such as Greenpeace and the Australian Unions with election-window spenders, while the content classifier alone picked up local-council and state-election candidates outside the federal window. Combining the two as a quadrant filter (`ec_index > 0.5` and `mean_prob_pre > 0.5`, excluding the `ongoing` band) produced a reasonable list. More advanced methods, such as transformer-based text models, may potentially improve the result.

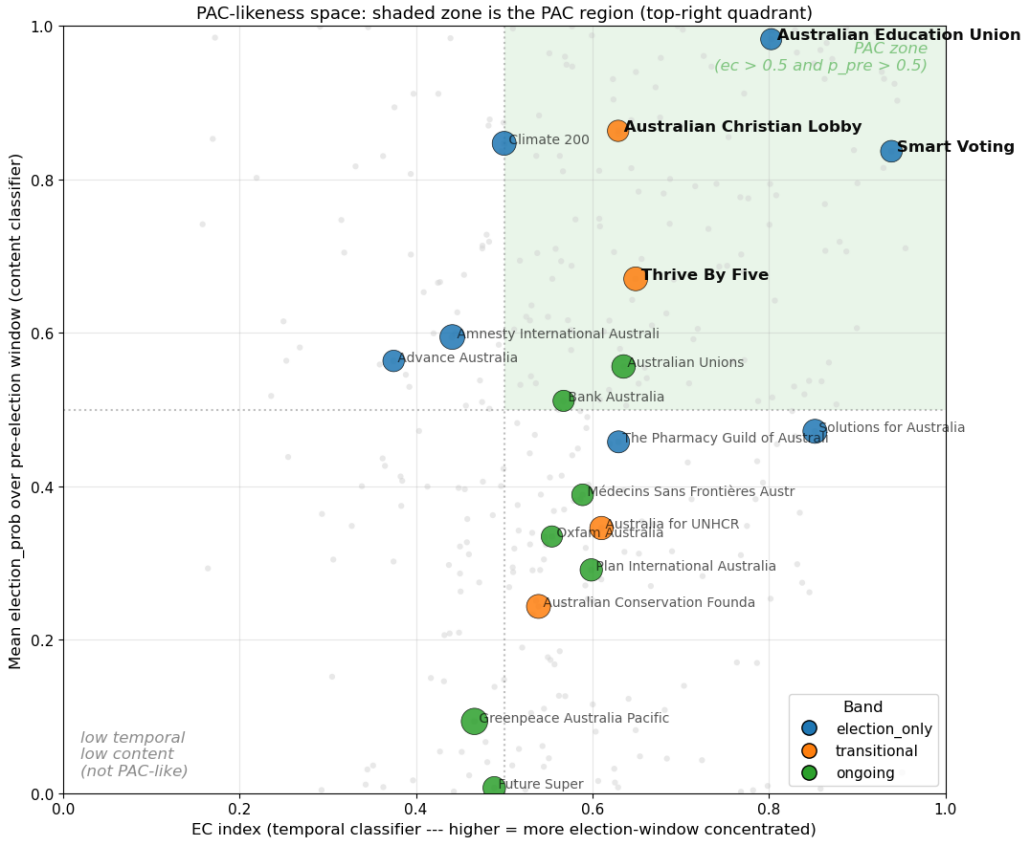


Figure 8: Combined PAC-likeness scatter: temporal classifier (`ec_index`, x-axis) against content classifier (`mean_prob_pre`, y-axis) for the top-20 advocacy advertisers. The shaded top-right quadrant is the PAC region. Greenpeace sits in the bottom-left despite the largest bubble ($1.5M).

Nevertheless several PAC-style advertisers were identified. These included Smart Voting, the Australian Education Union, Thrive By Five, the Australian Christian Lobby, GetUp!, ABC Friends, and It's Not A Race. Inspection of the top ad titles for each (Table 5) confirmed the classification. All of these advertisers combined a pre-election spend concentration with content that was explicitly partisan or directed at a specific policy outcome.

Advertiser	**Band**	**Top ad title**
Smart Voting	election_only	"Since the Coalition took power, prices have skyrocketed."
Australian Education Union	election_only	"Properly fund public schools."
Thrive By Five	transitional	"Let's make early learning and childcare fair for families."
Australian Christian Lobby	transitional	"Sign the petition to defend Christian schools."
It's Not A Race	election_only	"Government should be about setting the right priorities."
GetUp!	election_only	"Want climate leadership? Vote for it!"
ABC Friends	election_only	"Save our ABC."

Table 5: Top PAC-style advertisers identified by the combined quadrant filter, with their highest-spend ad title. All combine pre-election spend concentration with explicit issue or party-directional messaging.

The major outlier was Greenpeace. At $1.5M the largest non-government advertiser, it nevertheless satisfied none of the PAC criteria, with a fairly constant spend curve and an anti-Woodside Petroleum campaign rather than electoral content. High spend during an election window does not, on its own, indicate PAC-style behaviour.

Despite the limitations of the classifiers a reasonably confident list of Australian PAC-style advertisers can be identified from this dataset. Australia does not have formal PACs, but their informal equivalents do operate around federal elections. As discussed in the introduction, the boundary between issue advocacy and party-aligned third-party campaigning is one that traditional campaign-finance disclosure cannot easily examine. The use of the Facebook Ad Library may be one method to allow this boundary to be examined.

The scale of the dataset meant that the analysis was only practical using a distributed framework. The 40-million-row, 320 GB JSON corpus could not have been deduplicated, clustered with LDA, or joined against the candidate roll on a conventional workstation in any reasonable time. Distributed storage, lazy DataFrame evaluation, and the Spark MLlib library were all used to make the full analysis viable.

References

Armbrust, M., R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi, and M. Zaharia (2015). Spark SQL: relational data processing in Spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pp. 1383–1394.

Blei, D. M., A. Y. Ng, and M. I. Jordan (2003). Latent Dirichlet allocation. *Journal of Machine Learning Research 3*, 993–1022.

Manning, C. D., P. Raghavan, and H. Schütze (2008). *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press.

Meng, X., J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar (2016). MLlib: Machine learning in Apache Spark. *Journal of Machine Learning Research 17*(34), 1–7.

Zaharia, M., R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica (2016). Apache spark: a unified engine for big data processing. *Communications of the ACM 59*(11), 56–65.

A Use of Generative AI

Claude (Anthropic; `claude-opus-4-7`, accessed via the Claude Code CLI between April and May 2026) was used for proofreading and editing the report text, and for help with parts of the matplotlib visualisation code in notebooks 02, 04 and 05. All substantive content, the analytical argument, the PySpark and pandas analysis pipeline, and all decisions about what to include in the report were the author's. The author reviewed and verified all code, numerical results, and claims before inclusion.

B Code

Python scripts contain the PySpark code that generates each intermediate output. Notebooks contain exploratory code, validation and QC, and plotting code.

Python scripts

Listing 1: 01_data_loading.py

```
from pyspark.sql import SparkSession, DataFrame
from pyspark.sql.functions import (
    input_file_name,
    regexp_extract,
    to_date,
    format_string,
    substring,
    col,
    coalesce,
    array,
    row_number,
)
from pyspark.sql.window import Window


RAW_PATH = "/data/ProjectDatasetFacebookAU/"
PARQUET_PATH = "/user/s3348393/main/preprocessing/v1/parquet"
WINDOW_START = "2021-11-21"
WINDOW_END = "2022-11-21"


def load_raw(spark: SparkSession, path: str) -> DataFrame:
    """
    load the data from the raw jsons. include the filename in a column
    snapshot_date. the date format changes a bit so the regex is designed
    to handle that.
    """
    df = spark.read.json(path)
    df = df.withColumn("filename", input_file_name())

    date_re = r"-(\d{4})(\d{1,2})(\d{2})-"
    yyyy = regexp_extract("filename", date_re, 1).cast("int")
    mm = regexp_extract("filename", date_re, 2).cast("int")
    dd = regexp_extract("filename", date_re, 3).cast("int")
    date_str = format_string("%d-%02d-%02d", yyyy, mm, dd)

    df = df.withColumn("snapshot_date", to_date(date_str, "yyyy-MM-dd"))
    return df


def cast_dates(df: DataFrame) -> DataFrame:
    """
    cast datetime strings to date types.
    """
    for src in ["ad_creation_time", "ad_delivery_start_time", "
        ad_delivery_stop_time"]:
        dst = src.replace("_time", "_date")
        df = df.withColumn(dst, to_date(substring(src, 1, 10), "yyyy-MM-dd"))
    return df


def collapse_schema(df: DataFrame) -> DataFrame:
    """
    schema cleanup, based upon the analysis in notebook 01
    """
    df = df.select(
        "id",
        "page_id",
        "page_name",
        "snapshot_date",
```

```
        "ad_creation_date",
        "ad_delivery_start_date",
        "ad_delivery_stop_date",
        coalesce(col("ad_creative_bodies"), array(col("ad_creative_body"))).
            alias("creative_bodies"),
        coalesce(col("ad_creative_link_captions"), array(col("
            ad_creative_link_caption"))).alias("creative_link_captions"),
        coalesce(col("ad_creative_link_descriptions"), array(col("
            ad_creative_link_description"))).alias("creative_link_descs"),
        coalesce(col("ad_creative_link_titles"), array(col("
            ad_creative_link_title"))).alias("creative_link_titles"),
        "spend_lower_bound",
        "spend_upper_bound",
        "spend_mid",
        "impressions_lower_bound",
        "impressions_upper_bound",
        "impressions_mid",
        "audience_size_lower_bound",
        "audience_size_upper_bound",
        "audience_size_mid",
        "currency",
        "languages",
        "publisher_platforms",
        "demographic_distribution",
        coalesce(col("delivery_by_region"), col("region_distribution")).alias("
            delivery_by_region"),
        "ad_snapshot_url",
        coalesce(col("bylines"), col("funding_entity")).alias("bylines"),
        "ad_seq_no",
    )
    return df


def filter_election_window(df: DataFrame, start: str, end: str) -> DataFrame:
    """
    limit data to 6 months before/after the 2022 federal election
    """
    df = df.filter((col("ad_creation_date") >= start) & (col("ad_creation_date")
         <= end))
    return df


def flatten_numeric_structs(df: DataFrame) -> DataFrame:
    """
    we have a couple of structs like spend that have a min
    and max value as a string. this casts them to longs
    then finds the average value. drops the originals.
    """
    df = df.withColumn("spend_lower_bound", col("spend.lower_bound").cast("long"
        ))
    df = df.withColumn("spend_upper_bound", col("spend.upper_bound").cast("long"
        ))
    df = df.withColumn("spend_mid", (col("spend_lower_bound") + col("
        spend_upper_bound")) / 2.0)

    df = df.withColumn("impressions_lower_bound", col("impressions.lower_bound")
        .cast("long"))
    df = df.withColumn("impressions_upper_bound", col("impressions.upper_bound")
        .cast("long"))
    df = df.withColumn("impressions_mid", (col("impressions_lower_bound") + col(
        "impressions_upper_bound")) / 2.0)
```

```
    df = df.withColumn("audience_size_lower_bound", col("estimated_audience_size
        .lower_bound").cast("long"))
    df = df.withColumn("audience_size_upper_bound", col("estimated_audience_size
        .upper_bound").cast("long"))
    df = df.withColumn("audience_size_mid", (col("audience_size_lower_bound") +
        col("audience_size_upper_bound")) / 2.0)

    df = df.drop("spend", "impressions", "estimated_audience_size")
    return df


def add_snapshot_sequence(df: DataFrame) -> DataFrame:
    """window partition into ad id, then order by snapshot date,
    latest first, then label them with row numbers. e.g. filter by ad_seq_no = 1
    to get the deduped dataset, but at this stage we dont wipe the other rows
    """
    w = Window.partitionBy("id").orderBy(col("snapshot_date").desc())
    return df.withColumn("ad_seq_no", row_number().over(w))


def write_output(df: DataFrame, path: str) -> None:
    """ write into parquet with 8 extents"""
    df.coalesce(8).write.parquet(path, mode="overwrite")


def main() -> None:
    spark = (
        SparkSession.builder
        .appName("FB_API_topics")
        .config("spark.sql.parquet.output.committer.class", "org.apache.parquet.
            hadoop.ParquetOutputCommitter")
        .config("mapreduce.fileoutputcommitter.algorithm.version", "2")
        .getOrCreate()
    )

    df = load_raw(spark, RAW_PATH)
    df = cast_dates(df)
    df = filter_election_window(df, WINDOW_START, WINDOW_END)
    df = flatten_numeric_structs(df)
    df = add_snapshot_sequence(df)
    df = collapse_schema(df)
    write_output(df, PARQUET_PATH)

    spark.stop()


if __name__ == "__main__":
    main()
```

Listing 2: 02_preprocessing.py

```
import re
import pandas as pd
from pyspark.sql import SparkSession, DataFrame
from pyspark.sql.functions import col, coalesce, lit, pandas_udf
from pyspark.sql.types import StringType, StructType, StructField
from pyspark.ml.feature import RegexTokenizer, StopWordsRemover


IN_PATH = "/user/s3348393/main/preprocessing/v1/parquet"
OUT_PATH = "/user/s3348393/main/preprocessing/v2/parquet"
HOUSE_CSV = "../data/2022_election_candidates.csv"
SENATE_CSV = "../data/2022_senate_candidates.csv"
PARTIES_CSV = "../data/aec_parties.csv"
```

```
HONORIFICS = ["mp", "hon", "dr", "mr", "mrs", "ms", "sen", "senator", "rt", "
    authorised", "by", "for"]

GOV_BYLINE_SIGNATURES = [
    frozenset({"department"}),
    frozenset({"australian", "government"}),
    frozenset({"australian", "electoral", "commission"}),
    frozenset({"australian", "cyber", "security"}),
]

COMMERCIAL_BYLINE_SIGNATURES = [
    frozenset({"shell"}),
]

INTERMEDIATE_COLUMNS = [
    "page_name_safe", "bylines_safe",
    "page_name_tokens", "bylines_tokens",
    "page_name_terms", "bylines_terms",
]


def load_v1(spark: SparkSession, path: str) -> DataFrame:
    """
    read the v1 parquet written by 01_data_loading.py.
    """
    return spark.read.parquet(path)


def load_candidates(house_csv: str, senate_csv: str) -> pd.DataFrame:
    """
    load 2022 house + senate candidates from the AEC csvs and stack them
    into a single frame. i made a typo in the senate csv, so i'll fix it here
    instead of re-doing the csv.
    """
    house = pd.read_csv(house_csv, header=0)
    senate = pd.read_csv(senate_csv, header=0).rename(columns={"PartyAB": "
        PartyAb"})

    candidates = pd.concat(
        [house[["PartyAb", "PartyNm", "Surname", "GivenNm"]],
         senate[["PartyAb", "PartyNm", "Surname", "GivenNm"]]],
        ignore_index=True,
    )
    return candidates


def tokenise_text_columns(df: DataFrame) -> DataFrame:
    """
    ok we're going to match the page name and byline to the candidates.
    tokenising byline and page name
    """
    stopwords = StopWordsRemover.loadDefaultStopWords("english") + HONORIFICS

    #tokenize page names
    df = df.withColumn("page_name_safe", coalesce(col("page_name"), lit(""))) #
        cast nulls to empty strings
    df = RegexTokenizer(inputCol="page_name_safe", outputCol="page_name_tokens",
         pattern=r"\W+", toLowercase=True).transform(df)
    df = StopWordsRemover(inputCol="page_name_tokens", outputCol="
        page_name_terms",stopWords=stopwords).transform(df)

    #tokenize bylines
```

```
    df = df.withColumn("bylines_safe", coalesce(col("bylines"), lit("")))
    df = RegexTokenizer(inputCol="bylines_safe", outputCol="bylines_tokens",
        pattern=r"\W+", toLowercase=True).transform(df)
    df = StopWordsRemover(inputCol="bylines_tokens", outputCol="bylines_terms",
        stopWords=stopwords).transform(df)

    return df


def build_candidate_index(candidates: pd.DataFrame) -> list:
    """
    build the candidate match index.
    """
    def split_tokens(s):
        """
        tokenise an input string like the regextokeniser, on whitespace
        """
        if not isinstance(s, str):
            return []
        return [t for t in re.split(r"\W+", s.lower()) if t]

    candidate_list = []
    for _, row in candidates.iterrows():
        surname_tokens = split_tokens(row["Surname"])
        given_tokens = split_tokens(row["GivenNm"])
        candidate_tokens = frozenset(surname_tokens + given_tokens[:1])
        party = row["PartyAb"]
        if not surname_tokens:
            continue
        if not isinstance(party, str):
            continue
        candidate_list.append((candidate_tokens, party))

    return candidate_list


def load_party_byline_signatures(parties_csv: str) -> list:
    """
    load aec_parties.csv into a list of (frozenset_of_tokens, party_ab).
    preserves CSV order so specific signatures like {liberal, national} get a
        chance to match
    before generic ones like {liberal}.
    """
    aec_parties = pd.read_csv(parties_csv, header=0)

    signatures = []
    for _, row in aec_parties.iterrows():
        byline_tokens = row["byline_tokens"]
        if not isinstance(byline_tokens, str) or not byline_tokens.strip():
            continue
        signature = frozenset(t.strip() for t in byline_tokens.split("+") if t.
            strip())
        signatures.append((signature, row["party_ab"]))

    return signatures


def classify_ads(df: DataFrame, candidate_list: list, party_byline_signatures:
    list) -> DataFrame:
    """
    apply the priority classifier as a pandas_udf and add political_party
    and match_type columns. priority is candidate then party_org then
    government then commercial. replaced original python udf with pandas udf...
```

```
    it was too slow.
    """
    #output schema for the udf
    result_schema = StructType([
        StructField("political_party", StringType(), True),
        StructField("match_type", StringType(), True),
    ])

    def _classify_row(page_tokens, bylines_tokens):
        """ inner function to identify/label gov. advertising.

        pandas_udf. row by row.
        tries:
            candidate match against byline and page name
            party name against byline
            generic government keywords against byline
            a special case for shell
        if no matches, it returns all nones.
        """

        page_set = set(page_tokens) if page_tokens is not None else set()
        bylines_set = set(bylines_tokens) if bylines_tokens is not None else set
            ()

        # candidate name match - page name or byline
        parties = set()
        for candidate_tokens, party in candidate_list:
            if candidate_tokens.issubset(page_set) or candidate_tokens.issubset(
                bylines_set):
                parties.add(party)
        if len(parties) == 1:
            return (next(iter(parties)), "candidate")

        #party name match - byline only
        for sig, party in party_byline_signatures:
            if sig.issubset(bylines_set):
                return (party, "party_org")

        #gov. generic name match - byline only
        for sig in GOV_BYLINE_SIGNATURES:
            if sig.issubset(bylines_set):
                return (None, "government")

        #this is a special case for shell, which had a single, large non
        #political compaign during the election window
        for sig in COMMERCIAL_BYLINE_SIGNATURES:
            if sig.issubset(bylines_set):
                return (None, "commercial")

        return (None, None)

    #run the udf
    @pandas_udf(result_schema)
    def classify_ad(page_terms: pd.Series, bylines_terms: pd.Series) -> pd.
        DataFrame:
        results = [_classify_row(p, b) for p, b in zip(page_terms, bylines_terms
            )]
        return pd.DataFrame(results, columns=["political_party", "match_type"])

    #unpack the results
    df = df.withColumn("_class", classify_ad("page_name_terms", "bylines_terms")
        )
    df = df.withColumn("political_party", col("_class.political_party"))
```

```
    df = df.withColumn("match_type", col("_class.match_type"))
    df = df.drop("_class")
    return df


def drop_intermediate_columns(df: DataFrame) -> DataFrame:
    """
    cleanup
    """
    df = df.drop(*INTERMEDIATE_COLUMNS)
    return df


def write_v2(df: DataFrame, path: str) -> None:
    """
    write parquet partitioned by political_party. rows with null
    political_party land in political_party=__HIVE_DEFAULT_PARTITION__/
    which is expected - that's the bulk of the corpus.
    """
    df.write.partitionBy("political_party").parquet(path, mode="overwrite")


def main() -> None:
    spark = (
        SparkSession.builder
        .appName("FB_API_party_match")
        .config("spark.sql.parquet.output.committer.class", "org.apache.parquet.
            hadoop.ParquetOutputCommitter")
        .config("mapreduce.fileoutputcommitter.algorithm.version", "2")
        .getOrCreate()
    )

    df = load_v1(spark, IN_PATH)

    candidates = load_candidates(HOUSE_CSV, SENATE_CSV)
    candidate_list = build_candidate_index(candidates)
    party_byline_signatures = load_party_byline_signatures(PARTIES_CSV)

    df = tokenise_text_columns(df)
    df = classify_ads(df, candidate_list, party_byline_signatures)
    df = drop_intermediate_columns(df)
    write_v2(df, OUT_PATH)

    spark.stop()


if __name__ == "__main__":
    main()
```

Listing 3: 03_topics.py

```
import pandas as pd
from pyspark.sql import SparkSession, DataFrame
from pyspark.sql.functions import (
    array_contains, col, coalesce, concat_ws, desc, expr, length, lit,
        row_number,
)
from pyspark.sql.window import Window
from pyspark.ml import Pipeline
from pyspark.ml.clustering import LDA
from pyspark.ml.feature import CountVectorizer, RegexTokenizer, StopWordsRemover
from pyspark.ml.functions import vector_to_array
```

```
V2_PATH = "/user/s3348393/main/preprocessing/v2/parquet"
INTERMEDIATE_PATH = "/user/s3348393/main/preprocessing/v3/intermediate_parquet"
TOPIC_TERMS_CSV = "../data/topic_terms.csv"

# bylines we've identified as clearly non-political (commercial, recruitment,
# streaming services). grown iteratively when LDA surfaces a noise topic.
COMMERCIAL_BYLINES = {
    "Access",
    "Streamotion Pty Ltd",
    "SBS Australia",
    "SBS Arabic24",
    "SBS Mandarin\u4e2d\u6587\u666e\u901a\u8bdd",  # SBS Mandarin (Chinese)
    "The Squiz",
    "Hair Cooki",
    "Shell"
}

# english defaults plus URL fragments, contraction debris, and generic fillers.
# kept short - minDF and maxDF in CountVectorizer handle most frequency-based
# filtering for us.
DOMAIN_STOP_WORDS = [
    "https", "http", "www", "com", "org", "au", "co", "html",
    "re", "ve", "ll",
    "help", "time", "like", "need", "make", "take", "people",
    "year", "years", "today", "also", "will", "can", "get",
    "see", "know", "one", "two", "new", "now", "us",
    "click", "learn",'australia', 'australian', "2022"
]

# LDA + CountVectorizer hyperparameters - picked from the k-sweep in 03_topics.
    ipynb.
K = 25 #upped to 25 as we removed noise labels. TBH i'm not even sure if LDA is
    neccessary any more - our subsequent metrics should work either way
VOCAB_SIZE = 5000
MIN_DF = 100
MAX_DF = 0.3
LDA_MAX_ITER = 20
SEED = 42
TOP_TERMS_N = 15
TOP_BYLINES_N = 5


def load_v2(spark: SparkSession, path: str) -> DataFrame:
    """
    read the v2 parquet
    """
    return spark.read.parquet(path)


def first_non_empty(col_name: str):
    """
    spark way to get the first non-null value of an array
    """
    return expr(f"filter({col_name}, x -> x is not null and length(x) > 0)[0]")


def filter_to_residual_subset(df: DataFrame, commercial_bylines: set) ->
    DataFrame:
    """
    get the non-goverment, non-comercial, english language rows
    """
    return df.filter(
        (col("ad_seq_no") == 1)
```

```
        & col("match_type").isNull()
        & (col("languages").isNull() | array_contains("languages", "en"))
        & ~col("bylines").isin(list(commercial_bylines))
    )


def extract_body_text(df: DataFrame) -> DataFrame:
    """
    combine title, description and body, for those cases where the body is an
        image etc
    """
    df = df.withColumn("body_text", first_non_empty("creative_bodies"))
    df = df.withColumn("desc_text", first_non_empty("creative_link_descs"))
    df = df.withColumn("title_text", first_non_empty("creative_link_titles"))
    df = df.withColumn(
        "body",
        concat_ws(
            " ",
            coalesce(col("body_text"), lit("")),
            coalesce(col("desc_text"), lit("")),
            coalesce(col("title_text"), lit("")),
        ),
    )
    df = df.filter(length(col("body")) > 0)
    df = df.drop("body_text", "desc_text", "title_text")
    return df


def build_preprocessing_pipeline(stop_words: list) -> Pipeline:
    """
    three stage pipeline.
    """
    tokenizer = RegexTokenizer(
        inputCol="body", outputCol="raw_tokens",
        pattern=r"\W+", toLowercase=True, minTokenLength=2,
    )

    remover = StopWordsRemover(
        inputCol="raw_tokens", outputCol="tokens",
        stopWords=stop_words,
    )

    vectorizer = CountVectorizer(
        inputCol="tokens", outputCol="features",
        vocabSize=VOCAB_SIZE,
        minDF=MIN_DF,
        maxDF=MAX_DF,
    )

    return Pipeline(stages=[tokenizer, remover, vectorizer])


def fit_preprocessing(corpus: DataFrame, pipeline: Pipeline):
    """
    run prepro and cache
    """
    prep_model = pipeline.fit(corpus)
    features_df = prep_model.transform(corpus).cache()
    return prep_model, features_df


def fit_lda(features_df: DataFrame, k: int) -> "LDAModel":
    """
```

```
    fit LDA using 20 topics
    """
    lda = LDA(featuresCol="features", k=k, maxIter=LDA_MAX_ITER, seed=SEED)
    return lda.fit(features_df)


def attach_topic_ids(lda_model, features_df: DataFrame) -> DataFrame:
    """
    attach the LDA results back to the df
    """
    return (
        lda_model.transform(features_df)
        .withColumn("topic_array", vector_to_array("topicDistribution"))
        .withColumn("topic_id", expr("array_position(topic_array, array_max(
            topic_array)) - 1"))
    )


def compute_top_terms(lda_model, vocab: list) -> dict:
    """
    get the top terms for each topic
    """
    rows = lda_model.describeTopics(maxTermsPerTopic=TOP_TERMS_N).collect()
    return {row.topic: [vocab[i] for i in row.termIndices] for row in rows}


def compute_top_bylines(classified: DataFrame) -> dict:
    """
    get the top bylines for each topic
    """
    w = Window.partitionBy("topic_id").orderBy(desc("count"))
    rows = (
        classified.filter(col("bylines").isNotNull())
        .groupBy("topic_id", "bylines").count()
        .withColumn("rank", row_number().over(w))
        .filter(col("rank") <= TOP_BYLINES_N)
        .orderBy("topic_id", "rank")
        .collect()
    )

    top_bylines = {}
    for row in rows:
        top_bylines.setdefault(row.topic_id, []).append(row.bylines)
    return top_bylines


def write_topic_terms_csv(top_terms: dict, top_bylines: dict, path: str) -> None
    :
    """
    write out out to a csv so we can manually tag the topics
    """
    rows = []
    for tid in sorted(top_terms.keys()):
        rows.append({
            "topic_id":    tid,
            "top_terms":   " ".join(top_terms.get(tid, [])),
            "top_bylines": "|".join(top_bylines.get(tid, [])),
            "label":       "",
        })
    pd.DataFrame(rows).to_csv(path, index=False)


def drop_lda_intermediates(df: DataFrame) -> DataFrame:
```

```
    """
    clean up
    """
    return df.drop("raw_tokens", "tokens", "features", "topic_array")


def write_intermediate(df: DataFrame, path: str) -> None:
    """
    write out the corpus with the topic labels etc so we dont need to re-run
    """
    df.write.parquet(path, mode="overwrite")


def main() -> None:
    spark = (
        SparkSession.builder
        .appName("FB_API_topics")
        .config("spark.sql.parquet.output.committer.class", "org.apache.parquet.
            hadoop.ParquetOutputCommitter")
        .config("mapreduce.fileoutputcommitter.algorithm.version", "2")
        .getOrCreate()
    )

    df = load_v2(spark, V2_PATH)
    corpus = filter_to_residual_subset(df, COMMERCIAL_BYLINES)
    corpus = extract_body_text(corpus)

    stop_words = StopWordsRemover.loadDefaultStopWords("english") +
        DOMAIN_STOP_WORDS
    pipeline = build_preprocessing_pipeline(stop_words)
    prep_model, features_df = fit_preprocessing(corpus, pipeline)

    lda_model = fit_lda(features_df, K)
    classified = attach_topic_ids(lda_model, features_df)

    vocab = prep_model.stages[-1].vocabulary
    top_terms = compute_top_terms(lda_model, vocab)
    top_bylines = compute_top_bylines(classified)

    write_topic_terms_csv(top_terms, top_bylines, TOPIC_TERMS_CSV)
    classified = drop_lda_intermediates(classified)
    write_intermediate(classified, INTERMEDIATE_PATH)

    spark.stop()


if __name__ == "__main__":
    main()
```

Listing 4: 04_topic_join.py

```
import pandas as pd
from pyspark.sql import SparkSession, DataFrame
from pyspark.sql.functions import broadcast, col


V2_PATH = "/user/s3348393/main/preprocessing/v2/parquet"
INTERMEDIATE_PATH = "/user/s3348393/main/preprocessing/v3/intermediate_parquet"
V3_PATH = "/user/s3348393/main/preprocessing/v3/parquet"
TOPIC_LABELS_CSV = "../data/topic_labels.csv"


def load_v2_unique(spark: SparkSession, path: str) -> DataFrame:
    """
```

```
    load v2 parquet and keep one row per ad.
    """
    return spark.read.parquet(path).filter(col("ad_seq_no") == 1)


def load_intermediate(spark: SparkSession, path: str) -> DataFrame:
    """
    load nb 03's intermediate parquet, keep just the columns needed for the
    join + downstream analysis. id is the join key.
    """
    return spark.read.parquet(path).select("id", "topic_id", "topicDistribution"
        , "body")


def load_topic_labels(spark: SparkSession, path: str) -> DataFrame:
    """
    load the hand-edited topic labels csv. csv is on the edge node, not
    HDFS, so we go via pandas first to avoid spark trying to read it from
    HDFS. rename label to topic_label so it doesnt collide with anything.
    """
    labels_pdf = pd.read_csv(path)[["topic_id", "label", "category"]]
    return spark.createDataFrame(labels_pdf).withColumnRenamed("label", "
        topic_label")


def join_topic_labels(v2: DataFrame, intermediate: DataFrame, labels: DataFrame)
    -> DataFrame:
    """
    three way left join. v2 + LDA columns + topic labels. every v2 ad gets
    the LDA columns if it was in the residual corpus, otherwise null. the
    labels table is tiny (25 rows) so we broadcast it.
    """
    return (
        v2.join(intermediate, "id", "left")
          .join(broadcast(labels), "topic_id", "left")
    )


def filter_political_corpus(df: DataFrame) -> DataFrame:
    """
    keep registered party advertising (candidate, party_org) plus residual
    ads that came out of the LDA labelling with a category. drops
    government (out of scope), byline-tagged commercial (Shell etc.), and
    residual ads that didnt make it into the LDA corpus.
    """
    return df.filter(
        col("match_type").isin("candidate", "party_org")
        | (col("match_type").isNull() & col("category").isNotNull())
    )


def drop_intermediates(df: DataFrame) -> DataFrame:
    """
    drop body and topicDistribution before parquet write. body is big text
    and we have the structured columns we need; topicDistribution is the
    length-k probability vector and topic_id (argmax) is what downstream
    uses.
    """
    return df.drop("body", "topicDistribution")


def write_v3(df: DataFrame, path: str) -> None:
    """
```

```
    write v3 partitioned by category so nb 05 can read subsets selectively.
    """
    df.write.partitionBy("category").parquet(path, mode="overwrite")


def main() -> None:
    spark = (
        SparkSession.builder
        .appName("FB_API_v3_build")
        .config("spark.sql.parquet.output.committer.class", "org.apache.parquet.
            hadoop.ParquetOutputCommitter")
        .config("mapreduce.fileoutputcommitter.algorithm.version", "2")
        .getOrCreate()
    )

    v2 = load_v2_unique(spark, V2_PATH)
    intermediate = load_intermediate(spark, INTERMEDIATE_PATH)
    labels = load_topic_labels(spark, TOPIC_LABELS_CSV)

    classified = join_topic_labels(v2, intermediate, labels)
    v3 = filter_political_corpus(classified)
    v3 = drop_intermediates(v3)

    write_v3(v3, V3_PATH)

    spark.stop()


if __name__ == "__main__":
    main()
```

Notebooks

01_5_senate_party_codes.ipynb

the 2022 senate csv has the same columns as the house one, but PartyAB is blank. fill it in from PartyNm using data/aec_parties.csv (canonical names + aliases) and write the csv back so nb 02 doesnt have to deal with any of this. plain pandas, no spark.

load the inputs and take a peek.

In [1]:
```
import pandas as pd

SENATE_CSV  = '../data/2022_senate_candidates.csv'
PARTIES_CSV = '../data/aec_parties.csv'

aec_parties = pd.read_csv(PARTIES_CSV, header=0)
senate      = pd.read_csv(SENATE_CSV,  header=0)

print('AEC parties:', len(aec_parties))
print('Senate rows:', len(senate))
senate.head()
```

```
AEC parties: 38
Senate rows: 421
```

Out[1]:

	state	PartyAB	PartyNm	Surname	GivenNm
0	ACT	NaN	Australian Labor Party	GALLAGHER	Katy
1	ACT	NaN	Australian Labor Party	NORTHAM	Maddy
2	ACT	NaN	Sustainable Australia Party - Stop Overdevelop...	ANGEL	Joy
3	ACT	NaN	Sustainable Australia Party - Stop Overdevelop...	HAYDON	John
4	ACT	NaN	United Australia Party	SAVOULIDIS	James

build a name → party_ab lookup. every party_name and every alias from the pipe-separated aliases column maps to the row's party_ab. lowercased so we can match case-insensitively.

In [2]:
```
name_to_party_ab = {}
for _, row in aec_parties.iterrows():
    ab = row['party_ab']
    names = [row['party_name']]
    if isinstance(row['aliases'], str) and row['aliases'].strip():
        names.extend(row['aliases'].split('|'))
    for n in names:
        if isinstance(n, str) and n.strip():
            name_to_party_ab[n.strip().lower()] = ab

print('Lookup entries:', len(name_to_party_ab))
```

```
Lookup entries: 55
```

now resolve PartyAB for each senate row. rules in order. blank or NaN PartyNm goes to IND. exact (case-insensitive) match in the lookup goes to that party's PartyAb. personal-name

ticket (no "party" in the name) goes to IND. otherwise leave it blank and dump it in the unmapped list for triage.

In [3]:
```
def resolve_party_ab(party_nm):
    if not isinstance(party_nm, str) or not party_nm.strip():
        return 'IND'
    key = party_nm.strip().lower()
    if key in name_to_party_ab:
        return name_to_party_ab[key]
    if 'party' not in key:
        return 'IND'
    return None  # genuinely unmapped

senate['PartyAB'] = senate['PartyNm'].apply(resolve_party_ab)

unmapped = senate.loc[senate['PartyAB'].isna(), 'PartyNm'].dropna().unique()
print('Filled rows:', senate['PartyAB'].notna().sum())
print('Unmapped rows:', senate['PartyAB'].isna().sum())
if len(unmapped):
    print('\nUnmapped PartyNm values (triage needed — add to aec_parties.csv aliase
    for v in sorted(unmapped):
        print(' -', v)
```

```
Filled rows: 421
Unmapped rows: 0
```

quick sanity check on the distribution.

write the senate csv back with PartyAB filled in. column order's preserved so nb 02 can just read it as-is.

In [6]:
```
senate.to_csv(SENATE_CSV, index=False)
print('Wrote:', SENATE_CSV)
```

```
Wrote: ../data/2022_senate_candidates.csv
```

01_data_loading.ipynb

this is the exploration side of nb 01. reads the raw json straight out of hdfs and walks through what's in there - the two date formats, the schema drift across api versions, why we have to collapse some paired singular/plural columns. the actual pipeline (window filter, flattening the spend/impressions/audience structs, snapshot sequence number, parquet write) is over in 01_data_loading.py - nothing gets written from this notebook.

In [1]:
```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName('FB_API_topics') \
    .config('spark.sql.parquet.output.committer.class',
            'org.apache.parquet.hadoop.ParquetOutputCommitter') \
    .config('mapreduce.fileoutputcommitter.algorithm.version', '2') \
    .getOrCreate()
print("Master:", spark.sparkContext.master)
```

```
Master: yarn
```

```
26/05/17 01:00:05 WARN SparkSession: Using an existing Spark session; only runtime S
QL configurations will take effect.
```

Read all JSON files from HDFS into a df. check out the schema. lets incldue the filename in the table, so we can extract snapshot date as well

In [2]:
```
from pyspark.sql.functions import input_file_name, regexp_extract, to_date,format_s
df = spark.read.json("/data/ProjectDatasetFacebookAU/")
df = df.withColumn("filename", input_file_name())

# extract YYYYMMDD from filename. note there are both 7 and 8 digit dates - regex u

date_re = r"-(\d{4})(\d{1,2})(\d{2})-"

yyyy = regexp_extract("filename", date_re, 1).cast("int")
mm   = regexp_extract("filename", date_re, 2).cast("int")
dd   = regexp_extract("filename", date_re, 3).cast("int")

date_str = format_string("%d-%02d-%02d", yyyy, mm, dd)

df = df.withColumn("snapshot_date", to_date(date_str, "yyyy-MM-dd"))

print(df.schema)
```

```
[Stage 1:=============================================================>(905 + 3) / 908]
```

```
StructType([StructField('ad_creation_time', StringType(), True), StructField('ad_cre
ative_bodies', ArrayType(StringType(), True), True), StructField('ad_creative_body',
StringType(), True), StructField('ad_creative_link_caption', StringType(), True), St
ructField('ad_creative_link_captions', ArrayType(StringType(), True), True), StructF
ield('ad_creative_link_description', StringType(), True), StructField('ad_creative_l
ink_descriptions', ArrayType(StringType(), True), True), StructField('ad_creative_li
nk_title', StringType(), True), StructField('ad_creative_link_titles', ArrayType(Str
ingType(), True), True), StructField('ad_delivery_start_time', StringType(), True),
StructField('ad_delivery_stop_time', StringType(), True), StructField('ad_snapshot_u
rl', StringType(), True), StructField('bylines', StringType(), True), StructField('c
urrency', StringType(), True), StructField('delivery_by_region', ArrayType(StructTyp
e([StructField('percentage', StringType(), True), StructField('region', StringType
(), True)]), True), True), StructField('demographic_distribution', ArrayType(StructT
ype([StructField('age', StringType(), True), StructField('gender', StringType(), Tru
e), StructField('percentage', StringType(), True)]), True), True), StructField('esti
mated_audience_size', StructType([StructField('lower_bound', StringType(), True), St
ructField('upper_bound', StringType(), True)]), True), StructField('funding_entity',
StringType(), True), StructField('id', StringType(), True), StructField('impression
s', StructType([StructField('lower_bound', StringType(), True), StructField('upper_b
ound', StringType(), True)]), True), StructField('languages', ArrayType(StringType
(), True), True), StructField('page_id', StringType(), True), StructField('page_nam
e', StringType(), True), StructField('publisher_platforms', ArrayType(StringType(),
True), True), StructField('region_distribution', ArrayType(StructType([StructField
('percentage', StringType(), True), StructField('region', StringType(), True)]), Tru
e), True), StructField('spend', StructType([StructField('lower_bound', StringType(),
True), StructField('upper_bound', StringType(), True)]), True), StructField('filenam
e', StringType(), False), StructField('snapshot_date', DateType(), True)])
```

cast ad_creation_time to date. ad_creation_time is loaded as a string. add a typed ad_creation_datetime column alongside the original.

turns out there's a more than one date format - lets do a check.

In [3]:
```
from pyspark.sql.functions import length, count, first

df.groupBy(length("ad_creation_time").alias("len")) \
  .agg(count("*").alias("count"), first("ad_creation_time").alias("example")) \
  .orderBy("len") \
  .show(truncate=False)
```

```
[Stage 2:=====================================================>(898 + 10) / 908]
+---+--------+------------------------+
|len|count   |example                 |
+---+--------+------------------------+
|10 |39505667|2021-11-12              |
|24 |78472   |2020-08-28T12:54:44+0000|
+---+--------+------------------------+
```

given most of the data has the yyyy-mm-dd format, it's probably safe enough to just truncate everything to 10 characters to clean up the 80,000 odd ones

In [4]:
```
from pyspark.sql.functions import to_date, substring

for src in ["ad_creation_time", "ad_delivery_start_time", "ad_delivery_stop_time"]:
    dst = src.replace("_time", "_date")
    df = df.withColumn(dst, to_date(substring(src, 1, 10), "yyyy-MM-dd"))

df.select("ad_creation_time", "ad_creation_date").show(5)
```

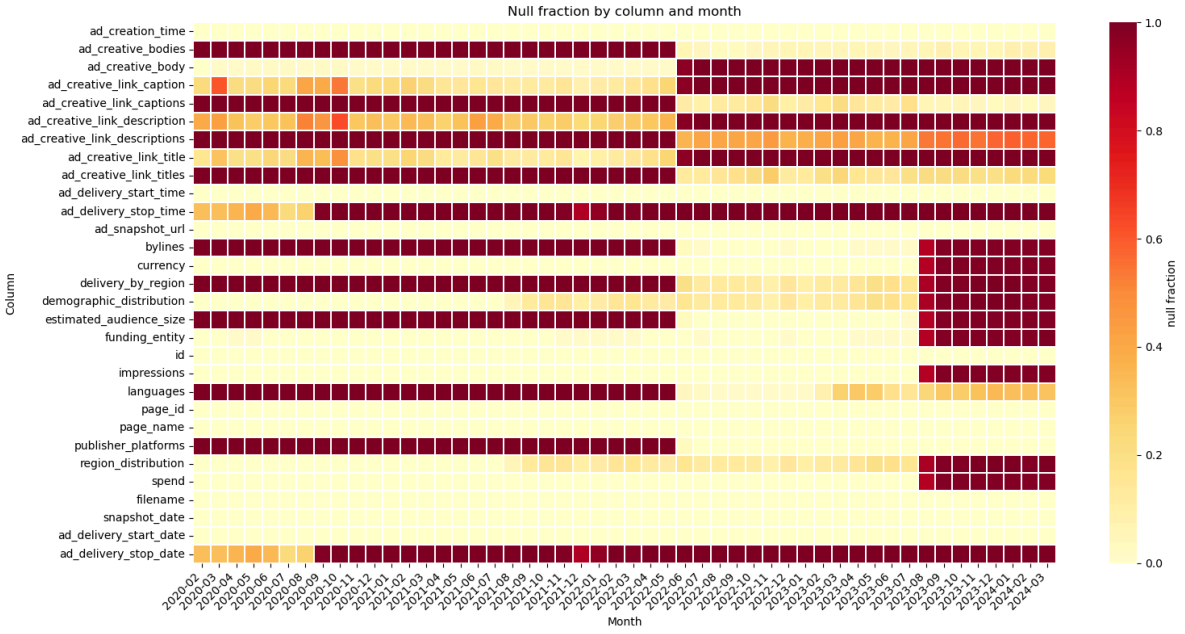
```
+----------------+----------------+
|ad_creation_time|ad_creation_date|
+----------------+----------------+
|      2021-11-21|      2021-11-21|
|      2021-12-02|      2021-12-02|
|      2021-12-02|      2021-12-02|
|      2021-12-02|      2021-12-02|
|      2021-12-02|      2021-12-02|
+----------------+----------------+
only showing top 5 rows
```

schema cleanup. there seems to be some adjacent schema entries, for example ad_creative_bodies as an array of strings alongside ad_creative_body as a singular string. what i'm going to try and do is groupBy month, then count the nulls in each column, then return the columns, months and counts to pandas/seaborn for plotting. this should let us visualise the schema changes over time.

In [5]:
```
from pyspark.sql.functions import col, sum as spark_sum, count, date_trunc
import seaborn as sns
import matplotlib.pyplot as plt

cols = [c for c in df.columns if c != "ad_creation_date"]
agg_exprs = [count("*").alias("_total")] + [spark_sum(col(c).isNull().cast("int")).

null_by_month = df \
    .withColumn("month", date_trunc("month", "snapshot_date")) \
    .groupBy("month") \
    .agg(*agg_exprs) \
    .orderBy("month")

pdf = null_by_month.toPandas()

months = pdf["month"]
totals = pdf["_total"]
frac = pdf[cols].divide(totals, axis=0)
frac.index = months
plt.figure(figsize=(16, 8))
frac.index = frac.index.strftime("%Y-%m")
sns.heatmap(
    frac.T,
    cmap="YlOrRd",
    vmin=0, vmax=1,
    linewidths=0.3,
    cbar_kws={"label": "null fraction"}
)
```

```
plt.title("Null fraction by column and month")
plt.xlabel("Month")
plt.ylabel("Column")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

```
26/05/17 01:01:30 WARN SparkStringUtils: Truncated the string representation of a pl
an since it was too large. This behavior can be adjusted by setting 'spark.sql.debu
g.maxToStringFields'.
```



so some things to note from comparing the heat map to the release notes:

there was a significant API update in June 2022, which loosely correlates with the release of Graph API version v13.0 in Feb 2022. The most noticable changes here where multiple changes from singular to arrays. There are also some renamed fields and some new fields.

looks like things broke singificantly in August 2023.

in 2022 some columns where changed from singular to arrays. choices are a) keep the first element from the array, drop the rest and keep everything as singular, or b) turn the singulars into single element arrays. I've gone with b) as it doesnt blow away the data.

we should also collapse a few more first: bylines and funding entiries, and delivery_by_region and region_distribution.

In [6]:
```
df.createOrReplaceTempView("ads")

df = spark.sql("""
    SELECT
        id,
        page_id,
        page_name,
```

```
        snapshot_date,
        ad_creation_date,
        ad_delivery_start_date,
        ad_delivery_stop_date,
        coalesce(ad_creative_bodies,             array(ad_creative_body))
        coalesce(ad_creative_link_captions,      array(ad_creative_link_caption))
        coalesce(ad_creative_link_descriptions, array(ad_creative_link_description)
        coalesce(ad_creative_link_titles,        array(ad_creative_link_title))
        impressions,
        spend,
        estimated_audience_size,
        currency,
        languages,
        publisher_platforms,
        demographic_distribution,
        coalesce(delivery_by_region, region_distribution) AS delivery_by_region,
        ad_snapshot_url,
        coalesce(bylines, funding_entity) AS bylines
    FROM ads
""")
```

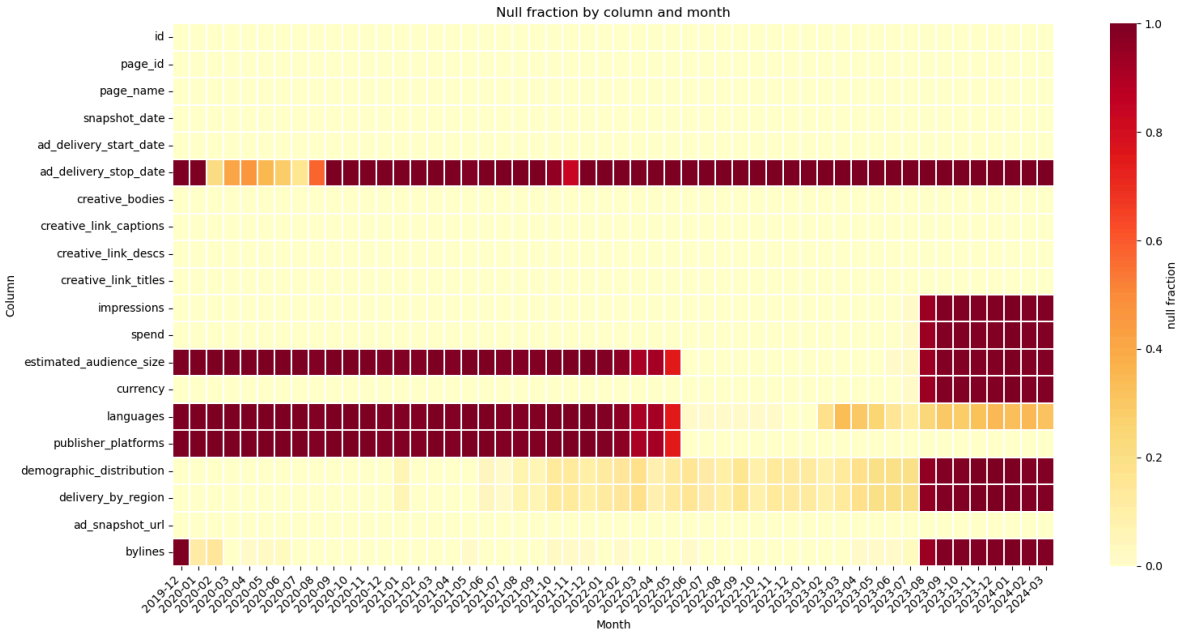
and lets regroup and replot.

do i have to do this again from scratch? it only takes a minute at this point, so i've just cut and paste for now

In [7]:
```
from pyspark.sql.functions import col, sum as spark_sum, count, date_trunc
import seaborn as sns
import matplotlib.pyplot as plt

cols = [c for c in df.columns if c != "ad_creation_date"]
print(cols)
agg_exprs = [count("*").alias("_total")] + [spark_sum(col(c).isNull().cast("int")).

null_by_month = df \
    .withColumn("month", date_trunc("month", "ad_creation_date")) \
    .groupBy("month") \
    .agg(*agg_exprs) \
    .orderBy("month")

pdf = null_by_month.toPandas()

months = pdf["month"]
totals = pdf["_total"]
frac = pdf[cols].divide(totals, axis=0)
frac.index = months
plt.figure(figsize=(16, 8))
frac.index = frac.index.strftime("%Y-%m")
sns.heatmap(
    frac.T,
    cmap="YlOrRd",
    vmin=0, vmax=1,
    linewidths=0.3,
    cbar_kws={"label": "null fraction"}
)
```

```
plt.title("Null fraction by column and month")
plt.xlabel("Month")
plt.ylabel("Column")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

```
['id', 'page_id', 'page_name', 'snapshot_date', 'ad_delivery_start_date', 'ad_delive
ry_stop_date', 'creative_bodies', 'creative_link_captions', 'creative_link_descs',
'creative_link_titles', 'impressions', 'spend', 'estimated_audience_size', 'currenc
y', 'languages', 'publisher_platforms', 'demographic_distribution', 'delivery_by_reg
ion', 'ad_snapshot_url', 'bylines']
```



we'll use that as the schema moving forward. i'll put the actual ingestion etc into a python script

02_preprocessing.ipynb

spin up spark.

In [1]:
```
from pyspark.sql import SparkSession
import pandas as pd
import re

# Override the parquet output committer — the cluster default points at an EMR
# class whose JAR isn't on the classpath, which breaks .write.parquet otherwise.
spark = SparkSession.builder \
    .appName('FB_API_party_match') \
    .config('spark.sql.parquet.output.committer.class',
            'org.apache.parquet.hadoop.ParquetOutputCommitter') \
    .config('mapreduce.fileoutputcommitter.algorithm.version', '2') \
    .getOrCreate()

print('Master:', spark.sparkContext.master)
print('Spark version:', spark.version)
```

```
Master: yarn
Spark version: 3.5.0
```

```
26/05/17 02:16:00 WARN SparkSession: Using an existing Spark session; only runtime S
QL configurations will take effect.
```

paths.

In [2]:
```
IN_PATH        = '/user/s3348393/main/preprocessing/v1/parquet'
HOUSE_CSV      = '../data/2022_election_candidates.csv'
SENATE_CSV     = '../data/2022_senate_candidates.csv'
PARTIES_CSV    = '../data/aec_parties.csv'
```

load the v1 parquet and peek at the schema.

In [3]:
```
from pyspark.sql.functions import col

df = spark.read.parquet(IN_PATH)
print(f'Total rows: {df.count():>10,}')
print(f'Unique ads: {df.filter(col("ad_seq_no") == 1).count():>10,}')
df.printSchema()
```

```
Total rows:  5,796,491
Unique ads:    151,578
root
 |-- id: string (nullable = true)
 |-- page_id: string (nullable = true)
 |-- page_name: string (nullable = true)
 |-- snapshot_date: date (nullable = true)
 |-- ad_creation_date: date (nullable = true)
 |-- ad_delivery_start_date: date (nullable = true)
 |-- ad_delivery_stop_date: date (nullable = true)
 |-- creative_bodies: array (nullable = true)
 |    |-- element: string (containsNull = true)
 |-- creative_link_captions: array (nullable = true)
 |    |-- element: string (containsNull = true)
 |-- creative_link_descs: array (nullable = true)
 |    |-- element: string (containsNull = true)
 |-- creative_link_titles: array (nullable = true)
 |    |-- element: string (containsNull = true)
 |-- spend_lower_bound: long (nullable = true)
 |-- spend_upper_bound: long (nullable = true)
 |-- spend_mid: double (nullable = true)
 |-- impressions_lower_bound: long (nullable = true)
 |-- impressions_upper_bound: long (nullable = true)
 |-- impressions_mid: double (nullable = true)
 |-- audience_size_lower_bound: long (nullable = true)
 |-- audience_size_upper_bound: long (nullable = true)
 |-- audience_size_mid: double (nullable = true)
 |-- currency: string (nullable = true)
 |-- languages: array (nullable = true)
 |    |-- element: string (containsNull = true)
 |-- publisher_platforms: array (nullable = true)
 |    |-- element: string (containsNull = true)
 |-- demographic_distribution: array (nullable = true)
 |    |-- element: struct (containsNull = true)
 |    |    |-- age: string (nullable = true)
 |    |    |-- gender: string (nullable = true)
 |    |    |-- percentage: string (nullable = true)
 |-- delivery_by_region: array (nullable = true)
 |    |-- element: struct (containsNull = true)
 |    |    |-- percentage: string (nullable = true)
 |    |    |-- region: string (nullable = true)
 |-- ad_snapshot_url: string (nullable = true)
 |-- bylines: string (nullable = true)
 |-- ad_seq_no: integer (nullable = true)
```

load the 2022 candidates — house + senate. the house csv (DivisionNm, PartyAb, PartyNm, Surname, GivenNm) and senate csv (state, PartyAB, PartyNm, Surname, GivenNm — PartyAB filled in by 01_5_senate_party_codes.ipynb) have the columns we need. rename the senate PartyAB to PartyAb and concat into one df so the candidate loop in cell 6 can treat them as a single source.

In [4]:
```
house  = pd.read_csv(HOUSE_CSV,  header=0)
senate = pd.read_csv(SENATE_CSV, header=0).rename(columns={'PartyAB': 'PartyAb'})
```

```
candidates = pd.concat(
    [house[['PartyAb', 'PartyNm', 'Surname', 'GivenNm']],
     senate[['PartyAb', 'PartyNm', 'Surname', 'GivenNm']]],
    ignore_index=True,
)

print('House candidates: ', len(house))
print('Senate candidates:', len(senate))
print('Combined:         ', len(candidates))
candidates.head()
```

```
House candidates:  1203
Senate candidates: 421
Combined:          1624
```

Out[4]:

	PartyAb	PartyNm	Surname	GivenNm
0	ALP	Australian Labor Party	ABBOTT	Martyn
1	GVIC	The Greens	ABBOUD	Natalie
2	GRN	The Greens (WA)	ABDUL RAZAK	Adam
3	LDP	Liberal Democrats	ABELMAN	Michael
4	ALP	Australian Labor Party	ABHIMANYU KUMAR	NaN

tokenise page_name and bylines separately. spark mllib's RegexTokenizer (splits on \W+, lowercases) then StopWordsRemover with the english defaults plus a few political honorifics. we keep the fields separate so a given name in page_name doesnt cross with a surname in bylines and spuriously match a candidate.

coalesce(col, lit('')) because RegexTokenizer blows up on nulls.

In [5]:
```
from pyspark.sql.functions import coalesce, col, lit
from pyspark.ml.feature import RegexTokenizer, StopWordsRemover

df = df.withColumn('page_name_safe', coalesce(col('page_name'), lit(''))) \
       .withColumn('bylines_safe',   coalesce(col('bylines'),   lit('')))

honorifics = ['mp', 'hon', 'dr', 'mr', 'mrs', 'ms', 'sen', 'senator', 'rt', 'author
stopwords = StopWordsRemover.loadDefaultStopWords('english') + honorifics

for src, tok_col, term_col in [
    ('page_name_safe', 'page_name_tokens', 'page_name_terms'),
    ('bylines_safe',   'bylines_tokens',   'bylines_terms'),
]:
    df = RegexTokenizer(inputCol=src, outputCol=tok_col, pattern=r'\W+', toLowercas
    df = StopWordsRemover(inputCol=tok_col, outputCol=term_col, stopWords=stopwords

df.select('page_name', 'page_name_terms', 'bylines', 'bylines_terms').show(5, trunc
```

```
[Stage 7:>                                                          (0 + 1) / 1]
```

```
+--------------------------------------------------+---------------------------------
--------------+----------------+----------------+
|                                         page_name|                                p
age_name_terms|         bylines|   bylines_terms|
+--------------------------------------------------+---------------------------------
--------------+----------------+----------------+
|Rob Mitchell MP - Federal Labor Member for McEwen|[rob, mitchell, federal, labor, m
ember, mcewen]|Rob Mitchell MP|[rob, mitchell]|
|Rob Mitchell MP - Federal Labor Member for McEwen|[rob, mitchell, federal, labor, m
ember, mcewen]|Rob Mitchell MP|[rob, mitchell]|
|Rob Mitchell MP - Federal Labor Member for McEwen|[rob, mitchell, federal, labor, m
ember, mcewen]|Rob Mitchell MP|[rob, mitchell]|
|Rob Mitchell MP - Federal Labor Member for McEwen|[rob, mitchell, federal, labor, m
ember, mcewen]|Rob Mitchell MP|[rob, mitchell]|
|Rob Mitchell MP - Federal Labor Member for McEwen|[rob, mitchell, federal, labor, m
ember, mcewen]|Rob Mitchell MP|[rob, mitchell]|
+--------------------------------------------------+---------------------------------
--------------+----------------+----------------+
only showing top 5 rows
```

build the candidate match index. for each candidate the required tokens are the first given-name token plus all surname tokens — e.g. Adam ABDUL RAZAK becomes {adam, abdul, razak}. token-set subset matching is order-insensitive so "Adam Abdul Razak" and "Abdul Razak, Adam" both match.

we'd rather not scan all ~1,500 candidates per ad — we'll just check candidates whose key token appears in the ad's token list.

In [6]:

```
def split_tokens(s):
    if not isinstance(s, str):
        return []
    return [t for t in re.split(r'\W+', s.lower()) if t]

candidate_list = []
for _, row in candidates.iterrows():
    surname_toks = split_tokens(row['Surname'])
    if not surname_toks:
        continue
    given_toks = split_tokens(row['GivenNm'])
    required = frozenset(surname_toks + given_toks[:1])
    party = row['PartyAb']
    if not isinstance(party, str):
        continue
    candidate_list.append((required, party))

print('Candidates indexed:', len(candidate_list))
```

```
Candidates indexed: 1623
```

classify each ad. two new columns.

political_party is the AEC party code (ALP, LP, etc) when the ad's from a registered-party candidate or party office. null otherwise.

match_type tells us how we got there. candidate means a 2022 candidate's name (house or senate) shows up in page_name or bylines. party_org means a party-name signature from aec_parties.csv matches the byline (e.g. byline = "Australian Labor Party"). government means the byline is a govt dept or agency (e.g. "Department of Social Services", "Australian Electoral Commission", "Australian Cyber Security Centre"). commercial means a known commercial advertiser with election-aligned spend timing but non-political content (e.g. "Shell") — gets filtered out downstream. null means none of the above.

priority is candidate then party_org then government then commercial. if more than one candidate matches and they disagree on party, we drop the row and let it fall through to the byline checks. filter on match_type IS NULL AND bylines IS NOT NULL to surface the leftover political-looking bylines (NGOs, unions, advocacy orgs, third-party advertisers) for triage.

In [7]:
```
from pyspark.sql.functions import pandas_udf
from pyspark.sql.types import StringType, StructType, StructField

# Party-name byline signatures: rows where byline_tokens is non-empty in aec_partie
# preserving CSV order so specific signatures (e.g. {liberal, national}) match befo
# single-token signatures ({liberal}).
aec_parties = pd.read_csv(PARTIES_CSV, header=0)

party_byline_signatures = []
for _, row in aec_parties.iterrows():
    bt = row['byline_tokens']
    if not isinstance(bt, str) or not bt.strip():
        continue
    sig = frozenset(t.strip() for t in bt.split('+') if t.strip())
    party_byline_signatures.append((sig, row['party_ab']))

# Government byline signatures. Narrow enough to avoid private-sector collisions.
# {commonwealth} was dropped — too high a false-positive risk against Commonwealth
GOV_BYLINE_SIGNATURES = [
    frozenset({'department'}),                          # "Department of X" — cover
    frozenset({'australian', 'government'}),            # "Australian Government" p
    frozenset({'australian', 'electoral', 'commission'}),
    frozenset({'australian', 'cyber', 'security'}),     # Australian Cyber Security
]

#shell is a pain in the butt - remove it as well. i checked and it's a greenwashing
COMMERCIAL_BYLINE_SIGNATURES = [
    frozenset({'shell'}),  # Shell (energy company)
]

print('Party-org byline signatures: ', len(party_byline_signatures))
print('Government byline signatures:', len(GOV_BYLINE_SIGNATURES))
print('Commercial byline signatures:', len(COMMERCIAL_BYLINE_SIGNATURES))

#replaced the python udf with a pandas udf
result_schema = StructType([
```

```
    StructField('political_party', StringType(), True),
    StructField('match_type',      StringType(), True),
])

def _classify_one(page_tokens, bylines_tokens):
    # pandas_udf hands array columns to Python as numpy arrays, not lists.
    # `array or []` would call bool(array) — ambiguous for multi-element arrays.
    # Use explicit None checks to convert to set safely.
    page_set    = set(page_tokens)    if page_tokens    is not None else set()
    bylines_set = set(bylines_tokens) if bylines_tokens is not None else set()

    # 1. Candidate name match (House + Senate)
    parties = set()
    for required, party in candidate_list:
        if required.issubset(page_set) or required.issubset(bylines_set):
            parties.add(party)
    if len(parties) == 1:
        return (next(iter(parties)), 'candidate')
    # Ambiguous (>1 party) or no match — fall through.

    # 2. Party-name byline signature
    for sig, party in party_byline_signatures:
        if sig.issubset(bylines_set):
            return (party, 'party_org')

    # 3. Government byline signature
    for sig in GOV_BYLINE_SIGNATURES:
        if sig.issubset(bylines_set):
            return (None, 'government')

    # 4. Commercial byline signature
    for sig in COMMERCIAL_BYLINE_SIGNATURES:
        if sig.issubset(bylines_set):
            return (None, 'commercial')

    return (None, None)


@pandas_udf(result_schema)
def classify_ad(page_terms: pd.Series, bylines_terms: pd.Series) -> pd.DataFrame:
    results = [_classify_one(p, b) for p, b in zip(page_terms, bylines_terms)]
    return pd.DataFrame(results, columns=['political_party', 'match_type'])


df = df.withColumn('_class', classify_ad('page_name_terms', 'bylines_terms'))
df = df.withColumn('political_party', col('_class.political_party')) \
       .withColumn('match_type',      col('_class.match_type')) \
       .drop('_class')
```

```
Party-org byline signatures:  14
Government byline signatures: 4
Commercial byline signatures: 1
```

sanity check the counts.

In [8]:
```
from pyspark.sql.functions import sum as spark_sum, when

df.groupBy('political_party').agg(spark_sum(lit(1)).alias('total_rows'),spark_sum(w
```

```
[Stage 8:================================================>       (7 + 1) / 8]
+---------------+----------+----------+
|political_party|total_rows|unique_ads|
+---------------+----------+----------+
|NULL           |4548612   |105103    |
|ALP            |487537    |20279     |
|LP             |315668    |11324     |
|GRN            |133851    |4178      |
|IND            |81228     |3171      |
|AJP            |81965     |1872      |
|LNP            |30111     |1325      |
|LDP            |15142     |822       |
|NP             |17902     |726       |
|UAPP           |12194     |597       |
|ON             |12906     |563       |
|GVIC           |14139     |381       |
|AUVA           |15895     |326       |
|JLN            |13064     |182       |
|TNL            |3211      |150       |
|REAS           |2665      |109       |
|KAP            |1805      |109       |
|CYA            |2391      |108       |
|CLP            |2326      |81        |
|SOPA           |510       |30        |
|TLOC           |579       |29        |
|VNS            |505       |23        |
|XEN            |891       |22        |
|DHJP           |577       |20        |
|SAL            |223       |16        |
|DPDA           |179       |12        |
|IMO            |281       |9         |
|GAP            |103       |7         |
|WAP            |8         |2         |
|SPP            |7         |1         |
|AUP            |16        |1         |
+---------------+----------+----------+
```

In [9]:
```
# breakdown by match_type — total rows alongside unique ads
from pyspark.sql.functions import sum as spark_sum, when

df.groupBy('match_type').agg(spark_sum(lit(1)).alias('total_rows'),spark_sum(when(c
```

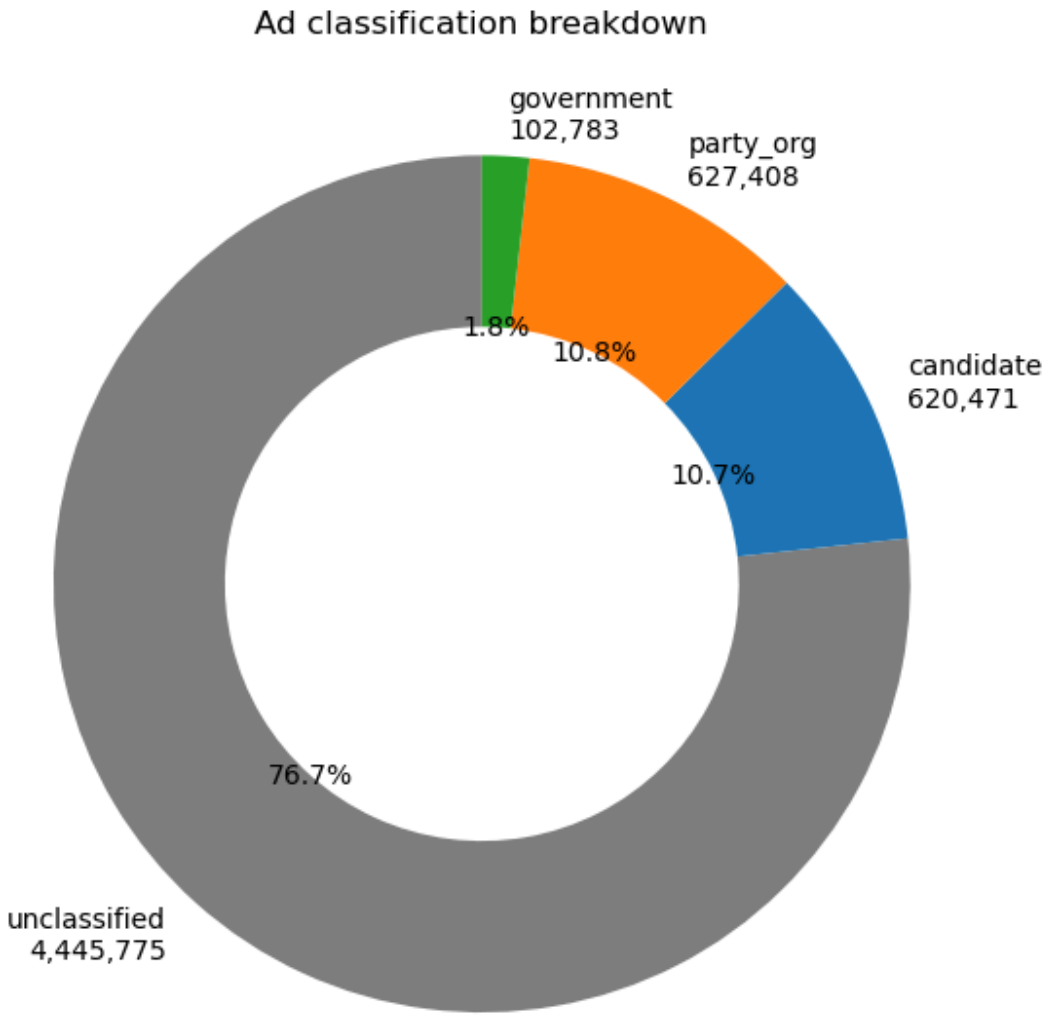
```
[Stage 11:================================================>       (7 + 1) / 8]
```

```
+----------+----------+----------+
|match_type|total_rows|unique_ads|
+----------+----------+----------+
|NULL      |4445775   |103881    |
|candidate |620471    |23327     |
|party_org |627408    |23148     |
|government|102783    |1219      |
|commercial|54        |3         |
+----------+----------+----------+
```

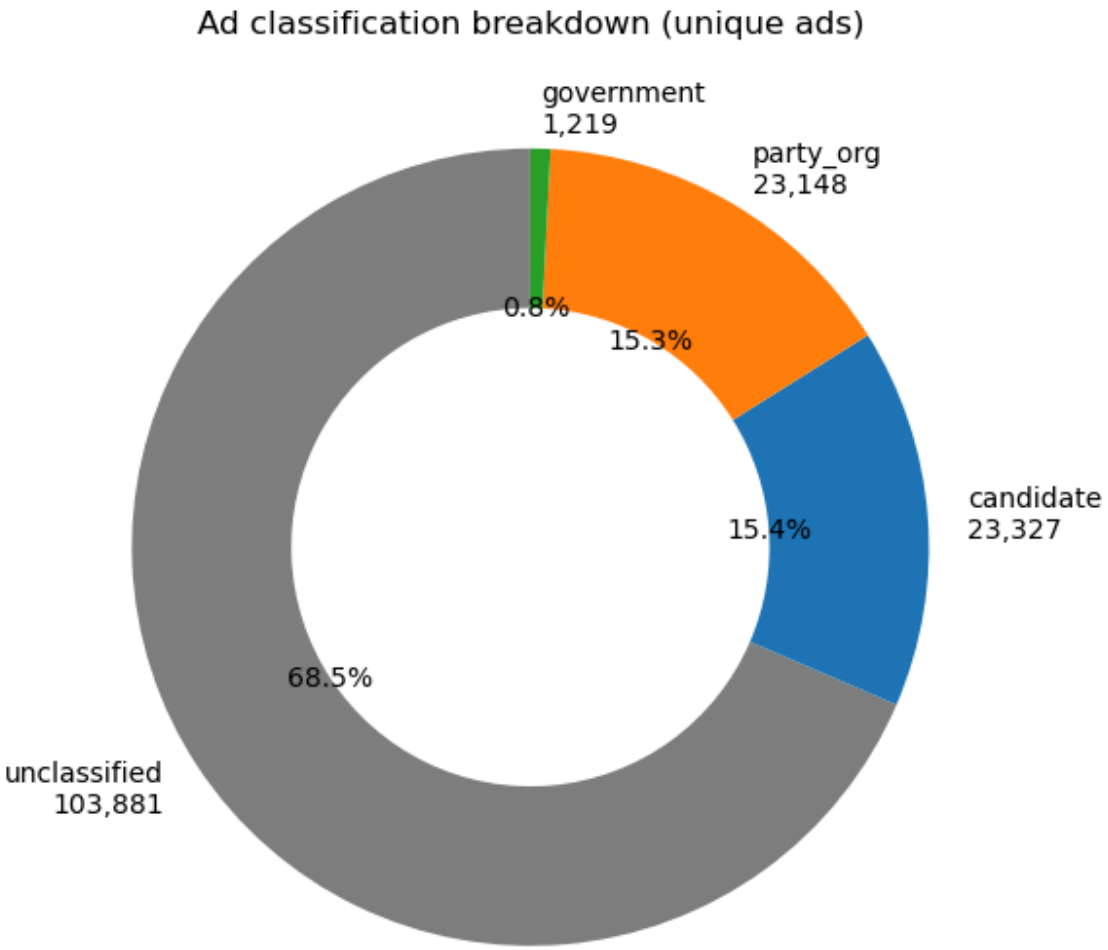
same diagnostics, but plotted for the report. each chart aggregates in spark on the cached df, then materialises a small result to pandas for matplotlib — no full corpus collection.

four views. classification breakdown — donut showing the candidate / party_org / government / unclassified split. top parties — horizontal bar of classified ad counts (nulls excluded). classification source per party — for the top 8 parties, candidate-name match vs party-org byline fallback. weekly spend by classification — the election spike in candidate + party_org against the steady government / unclassified baselines is the temporal story we pick up in nb 05.

In [19]:
```python
import matplotlib.pyplot as plt

# Consistent palette for the five classification categories — reused across
# every chart in this section so they read as one figure family.
MT_COLORS = {
    'unclassified': '#7f7f7f',
    'candidate':    '#1f77b4',
    'party_org':    '#ff7f0e',
    'government':   '#2ca02c',
    }

# Headline classification breakdown — what fraction of ads landed in each bucket.
mt = df.groupBy('match_type').count().toPandas()
mt = mt[mt['match_type'] != "commercial"] #shell has like 3 ads. they're big spend,
mt['match_type'] = mt['match_type'].fillna('unclassified')
mt = mt.set_index('match_type').reindex(MT_COLORS.keys()).dropna()

fig, ax = plt.subplots(figsize=(6, 6))
ax.pie(
    mt['count'],
    labels=[f'{k}\n{int(v):,}' for k, v in mt['count'].items()],
    colors=[MT_COLORS[k] for k in mt.index],
    autopct='%1.1f%%',
    startangle=90,
    wedgeprops=dict(width=0.4),
)
ax.set_title('Ad classification breakdown')
plt.tight_layout()
plt.show()
```

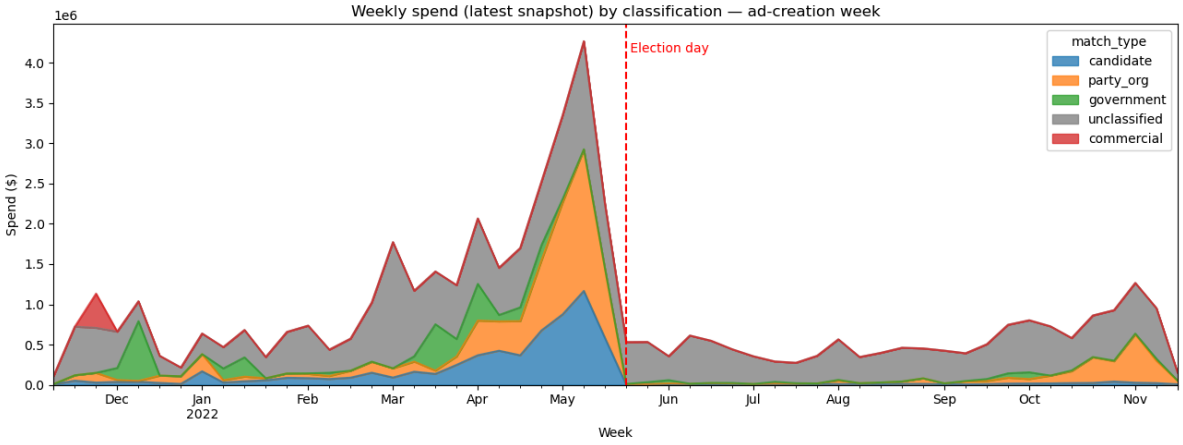


In [20]:
```
# Same breakdown but on *unique* ads — filter to one row per ad (ad_seq_no == 1)
# so the chart reflects ad counts rather than the multi-row v2 representation.
mt_unique = (df.filter(col('ad_seq_no') == 1).groupBy('match_type').count().toPanda
mt_unique = mt_unique[mt_unique['match_type'] != "commercial"] #shell has like 3 ad
mt_unique['match_type'] = mt_unique['match_type'].fillna('unclassified')
mt_unique = mt_unique.set_index('match_type').reindex(MT_COLORS.keys()).dropna()

fig, ax = plt.subplots(figsize=(6, 6))
ax.pie(
    mt_unique['count'],
    labels=[f'{k}\n{int(v):,}' for k, v in mt_unique['count'].items()],
    colors=[MT_COLORS[k] for k in mt_unique.index],
    autopct='%1.1f%%',
    startangle=90,
    wedgeprops=dict(width=0.4),
)
ax.set_title('Ad classification breakdown (unique ads)')
plt.tight_layout()
plt.show()
```



In [14]:
```
# Weekly spend by match_type — temporal preview of the classifier's separation powe
# Aggregate in Spark on the cached df, then materialise to pandas for matplotlib.
from pyspark.sql.functions import date_trunc, sum as spark_sum
import pandas as pd

weekly = (df.filter(col('ad_seq_no') == 1)
            .filter(col('spend_mid').isNotNull())
            .withColumn('week', date_trunc('week', 'ad_creation_date'))
            .groupBy('week', 'match_type')
            .agg(spark_sum('spend_mid').alias('spend'))
            .orderBy('week')
            .toPandas())

weekly['match_type'] = weekly['match_type'].fillna('unclassified')
pivot = weekly.pivot(index='week', columns='match_type', values='spend').fillna(0)
order = [c for c in ['candidate', 'party_org', 'government', 'unclassified', 'comme
pivot = pivot[order]

fig, ax = plt.subplots(figsize=(13, 5))
pivot.plot.area(ax=ax, color=[MT_COLORS[c] for c in pivot.columns], alpha=0.75)
ax.axvline(pd.Timestamp('2022-05-21'), linestyle='--', color='red', linewidth=1.5)
```

```
ax.text(pd.Timestamp('2022-05-22'), ax.get_ylim()[1] * 0.95, ' Election day',
        color='red', va='top')
ax.set_title('Weekly spend (latest snapshot) by classification — ad-creation week')
ax.set_ylabel('Spend ($)')
ax.set_xlabel('Week')
ax.legend(title='match_type', loc='upper right')
plt.tight_layout()
plt.show()
```



In [13]:
```
# Residual third-party political-looking bylines:
# match_type IS NULL excludes candidate, party_org, AND government matches —
# what's left is non-party non-government political-adjacent advertising
# (NGOs, unions, advocacy orgs, industry bodies, individual political advertisers).
from pyspark.sql.functions import sum as spark_sum, when

(df.filter(col('match_type').isNull() & col('bylines').isNotNull())
   .groupBy('bylines')
   .agg(
       spark_sum(lit(1)).alias('total_rows'),
       spark_sum(when(col('ad_seq_no') == 1, 1).otherwise(0)).alias('unique_ads'),
   )
   .orderBy(col('unique_ads').desc())
   .show(30, truncate=80))
```

```
[Stage 28:================================================>       (7 + 1) / 8]
```

```
+-----------------------------------+----------+----------+
|                            bylines|total_rows|unique_ads|
+-----------------------------------+----------+----------+
|          Greenpeace Australia Pacific|    546415|      9661|
|                                Access|     91188|      6787|
|                          Private Media|     51370|      3021|
|       Amnesty International Australia|     71893|      2262|
|    Australian Conservation Foundation|     82823|      2229|
|                       Oxfam Australia|    121311|      2139|
|               Solutions for Australia|     32078|      1819|
|                The Wilderness Society|     74450|      1518|
|     Australian Automobile Association|     58804|      1397|
|                           Climate 200|     28285|      1351|
|                  Australia for UNHCR|    168107|      1331|
|                       Thrive By Five|     72557|      1100|
|           Save the Children Australia|     46915|      1036|
|       The Pharmacy Guild of Australia|     44735|      1017|
|  Asylum Seeker Resource Centre (ASRC)|     23755|       890|
|                            Greenfleet|     24273|       870|
|                           ABC Friends|     18712|       864|
|              Invasive Species Council|     40687|       758|
|                   ActionAid Australia|     34910|       742|
|           Festival of Dangerous Ideas|     17506|       730|
|                  Rebecca M Thistleton|     10520|       722|
|          Special Broadcasting Service|     22673|       666|
|                     Advance Australia|     44397|       613|
|                           Prime Super|     79642|       604|
|         Australian Ethical Investment|     43065|       599|
|                     Animals Australia|     13327|       572|
|                     Daniel Andrews MP|     14374|       565|
|                     Australian Unions|     97509|       545|
|                   The Climate Council|     14159|       526|
|The Australian Institute of Architects|     14383|       525|
+-----------------------------------+----------+----------+
only showing top 30 rows
```

that's the whole classifier story - candidate then party_org then government then commercial, with the leftover null+byline rows shown at the end as a triage list. the actual write of the v2 parquet, partitioned by political_party, lives in 02_preprocessing.py.

`03_topics.ipynb`

spark session.

In [1]:
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    array_contains, col, coalesce, concat_ws, expr, length, lit,
)

# Same parquet-committer override as notebook 02 — cluster default points at an EMR
# class whose JAR isn't on the classpath.
spark = SparkSession.builder \
    .appName('FB_API_topics') \
    .config('spark.sql.parquet.output.committer.class',
            'org.apache.parquet.hadoop.ParquetOutputCommitter') \
    .config('mapreduce.fileoutputcommitter.algorithm.version', '2') \
    .getOrCreate()

print('Master:', spark.sparkContext.master)
print('Spark version:', spark.version)
```

```
Master: yarn
Spark version: 3.5.0
```

```
26/05/17 07:30:22 WARN SparkSession: Using an existing Spark session; only runtime S
QL configurations will take effect.
```

paths.

In [2]:
```
V2_PATH = '/user/s3348393/main/preprocessing/v2/parquet'
```

load v2 and filter to the residual corpus. filter out the duplicates and only keep match_type is null, and in english. Ignore some commercial entities found while working through the problem up front.

the body text etc are arrays - pull out the first non-null value from each array (probably the first, but just being careful)

some of the bodies are empty - add in the title and description to try and up the number of rows we can use

In [3]:
```
df = spark.read.parquet(V2_PATH)
print('All v2 rows:    ', df.count())


def first_non_empty(col_name):
    """First non-null, non-empty element of an array column. Returns null if none."
    return expr(f"filter({col_name}, x -> x is not null and length(x) > 0)[0]")


#rejects
COMMERCIAL_BYLINES = {
    'Access',                           # Indigenous Employment Australia — job li
    'Streamotion Pty Ltd',              # Kayo / Binge sports + entertainment stre
    'SBS Australia',                    # SBS On Demand streaming promotions
```

```
    'SBS Arabic24',                          # SBS language-stream marketing
    'SBS Mandarin中文普通话',                 # SBS language-stream marketing
    'The Squiz',                             # paid news newsletter
    'Hair Cooki',                               #hair care ad
    "Shell"
}


corpus = df.filter(
        (col('ad_seq_no') == 1) &
        col('match_type').isNull() &
        # most of the language values are null - keep nulls and english
        (col('languages').isNull() | array_contains('languages', 'en')) &
        ~col('bylines').isin(list(COMMERCIAL_BYLINES))
    ) \
    .withColumn('body_text',  first_non_empty('creative_bodies')) \
    .withColumn('desc_text',  first_non_empty('creative_link_descs')) \
    .withColumn('title_text', first_non_empty('creative_link_titles')) \
    .withColumn('body',
        concat_ws(' ',
            coalesce(col('body_text'),  lit('')),
            coalesce(col('desc_text'),  lit('')),
            coalesce(col('title_text'), lit('')),
        )
    ) \
    .filter(length(col('body')) > 0) \
    .drop('body_text', 'desc_text', 'title_text')

print('Residual corpus:', corpus.count())
corpus.select('page_name', 'bylines', 'body').show(3, truncate=80)
```

```
All v2 rows:     5796491
```

```
Residual corpus: 94358
+---------------------------------+----------------------------+-----------------
-------------------------------------------------------------+
|                        page_name|                    bylines|
body|
+---------------------------------+----------------------------+-----------------
-------------------------------------------------------------+
|                    Thrive by Five|              Thrive By Five|Our early learning
and childcare centres are under stress, and early childhoo...|
|Time for Change - Change Aged Care|         United Workers Union|Important Informat
ion for Aged Care workers. \n\nYour union has compiled a li...|
|     Australian Ethical Investment|Australian Ethical Investment|The sooner you swi
tch to animal-friendly super, the sooner factory farming an...|
+---------------------------------+----------------------------+-----------------
-------------------------------------------------------------+
only showing top 3 rows
```

add to the stop words some generic keywords that kept popping up.

In [4]:
```
from pyspark.ml.feature import StopWordsRemover

stop_words = StopWordsRemover.loadDefaultStopWords('english') + [
    # URL / web junk that survives tokenisation
    'https', 'http', 'www', 'com', 'org', 'au', 'co', 'html',
    # contraction fragments surviving minTokenLength=2
    're', 've', 'll',
    # generic fillers (high frequency, low topic-discrimination value)
    'help', 'time', 'like', 'need', 'make', 'take', 'people',
    'year', 'years', 'today', 'also', 'will', 'can', 'get',
    'see', 'know', 'one', 'two', 'new', 'now', 'us',
    # generic CTA (kept short — don't strip 'petition', 'donate', 'sign', etc.
    # since those carry topic signal)
    'click', 'learn',
    #others
    'australia', 'australian', "2022"
]

print('Stop-words list size:', len(stop_words))
```

```
Stop-words list size: 218
```

preprog pipeline.

regex tokenizer, stop word remover count vectoriser.

re-run the pipeline a few times to tune the count vectoriser.

In [5]:
```
from pyspark.ml import Pipeline
from pyspark.ml.feature import RegexTokenizer, CountVectorizer

tokenizer = RegexTokenizer(
    inputCol='body', outputCol='raw_tokens',
    pattern=r'\W+', toLowercase=True, minTokenLength=2,
)

remover = StopWordsRemover(
    inputCol='raw_tokens', outputCol='tokens',
    stopWords=stop_words,
)

vectorizer = CountVectorizer(
    inputCol='tokens', outputCol='features',
    vocabSize=5000, #was 10000
    minDF=100,    # was 50
    maxDF=0.3,    # seems ok?
)

prep_pipeline = Pipeline(stages=[tokenizer, remover, vectorizer])
```

i did a bunch of runs, and apparently cache should help.

In [6]:
```
prep_model  = prep_pipeline.fit(corpus)
features_df = prep_model.transform(corpus).cache()
```

```
vocab = prep_model.stages[-1].vocabulary
print('Vocabulary size:', len(vocab))
print('\nTop 30 vocabulary terms (most frequent first):')
for i in range(0, 30, 3):
    print(f"{vocab[i]:<18}{vocab[i+1]:<18}{vocab[i+2]}")
```

```
Vocabulary size: 4251

Top 30 vocabulary terms (most frequent first):
sign              climate           government
support           community         petition
change            vote              world
protect           women             future
action            local             join
free              election          stop
woodside          share             council
children          life              donate
energy            every             labor
gas               tell              health
```

```
26/05/17 07:30:44 WARN SparkStringUtils: Truncated the string representation of a pl
an since it was too large. This behavior can be adjusted by setting 'spark.sql.debu
g.maxToStringFields'.
```

test some different group sizes on 10% data sample.

In [7]:
```
from pyspark.ml.clustering import LDA

#10%
sample_df = features_df.sample(0.1, seed=42).cache()

# Lookup from CountVectorizer integer term indices back to readable words.
vocab = prep_model.stages[-1].vocabulary

# Fit LDA at each k. seed=42 fixed so k=5 vs k=10 etc. are comparable.
for k in [5, 10, 15, 20, 25, 30]:
    print(k)
    lda = LDA(featuresCol='features', k=k, maxIter=20, seed=42)
    model = lda.fit(sample_df)
    topics = model.describeTopics(maxTermsPerTopic=10).collect()
    for row in topics:
        words = ' '.join(vocab[i] for i in row.termIndices)
        print(f'  Topic {row.topic:>2}: {words}')

sample_df.unpersist()
```

```
5
```

```
26/05/17 07:30:53 WARN OnlineLDAOptimizer: The input data is not directly cached, wh
ich may hurt performance if its parent RDDs are also uncached.
```

```
  Topic  0: vote government community women council children support local election
every
  Topic  1: sign petition government woodside enough stop support tell gas donate
  Topic  2: early solar learning support childcare refugees families day world educa
tors
  Topic  3: climate sign action community change government support free petition sh
are
  Topic  4: protect oceans woodside donate climate gas project ocean stop sign
10
26/05/17 07:31:05 WARN OnlineLDAOptimizer: The input data is not directly cached, wh
ich may hurt performance if its parent RDDs are also uncached.
  Topic  0: product description name child super add join animal ethical queensland
  Topic  1: sign woodside petition oceans share tell ocean enough stop global
  Topic  2: solar human world survey watch interview ants breakthrough condition yel
low
  Topic  3: election abc independent climate parliament sign party free women federa
l
  Topic  4: gas woodside climate donate protect project whales future fossil stop
  Topic  5: wrc forest forestry industry 50 climate mayoral communities products tas
manian
  Topic  6: support sign children donate plastic life women use free lives
  Topic  7: government labor school cost every morrison medicines education million
support
  Topic  8: climate community government vote council action energy local change sup
port
  Topic  9: children early women sign learning kids agl petition every childcare
15
26/05/17 07:31:15 WARN OnlineLDAOptimizer: The input data is not directly cached, wh
ich may hurt performance if its parent RDDs are also uncached.
  Topic  0: super join switch ethical product minutes consider heart read pds
  Topic  1: care aged workers religious better older richard companies bill australi
ans
  Topic  2: clive palmer november elizabeth study sure abroad elections ballot vote
  Topic  3: wrc morrison media toyota sydney climate news scott mayoral truth
  Topic  4: planet future protect change gift generations september 400 find join
  Topic  5: forestry industry forest register tasmanian products tasmania climate in
dustries timber
  Topic  6: support donate children life women lives provide food day refugees
  Topic  7: medicines cost bit level ly prescription visit australians rate labor
  Topic  8: government climate community council vote local action change labor supp
ort
  Topic  9: children early every sign learning petition school education agl schools
  Topic 10: vote name stand child add children sex election sexual pledge
  Topic 11: sign woodside stop climate petition gas women tell share enough
  Topic 12: genus kids sustainability fun planet platform generation offer always fu
ture
  Topic 13: sign oceans ocean world petition global treaty protect demand government
s
  Topic 14: energy power solar electricity renewable bills mp coal battery human
20
26/05/17 07:31:25 WARN OnlineLDAOptimizer: The input data is not directly cached, wh
ich may hurt performance if its parent RDDs are also uncached.
```

```
  Topic  0: yemen name add stop children countries deforestation war child day
  Topic  1: aged care richard companies issue workers older media minister test
  Topic  2: abroad study november americans sure elections ballot vote votefromabroa
d ballots
  Topic  3: wrc toyota mayoral clean transport car 50 1977 company forensic
  Topic  4: super future join switch planet protect esg gift ethical superannuation
  Topic  5: industry climate forestry register forest join sydney book tasmania cent
re
  Topic  6: support donate refugees donation provide food ukraine crisis life world
  Topic  7: school government morrison medicines cost schools every australians pres
cription students
  Topic  8: climate community government vote change action local council support el
ection
  Topic  9: children women sign early petition every violence learning support famil
y
  Topic 10: stand name child sex vote wildlife nature protect may sexual
  Topic 11: sign woodside stop gas petition tell enough share project women
  Topic 12: kids genus sustainability fun future planet platform generation offer lo
cal
  Topic 13: electricity green switch land world melbourne festival sydney nsw voice
  Topic 14: energy solar power renewable electricity bills renewables wind battery p
rice
  Topic 15: electric mp batteries government jobs vehicles federal minerals clean ma
nufacturing
  Topic 16: christmas vaccine freedom australians choice show sign vaxxed fair passp
ort
  Topic 17: sign oceans ocean demand world petition global treaty share protect
  Topic 18: parliament grace indigenous tame tea higgins referendum prime culture ch
ange
  Topic 19: digital lgbtqia teachers contract screening body qantas byron might drin
k
25
26/05/17 07:31:36 WARN OnlineLDAOptimizer: The input data is not directly cached, wh
ich may hurt performance if its parent RDDs are also uncached.
```

```
  Topic  0: child deforestation labour name add european 10 end globally experience
  Topic  1: weekly media issue companies urge oz arab care multinational fuelling
  Topic  2: november sure counted 2020 early learning americans tropics mt overseas
  Topic  3: wrc mayoral 50 1977 forensic communities hood heard candidate life
  Topic  4: super ethical join september 400 pds consider read information product
  Topic  5: forestry industry forest children yemen products war countries tasmanian
timber
  Topic  6: donate refugees support provide ukraine donation families food refugee c
risis
  Topic  7: level laws crossing crossings defence strong quoll environment gone spot
ted
  Topic  8: community climate council government action emissions business working f
uture health
  Topic  9: early learning every childcare sign children families petition child kid
s
  Topic 10: native child wildlife name stand sex add protect logging sexual
  Topic 11: climate vote sign election women woodside enough stop change tell
  Topic 12: genus kids sustainability fun platform planet generation always offer wa
rning
  Topic 13: sign plastic abc petition use single plastics big free share
  Topic 14: energy solar power cost medicines prescription electricity bills visit r
enewable
  Topic 15: mp ballarat paul guy nationals liberals renters community matthew auspol
  Topic 16: vaccine freedom christmas australians choice sign show vaxxed fair passp
ort
  Topic 17: sign oceans ocean petition demand world global share treaty protect
  Topic 18: grace parliament tame pm dutton indigenous higgins tea albanese design
  Topic 19: topstory joyce might modern aboriginal lgbtqia advocate indigenous alan
victoria
  Topic 20: seats vote ballot voters abroad twu members request november team
  Topic 21: government support labor children state local morrison change school lif
e
  Topic 22: climate gas join woodside donate project future super fossil whales
  Topic 23: violence support domestic emergency safe sleep women vinnies place 64
  Topic 24: electricity festival switch green program greener fodi22 sign live festi
valofdangerousideas
30
26/05/17 07:31:47 WARN OnlineLDAOptimizer: The input data is not directly cached, wh
ich may hurt performance if its parent RDDs are also uncached.
```

```
  Topic  0: victor local community council experience city including nominate servic
es children
  Topic  1: issue media urge weekly companies oz arab multinational fuelling gave
  Topic  2: offshore emissions almost report carbon early learning greenhouse instit
ute religious
  Topic  3: wrc mayoral 50 1977 forensic communities hood life bring back
  Topic  4: super ethical join september consider 400 pds information switch product
  Topic  5: forestry industry forest children war yemen tasmanian products tasmania
countries
  Topic  6: support donate refugees ukraine provide food donation families crisis li
ves
  Topic  7: level laws defence crossings crossing strong quoll gone spotted environm
ent
  Topic  8: climate action community change government native emissions crisis counc
il victoria
  Topic  9: early learning childcare every families sign petition children affordabl
e fair
  Topic 10: name child product vote stand sex add sexual protect wildlife
  Topic 11: sign women vote woodside election enough tell stop share voice
  Topic 12: genus kids sustainability fun planet generation platform always offer wa
rning
  Topic 13: sign plastic use single share petition support plastics world big
  Topic 14: solar cost medicines energy power prescription visit electricity bills s
ite
  Topic 15: ballarat paul renters auspol guy liberals fairer renting matthew harbour
  Topic 16: vaccine freedom christmas australians sign choice show passport vaxxed f
air
  Topic 17: sign petition demand ocean oceans treaty global share quiz human
  Topic 18: grace parliament tame indigenous tea higgins design referendum albanese
change
  Topic 19: body required hopes victoria police accountability community issues tops
tory warren
  Topic 20: electricity switch green provider guide greener seats voters ballot abro
ad
  Topic 21: government community support labor local children state council change w
ork
  Topic 22: join climate energy future better super good clean government electric
  Topic 23: zero energy net wa renewable david dr emissions electricity hydrogen
  Topic 24: festival program fodi22 live festivalofdangerousideas book tickets artis
t multipack 18
  Topic 25: ukraine support action leah stop funds sleep safe climate free
  Topic 26: emissions getting local reducing action send medicines downward power cl
imate
  Topic 27: donate gas protect woodside stop climate abc project oceans save
  Topic 28: child labour 10 switching nick wakeling ferntree gully kiln companies
  Topic 29: agl coal sign petition demerger graham vanessa climate act power
```

Out[7]:
```
DataFrame[id: string, page_id: string, page_name: string, snapshot_date: date, ad_
creation_date: date, ad_delivery_start_date: date, ad_delivery_stop_date: date, cr
eative_bodies: array<string>, creative_link_captions: array<string>, creative_link
_descs: array<string>, creative_link_titles: array<string>, spend_lower_bound: big
int, spend_upper_bound: bigint, spend_mid: double, impressions_lower_bound: bigin
t, impressions_upper_bound: bigint, impressions_mid: double, audience_size_lower_b
ound: bigint, audience_size_upper_bound: bigint, audience_size_mid: double, curren
cy: string, languages: array<string>, publisher_platforms: array<string>, demograp
hic_distribution: array<struct<age:string,gender:string,percentage:string>>, deliv
ery_by_region: array<struct<percentage:string,region:string>>, ad_snapshot_url: st
ring, bylines: string, ad_seq_no: int, match_type: string, political_party: strin
g, body: string, raw_tokens: array<string>, tokens: array<string>, features: vecto
r]
```

below 20 blurs, above 20 some of the bags dont make sense. at 20 we can clearly see some topic clusters which dont look political, and can remove them.

In [ ]:
```python
from pyspark.ml.clustering import LDA
from pyspark.ml.functions import vector_to_array
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, desc

K = 20

# Fit LDA at k=K on the full cached features. seed=42 for reproducibility.
lda = LDA(featuresCol='features', k=K, maxIter=20, seed=42)
lda_model = lda.fit(features_df)

# Transform: attach topicDistribution and dominant topic_id to every ad.
classified = lda_model.transform(features_df) \
    .withColumn('topic_array', vector_to_array('topicDistribution')) \
    .withColumn('topic_id', expr('array_position(topic_array, array_max(topic_array

# Vocabulary for term-index -> word translation.
vocab = prep_model.stages[-1].vocabulary

# Top 15 terms per topic from describeTopics.
topics_rows = lda_model.describeTopics(maxTermsPerTopic=15).collect()
topic_terms = {row.topic: [vocab[i] for i in row.termIndices] for row in topics_row

# Top 5 bylines per topic — Spark window function over (topic_id, bylines, count).
w = Window.partitionBy('topic_id').orderBy(desc('count'))
top_bylines_rows = classified.filter(col('bylines').isNotNull()) \
    .groupBy('topic_id', 'bylines').count() \
    .withColumn('rank', row_number().over(w)) \
    .filter(col('rank') <= 5) \
    .orderBy('topic_id', 'rank') \
    .collect()

top_bylines = {}
for row in top_bylines_rows:
    top_bylines.setdefault(row.topic_id, []).append(row.bylines)

# Print to console — easy to scan while you draft labels.
print(f'k = {K}\n')
```

```
for tid in range(K):
    words = ' '.join(topic_terms.get(tid, []))
    bls   = ' | '.join(top_bylines.get(tid, []))
    print(f'Topic {tid:>2}:')
    print(f'  Terms:   {words}')
    print(f'  Bylines: {bls}')
    print()
```

```
26/05/17 07:32:00 WARN OnlineLDAOptimizer: The input data is not directly cached, wh
ich may hurt performance if its parent RDDs are also uncached.
```

```
k = 20

Topic  0:
  Terms:   super product switch feral join ethical native animals future species min
utes performance description horses consider
  Bylines: Australian Ethical Investment | Invasive Species Council | Prime Super |
1in50 Incorporated | Future Super

Topic  1:
  Terms:   donate christmas provide give food ukraine support crisis donation emerge
ncy please care gift families water
  Bylines: Australia for UNHCR | Médecins Sans Frontières Australia | Wayside Chapel
| Oxfam Australia | Invasive Species Council

Topic  2:
  Terms:   early learning childcare educators families support petition sign childre
n quiz parents childhood affordable many accessible
  Bylines: Thrive By Five | Amnesty International Australia | TWU Members First Team
| Danny Pearson MP for Essendon | Gender Awareness Australia

Topic  3:
  Terms:   government parliament nsw community mp state council public federal membe
r news minister kids national armenian
  Bylines: Private Media | Amnesty International Australia | Genus Earth Pty Ltd | G
reenpeace Australia Pacific | Armenian National Committee of Australia

Topic  4:
  Terms:   gas wa project woodside climate stop fossil fuel marine polluting life pr
otect expansion proposed toxic
  Bylines: Greenpeace Australia Pacific | Impact X Pty Ltd | Invasive Species Counci
l | The Wilderness Society | Festival of Dangerous Ideas

Topic  5:
  Terms:   climate action government sign change protect petition reef crisis fossil
hunger extreme world conflict future
  Bylines: Oxfam Australia | Greenpeace Australia Pacific | Greenfleet | The Wildern
ess Society | Climate 200

Topic  6:
  Terms:   support council community local ward home good better residents care work
donate life business families
  Bylines: Australia for UNHCR | Asylum Seeker Resource Centre (ASRC) | Goods Intern
ational Pty Ltd | The Wilderness Society | Parliament House of Cards Pty Ltd

Topic  7:
  Terms:   solar school every education schools government public climb free student
s challenge may funding morrison right
  Bylines: Australian Conservation Foundation | School for Life Foundation | Greenpe
ace Australia Pacific | Australian Education Union Victorian Branch | Amnesty Intern
ational Australia

Topic  8:
  Terms:   community climate government local council health future change state act
ion want labor city school road
  Bylines: The Climate Council | Greenfleet | Prime Super | Jackson Taylor MP | Gord
on Rich-Phillips MLC
```

Topic 9:
  Terms: women children violence support child every day refugees girls lives family world sign end safety
  Bylines: Asylum Seeker Resource Centre (ASRC) | Amnesty International Australia | Save the Children Australia | Plan International Australia | Destiny Rescue

Topic 10:
  Terms: vote election labor government party liberal climate morrison power federal independent candidate victoria electricity support
  Bylines: Climate 200 | Private Media | Rebecca M Thistleton | Animals Australia | Greenpeace Australia Pacific

Topic 11:
  Terms: sign woodside tell enough gas share stop send big message petition project women ceo meg
  Bylines: Greenpeace Australia Pacific | Oxfam Australia | ActionAid Australia | Save the Children Australia | World Animal Protection Australia

Topic 12:
  Terms: vote council mayor city voting elections group december watch deforestation team free human authorised hobart
  Bylines: World Transformation Movement | The Wilderness Society | NSW Electoral Commission | Bernard Anthony Purcell | Jilly Gibson

Topic 13:
  Terms: sign plastic future use protect single donate donation plastics whales planet tax gift world gas
  Bylines: Greenpeace Australia Pacific | Australian Conservation Foundation | Bank Australia | National Retail Association | Environmental Defenders Office

Topic 14:
  Terms: energy power cost medicines renewable agl government coal laws nature prescription living australians electricity visit
  Bylines: The Pharmacy Guild of Australia | Solutions for Australia | Greenpeace Australia Pacific | Australian Conservation Foundation | The Wilderness Society

Topic 15:
  Terms: festival electric jobs program batteries live create tech clean government minerals refining world mining support
  Bylines: Festival of Dangerous Ideas | Solutions for Australia | The Chamber of Minerals and Energy of Western Australia | Sydney Opera House | St Vincent de Paul Society in Australia

Topic 16:
  Terms: abc sign petition workers care australians aged save health union independent deserve product freedom vaccine
  Bylines: ABC Friends | United Workers Union | Advance Australia | Animals Australia | Australian Unions

Topic 17:
  Terms: sign oceans ocean petition demand global treaty protect share world heard better rights show act
  Bylines: Greenpeace Australia Pacific | Australian Automobile Association | Australian Conservation Foundation | UN Women Australia | Amnesty International Australia

Topic 18:
  Terms: climate action support cancer energy join voice melbourne campaign power nzf australians families world aboriginal
  Bylines: NATIONAL ZAKAT FOUNDATION INCORPORATED | Private Media | Australian Parents for Climate Action | Rebecca M Thistleton | Australian Conservation Foundation

Topic 19:
  Terms: event community work join free young day life support ly coffee june online local bit
  Bylines: The Australian Institute of Architects | Private Media | National Youth Commission Australia | Kua | The University of Queensland

```
26/05/17 09:26:23 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_6 !
26/05/17 09:26:23 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_1 !
26/05/17 09:26:23 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_0 !
26/05/17 09:26:23 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_12 !
26/05/17 09:26:23 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_7 !
26/05/17 09:26:23 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_9 !
26/05/17 09:26:23 WARN YarnSchedulerBackend$YarnSchedulerEndpoint: Requesting driver
to remove executor 1 for reason Container marked as failed: container_1776819796568_
1546_01_000004 on host: ip-100-64-73-193.ap-southeast-2.compute.internal. Exit statu
s: -100. Diagnostics: Container released on a *lost* node.
26/05/17 09:26:23 ERROR YarnScheduler: Lost executor 1 on ip-100-64-73-193.ap-southe
ast-2.compute.internal: Container marked as failed: container_1776819796568_1546_01_
000004 on host: ip-100-64-73-193.ap-southeast-2.compute.internal. Exit status: -100.
Diagnostics: Container released on a *lost* node.
26/05/17 09:26:52 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_10 !
26/05/17 09:26:52 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_2 !
26/05/17 09:26:52 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_11 !
26/05/17 09:26:52 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_14 !
26/05/17 09:26:52 ERROR YarnScheduler: Lost executor 3 on ip-100-64-73-111.ap-southe
ast-2.compute.internal: Container marked as failed: container_1776819796568_1546_01_
000010 on host: ip-100-64-73-111.ap-southeast-2.compute.internal. Exit status: -100.
Diagnostics: Container released on a *lost* node.
26/05/17 09:26:52 WARN YarnSchedulerBackend$YarnSchedulerEndpoint: Requesting driver
to remove executor 3 for reason Container marked as failed: container_1776819796568_
1546_01_000010 on host: ip-100-64-73-111.ap-southeast-2.compute.internal. Exit statu
s: -100. Diagnostics: Container released on a *lost* node.
26/05/17 09:26:53 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_4 !
26/05/17 09:26:53 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_13 !
26/05/17 09:26:53 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_3 !
26/05/17 09:26:53 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_8 !
26/05/17 09:26:53 WARN BlockManagerMasterEndpoint: No more replicas available for rd
d_45_5 !
26/05/17 09:26:53 ERROR YarnScheduler: Lost executor 2 on ip-100-64-73-142.ap-southe
ast-2.compute.internal: Container marked as failed: container_1776819796568_1546_01_
000009 on host: ip-100-64-73-142.ap-southeast-2.compute.internal. Exit status: -100.
Diagnostics: Container released on a *lost* node.
26/05/17 09:26:53 WARN YarnSchedulerBackend$YarnSchedulerEndpoint: Requesting driver
to remove executor 2 for reason Container marked as failed: container_1776819796568_
1546_01_000009 on host: ip-100-64-73-142.ap-southeast-2.compute.internal. Exit statu
s: -100. Diagnostics: Container released on a *lost* node.
```

`04_topic_join.ipynb`

setup.

In [1]:
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, broadcast, desc

spark = SparkSession.builder \
    .appName('FB_API_v3_build') \
    .config('spark.sql.parquet.output.committer.class',
            'org.apache.parquet.hadoop.ParquetOutputCommitter') \
    .config('mapreduce.fileoutputcommitter.algorithm.version', '2') \
    .getOrCreate()

print('Master:', spark.sparkContext.master)
print('Spark version:', spark.version)
```

```
Master: yarn
Spark version: 3.5.0
```

```
26/05/17 09:32:37 WARN SparkSession: Using an existing Spark session; only runtime S
QL configurations will take effect.
```

paths.

In [2]:
```
V2_PATH           = '/user/s3348393/main/preprocessing/v2/parquet'
INTERMEDIATE_PATH = '/user/s3348393/main/preprocessing/v3/intermediate_parquet'
V3_PATH           = '/user/s3348393/main/preprocessing/v3/parquet'
TOPIC_LABELS_CSV  = '../data/topic_labels.csv'
```

join labels and topic info onto v2. three inputs combined. v2 parquet (one row per ad via ad_seq_no = 1) — every ad with match_type and political_party. intermediate parquet (residual subset only, with topic_id + topicDistribution) from nb 03. topic_labels.csv — hand-edited labels with topic_label and category.

In [3]:
```
import pandas as pd

v2 = spark.read.parquet(V2_PATH).filter(col('ad_seq_no') == 1)

intermediate = spark.read.parquet(INTERMEDIATE_PATH).select('id', 'topic_id', 'topi

#Load the data labels using pandas on local
labels_pdf = pd.read_csv(TOPIC_LABELS_CSV)[['topic_id', 'label', 'category']]
labels = spark.createDataFrame(labels_pdf).withColumnRenamed('label', 'topic_label'

classified = v2.join(intermediate, 'id', 'left').join(broadcast(labels), 'topic_id'

print(f'All ads (ad_seq_no=1): {classified.count():,}')
print(f'  with topic info:    {classified.filter(col("topic_id").isNotNull()).count
print(f'  without topic info: {classified.filter(col("topic_id").isNull()).count():
```

```
26/05/17 09:32:53 WARN YarnScheduler: Initial job has not accepted any resources; ch
eck your cluster UI to ensure that workers are registered and have sufficient resour
ces
26/05/17 09:33:08 WARN YarnScheduler: Initial job has not accepted any resources; ch
eck your cluster UI to ensure that workers are registered and have sufficient resour
ces
26/05/17 09:33:23 WARN YarnScheduler: Initial job has not accepted any resources; ch
eck your cluster UI to ensure that workers are registered and have sufficient resour
ces
26/05/17 09:33:38 WARN YarnScheduler: Initial job has not accepted any resources; ch
eck your cluster UI to ensure that workers are registered and have sufficient resour
ces
26/05/17 09:33:53 WARN YarnScheduler: Initial job has not accepted any resources; ch
eck your cluster UI to ensure that workers are registered and have sufficient resour
ces
26/05/17 09:34:08 WARN YarnScheduler: Initial job has not accepted any resources; ch
eck your cluster UI to ensure that workers are registered and have sufficient resour
ces
26/05/17 09:34:23 WARN YarnScheduler: Initial job has not accepted any resources; ch
eck your cluster UI to ensure that workers are registered and have sufficient resour
ces
26/05/17 09:34:39 WARN SparkStringUtils: Truncated the string representation of a pl
an since it was too large. This behavior can be adjusted by setting 'spark.sql.debu
g.maxToStringFields'.
```

```
All ads (ad_seq_no=1): 151,578
  with topic info:    94,358
  without topic info: 57,220
```

qc - check counts to make sure everything's tagged properly

In [4]:

```
print('match_type × category:')
classified.groupBy('match_type', 'category').count() \
    .orderBy('match_type', 'category') \
    .show(50, truncate=False)

# Belt-and-braces: did any topic_id end up unmatched in the labels CSV?
unmatched = classified.filter(col('topic_id').isNotNull() & col('topic_label').isNu
print(f'Rows with topic_id but no topic_label (should be 0): {unmatched}')
```

```
match_type × category:
+----------+------------------+-----+
|match_type|category          |count|
+----------+------------------+-----+
|NULL      |NULL              |9523 |
|NULL      |climate           |28981|
|NULL      |cost_of_living    |6359 |
|NULL      |humanitarian_rights|9076 |
|NULL      |mixed             |15834|
|NULL      |political_advocacy |34108|
|candidate |NULL              |23327|
|commercial|NULL              |3    |
|government|NULL              |1219 |
|party_org |NULL              |23148|
+----------+------------------+-----+

Rows with topic_id but no topic_label (should be 0): 0
```

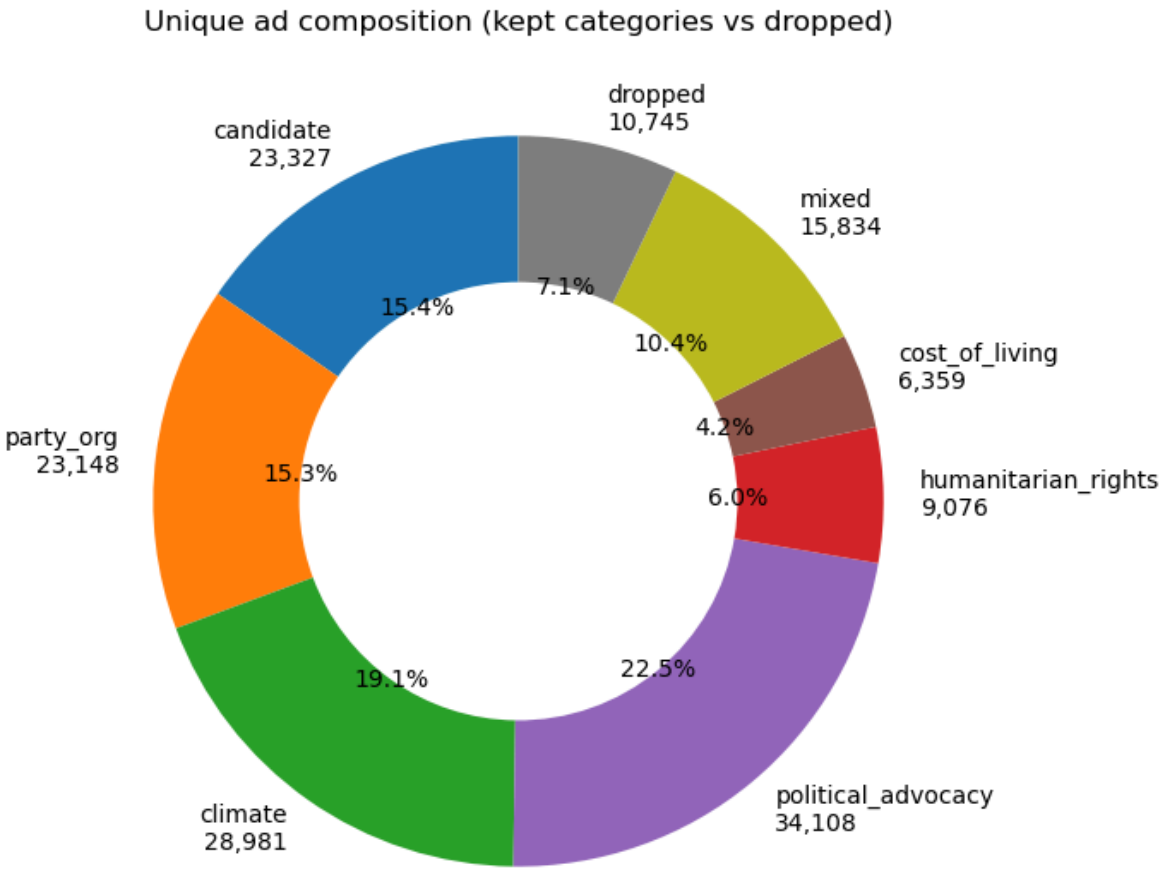
drop government and commercial rows

In [5]:
```
before = classified.count()

v3_candidate = classified.filter(
    col('match_type').isin('candidate', 'party_org')
    | (col('match_type').isNull() & col('category').isNotNull())
)
v3_candidate.cache()
after = v3_candidate.count()

print(f'Before filter: {before:,}')
print(f'After filter:  {after:,}')
print(f'Dropped:       {before - after:,}  ({100 * (before - after) / before:.1f}%)')

print('\nKept rows by match_type × category:')
v3_candidate.groupBy('match_type', 'category').count().orderBy('match_type', 'categ
```

```
Before filter: 151,578
After filter:  140,833
Dropped:       10,745  (7.1%)

Kept rows by match_type × category:
+----------+------------------+-----+
|match_type|category          |count|
+----------+------------------+-----+
|NULL      |climate           |28981|
|NULL      |cost_of_living    |6359 |
|NULL      |humanitarian_rights|9076 |
|NULL      |mixed             |15834|
|NULL      |political_advocacy |34108|
|candidate |NULL              |23327|
|party_org |NULL              |23148|
+----------+------------------+-----+
```

visualise the breakdown

In [6]:
```
import matplotlib.pyplot as plt

# Kept slices come from v3_candidate (already grouped by match_type × category abov
# match_type ∈ {candidate, party_org} → slice keyed by match_type;
# match_type IS NULL → slice keyed by category.
kept_pdf = v3_candidate.groupBy('match_type', 'category').count().toPandas()

slices = {}
for _, r in kept_pdf.iterrows():
    key = r['match_type'] if r['match_type'] in ('candidate', 'party_org') else r['
    slices[key] = slices.get(key, 0) + int(r['count'])

# Single 'dropped' slice = everything filter excluded (government + noise + residua
slices['dropped'] = classified.count() - v3_candidate.count()

order = ['candidate', 'party_org',
         'climate', 'political_advocacy', 'humanitarian_rights', 'cost_of_living',
colors = {
    'candidate':           '#1f77b4',
    'party_org':           '#ff7f0e',
    'climate':             '#2ca02c',
    'political_advocacy':  '#9467bd',
    'humanitarian_rights': '#d62728',
    'cost_of_living':      '#8c564b',
    'mixed':               '#bcbd22',
    'dropped':             '#7f7f7f',
}

sizes  = [slices[k] for k in order]
labels = [f'{k}\n{slices[k]:,}' for k in order]

fig, ax = plt.subplots(figsize=(7, 7))
ax.pie(
    sizes,
    labels=labels,
    colors=[colors[k] for k in order],
    autopct='%1.1f%%',
    startangle=90,
    wedgeprops=dict(width=0.4),
)
ax.set_title('Unique ad composition (kept categories vs dropped)')
plt.tight_layout()
plt.show()
```



i havent dropped mixed at this point. lets check false positives and false negatives.

In [9]:
```
# --- False positives: top-spend ads in each kept category ---

for cat in ['climate', 'humanitarian_rights', 'political_advocacy', 'cost_of_living
    print(f'\n--- {cat} ---')
    v3_candidate.filter((col('category') == cat) & (col('spend_mid').isNotNull()))
        .orderBy(desc('spend_mid')) \
        .select('page_name', 'bylines') \
        .distinct() \
        .show(5, truncate=60)

# Also: top-spend candidate + party_org
print('\n=== FP check: top-spend candidate + party_org ads ===')
v3_candidate.filter(col('match_type').isin('candidate', 'party_org') & col('spend_m
    .orderBy(desc('spend_mid')) \
    .select('page_name', 'bylines', 'political_party') \
    .distinct() \
    .show(10, truncate=60)

# --- False negatives: top bylines in dropped noise topics ---
print('\n=== FN check: top bylines in DROPPED noise-labelled topics ===')
classified.filter(col('category') == 'noise') \
    .groupBy('topic_label', 'bylines').count() \
    .orderBy(desc('count')) \
```

```
    .show(20, truncate=80)

# --- False negatives: top bylines in residual with no topic_id ---
print('\n=== FN check: top bylines in residual with NO topic_id (empty body / non-E
classified.filter(col('match_type').isNull() & col('topic_id').isNull() & col('byli
    .groupBy('bylines').count() \
    .orderBy(desc('count')) \
    .show(20, truncate=80)
```

```
--- climate ---
+--------------------+---------------------------------------------+
|           page_name|                                      bylines|
+--------------------+---------------------------------------------+
|  David Southwick MP|                          David J Southwick|
|  Michael O'Brien MP|The Hon. Michael O'Brien MP, Member for Malvern|
|Great Plastic Rescue|                       Great Plastic Rescue|
|      Save Our Spuds|                               Glen F Jones|
|Double It For Nature|                       Double It For Nature|
+--------------------+---------------------------------------------+
only showing top 5 rows


--- humanitarian_rights ---
+----------------------------------+--------------------------+
|                         page_name|                   bylines|
+----------------------------------+--------------------------+
|                David Southwick MP|         David J Southwick|
|                    Wayside Chapel|            Wayside Chapel|
|        Democrats Abroad Australia|Democrats Abroad Australia|
|                    Matt Fregon MP|            Matt Fregon MP|
|Lauren Kathage - Labor for Yan Yean|            Lauren Kathage|
+----------------------------------+--------------------------+
only showing top 5 rows


--- political_advocacy ---
+----------------------------------+-----------------------------------------
-+
|                         page_name|                                    byline
s|
+----------------------------------+-----------------------------------------
-+
|                David Southwick MP|                            David J Southwic
k|
|       Rockhampton Regional Council|                  Rockhampton Regional Counci
l|
|           Keep Corporations Honest|                       Class Actions Australi
a|
|Kaspar Deane for Kingborough Council|                          Kaspar Wilson Dean
e|
|                Michael O'Brien MP|The Hon. Michael O'Brien MP, Member for Malver
n|
+----------------------------------+-----------------------------------------
-+
only showing top 5 rows


--- cost_of_living ---
+--------------------------+--------------------------------+
|                 page_name|                         bylines|
+--------------------------+--------------------------------+
|Plan International Australia|     Plan International Australia|
|          Dr David Honey MLA|               Dr David Honey MLA|
|          City of Parramatta|               City of Parramatta|
|              Michele O'Neil|Australian Council of Trade Unions|
```

```
|                  DPV Health|                         DPV Health|
+----------------------------+-----------------------------------+
only showing top 5 rows


=== FP check: top-spend candidate + party_org ads ===
+-----------------------------------------------------+-----------------------+--
-------------+
|                                            page_name|                bylines|po
litical_party|
+-----------------------------------------------------+-----------------------+--
-------------+
|                               Josh Bull MP for Sunbury|         Victorian Labor|
ALP|
|Sonia Bailey - Liberal Democrats Candidate for Paterson|       Liberal Democrats|
LDP|
|                      Lauren O'Dwyer Labor for Richmond|         Victorian Labor|
ALP|
|                                     Kristina Keneally|       Kristina Keneally|
ALP|
|                                           Zed Seselja|             Zed Seselja|
LP|
|                                       Zali Steggall|           Zali Steggall|
IND|
|                                        Jack Dempsey|Jack Dempsey for Hinkler|
IND|
|                  James Thomson - Nationals for Hunter| James Jefferson Thomson|
NP|
|          Nadia Sirninger Rankin - Greens for Bayswater|        Victorian Greens|
GRN|
|                               Kristie Nash UAP Hinkler|            Kristie Nash|
UAPP|
+-----------------------------------------------------+-----------------------+--
-------------+
only showing top 10 rows


=== FN check: top bylines in DROPPED noise-labelled topics ===
+-----------+-------+-----+
|topic_label|bylines|count|
+-----------+-------+-----+
+-----------+-------+-----+


=== FN check: top bylines in residual with NO topic_id (empty body / non-English / c
ommercial bylines) ===
+-----------------------------+-----+
|                      bylines|count|
+-----------------------------+-----+
|                       Access| 6787|
|  Special Broadcasting Service|  427|
|           Streamotion Pty Ltd|  376|
|                    Hair Cooki|  179|
|                 SBS Australia|  177|
|             WorkSafe Victoria|  144|
|                  SBS Arabic24|  130|
```

```
|                BPH ENERGY LTD|  127|
|         Cultural Perspectives|  123|
|             Daniel Andrews MP|   91|
|                     The Squiz|   56|
|Victorian Electoral Commission|   49|
|        SBS Mandarin中文普通话|   44|
|                      SBS Dari|   29|
| Victorian Trades Hall Council|   27|
|Jean Hailes for Women's Health|   25|
|              Jane Lomax-Smith|   16|
|                   SBS Russian|   14|
|     Australian Workers Union |   13|
|                   SBS Persian|   11|
+------------------------------+-----+
only showing top 20 rows
```

04b_hashtag_analysis.ipynb

extract every hashtag from the v3 advocacy corpus, count occurrences, and export anything appearing at least 10 times. broken down by band (from nb 05) so the election-themed hashtags can be lifted into the seed list for nb 06.

In [ ]:
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, lower, regexp_extract_all, explode, concat_ws, array_join,
    count as spark_count, sum as spark_sum, desc, broadcast, lit,
)
import pandas as pd

spark = SparkSession.builder \
    .appName('FB_API_hashtag_analysis') \
    .config('spark.sql.parquet.output.committer.class',
            'org.apache.parquet.hadoop.ParquetOutputCommitter') \
    .config('mapreduce.fileoutputcommitter.algorithm.version', '2') \
    .getOrCreate()

print('Master:', spark.sparkContext.master)
print('Spark version:', spark.version)
```

In [ ]:
```
V3_PATH    = '/user/s3348393/main/preprocessing/v3/parquet'
BANDS_CSV  = '../data/all_bylines_bands.csv'
OUTPUT_CSV = '../data/hashtags.csv'
MIN_COUNT  = 10
```

load v3 advocacy ads and attach band labels.

In [ ]:
```
v3 = spark.read.parquet(V3_PATH).filter(col('match_type').isNull())

# combine body + title arrays into one text blob per ad. don't lowercase yet —
# we want to extract hashtags first, then lowercase them so #AusVotes and
# #ausvotes collapse to the same row.
v3 = v3.withColumn('text', concat_ws(' ',
    array_join('creative_bodies', ' '),
    array_join('creative_link_titles', ' ')))

v3 = v3.filter(col('text').isNotNull() & (col('text') != ''))

# attach band labels (left join — bylines outside the qualifying set get null band)
try:
    bands_pdf = pd.read_csv(BANDS_CSV)[['bylines', 'band']]
    bands_sdf = spark.createDataFrame(bands_pdf)
    v3 = v3.join(broadcast(bands_sdf), 'bylines', 'left')
    have_bands = True
    print(f'Loaded {len(bands_pdf)} band-classified bylines.')
except FileNotFoundError:
    v3 = v3.withColumn('band', lit(None).cast('string'))
    have_bands = False
    print(f'{BANDS_CSV} not found — running without band breakdown. '
          'run nb 05 to generate the bands csv.')
```

```
v3 = v3.cache()
print(f'Advocacy ads with text: {v3.count():,}')
```

extract hashtags with a regex. regexp_extract_all returns every match per row as an array, which we explode to one hashtag per row. lowercase after extraction so capitalisation variants collapse.

In [ ]:
```
# matches #word — letters, digits and underscores after the hash, at least 2 chars.
HASHTAG_RE = r'#([A-Za-z0-9_]{2,})'

hashtag_rows = (v3
    .withColumn('hashtags', regexp_extract_all(col('text'), lit(HASHTAG_RE), 1))
    .filter(col('hashtags').isNotNull())
    .select('bylines', 'band', explode(col('hashtags')).alias('hashtag'))
    .withColumn('hashtag', lower(col('hashtag'))))

hashtag_rows = hashtag_rows.cache()
print(f'Total hashtag occurrences: {hashtag_rows.count():,}')
print(f'Distinct hashtags:         {hashtag_rows.select("hashtag").distinct().count
```

count by hashtag overall, then pivot in counts per band so election-themed hashtags are visually obvious. filter to hashtags occurring at least MIN_COUNT times.

In [ ]:
```
overall = (hashtag_rows.groupBy('hashtag').count()
    .withColumnRenamed('count', 'count_total')
    .filter(col('count_total') >= MIN_COUNT))

if have_bands:
    per_band = (hashtag_rows
        .groupBy('hashtag', 'band').count()
        .groupBy('hashtag')
        .pivot('band')
        .sum('count')
        .na.fill(0))
    out = overall.join(per_band, 'hashtag', 'left').orderBy(desc('count_total'))
else:
    out = overall.orderBy(desc('count_total'))

out_pdf = out.toPandas()

# tidy column order — overall count first, then per-band counts (if present).
band_cols = [c for c in ['election_only', 'transitional', 'ongoing', 'off_campaign'
             if c in out_pdf.columns]
out_pdf = out_pdf[['hashtag', 'count_total'] + band_cols +
                  [c for c in out_pdf.columns if c not in ['hashtag', 'count_total

print(f'Hashtags with count >= {MIN_COUNT}: {len(out_pdf):,}')
out_pdf.head(40)
```

write the csv. one row per hashtag, sorted by total count descending.

In [ ]:
```
out_pdf.to_csv(OUTPUT_CSV, index=False)
print(f'Wrote {OUTPUT_CSV} ({len(out_pdf):,} hashtags)')
```

top hashtags in the election_only band only — these are the candidates for the seed list in nb 06.

In [ ]:
```
if have_bands and 'election_only' in out_pdf.columns:
    election_top = (out_pdf[out_pdf['election_only'] >= MIN_COUNT]
                    .sort_values('election_only', ascending=False)
                    .head(40)
                    .reset_index(drop=True))
    display(election_top[['hashtag', 'election_only', 'count_total']])
else:
    print('Run nb 05 to generate band labels, then re-run this cell.')
```

05_election_spenders.ipynb

who else spent big on political-ish FB ads around the may 2022 election, and how do their patterns compare. input is v3 from notebook 04, advocacy rows only.

setup

In [2]:
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, sum as spark_sum, count as spark_count, countDistinct,
    when, lit, date_trunc, desc, broadcast,
)
from pyspark.sql.window import Window
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns

sns.set_theme(style='whitegrid', context='notebook')

spark = SparkSession.builder \
    .appName('FB_API_election_spenders') \
    .config('spark.sql.parquet.output.committer.class',
            'org.apache.parquet.hadoop.ParquetOutputCommitter') \
    .config('mapreduce.fileoutputcommitter.algorithm.version', '2') \
    .getOrCreate()

print('Master:', spark.sparkContext.master)
print('Spark version:', spark.version)
```

```
Master: yarn
Spark version: 3.5.0
```

```
26/05/18 07:48:12 WARN SparkSession: Using an existing Spark session; only runtime S
QL configurations will take effect.
```

In [3]:
```
V3_PATH = '/user/s3348393/main/preprocessing/v3/parquet'

ELECTION_DATE = pd.Timestamp('2022-05-21')
WINDOW_START  = pd.Timestamp('2021-11-21')
WINDOW_END    = pd.Timestamp('2022-11-21')

# Consistent palette across every chart in this notebook so the same advertiser
# type reads the same colour everywhere.
GROUP_COLORS = {
    'candidate':           '#1f77b4',   # blue
    'party_org':           '#ff7f0e',   # orange
    'climate':             '#2ca02c',   # green
    'humanitarian_rights': '#d62728',   # red
    'political_advocacy':  '#9467bd',   # purple
    'cost_of_living':      '#8c564b',   # brown
    'mixed':               '#bcbd22',   # olive
}
```

load v3, drop candidates and parties, tag group and pre/post period, cache.

In [4]:
```
v3 = spark.read.parquet(V3_PATH)

# Drop candidate and party_org — focus on advocacy ecosystem only.
v3 = v3.filter(col('match_type').isNull())

# Use category as the group label (climate / humanitarian_rights / political_advoca
v3 = v3.withColumn('group', col('category'))

# Tag pre/post election by ad creation date.
v3 = v3.withColumn(
    'period',
    when(col('ad_creation_date') < lit('2022-05-21'), 'pre')
    .otherwise('post')
)

v3 = v3.cache()
print(f'Advocacy ads in v3: {v3.count():,}')
print('\nGroup distribution:')
v3.groupBy('group').agg(
    spark_count('*').alias('ads'),
    spark_sum('spend_mid').alias('total_spend'),
).orderBy(desc('total_spend')).show(truncate=False)
```

```
26/05/18 07:48:20 WARN SparkStringUtils: Truncated the string representation of a pl
an since it was too large. This behavior can be adjusted by setting 'spark.sql.debu
g.maxToStringFields'.

Advocacy ads in v3: 94,358

Group distribution:
+-------------------+-----+-----------+
|group              |ads  |total_spend|
+-------------------+-----+-----------+
|political_advocacy |34108|7886996.0  |
|climate            |28981|7201909.5  |
|mixed              |15834|3498133.0  |
|humanitarian_rights|9076 |3236462.0  |
|cost_of_living     |6359 |2267220.5  |
+-------------------+-----+-----------+
```

top 20 bylines by pre-election spend, with post spend and modal group. banding happens later in cell 11.

In [5]:
```
# per-byline pre and post spend totals.
byline_spend = (v3
    .filter(col('spend_mid').isNotNull() & col('bylines').isNotNull())
    .groupBy('bylines')
    .pivot('period', ['pre', 'post'])
    .agg(spark_sum('spend_mid'))
).na.fill(0)

# modal group per byline.
group_per_byline = (v3.filter(col('bylines').isNotNull())
    .groupBy('bylines', 'group').count()
```

```
    .toPandas())

primary_group = (group_per_byline
    .sort_values(['bylines', 'count'], ascending=[True, False])
    .drop_duplicates('bylines', keep='first')
    [['bylines', 'group']])

spend_pdf = byline_spend.toPandas()
spend_pdf.columns = ['bylines', 'pre_spend', 'post_spend']
spend_pdf = spend_pdf.merge(primary_group, on='bylines', how='left')
spend_pdf['total_spend'] = spend_pdf['pre_spend'] + spend_pdf['post_spend']

top20 = spend_pdf.sort_values('pre_spend', ascending=False).head(20).reset_index(dr
top20[['bylines', 'group', 'pre_spend', 'post_spend', 'total_spend']]
```

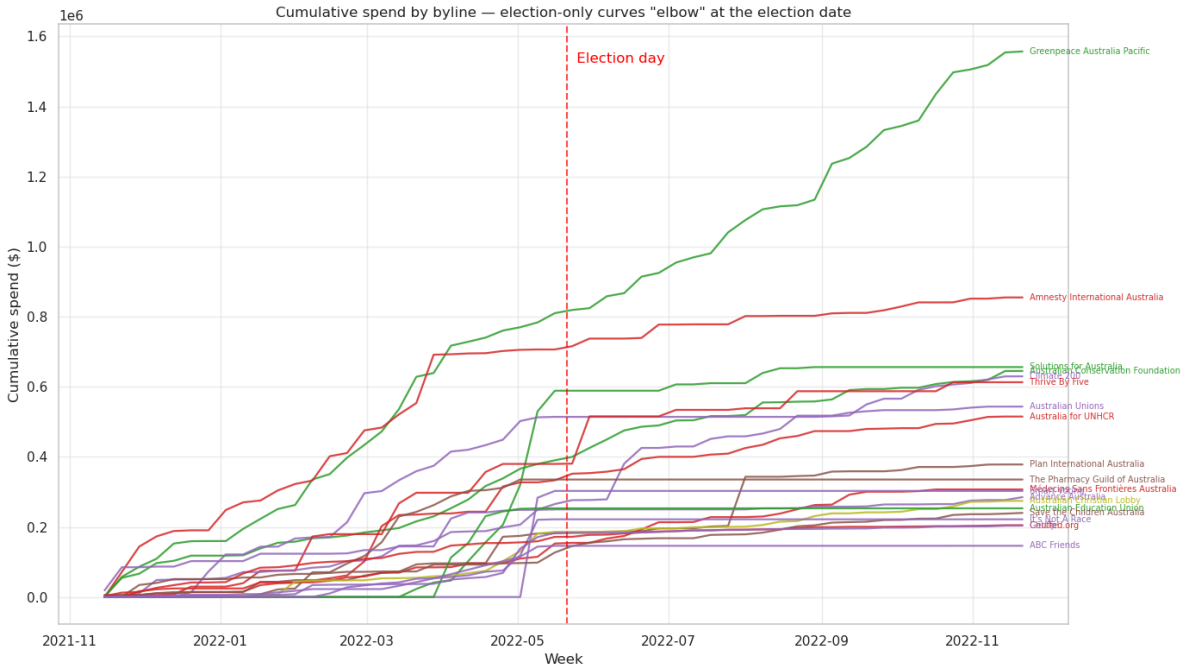
Out[5]:

	bylines	group	pre_spend	post_spend	total_spend
0	Greenpeace Australia Pacific	climate	809723.0	747046.5	1556769.5
1	Amnesty International Australia	humanitarian_rights	706331.5	148487.5	854819.0
2	Solutions for Australia	climate	588735.5	67405.0	656140.5
3	Climate 200	political_advocacy	514056.5	115618.0	629674.5
4	Australian Conservation Foundation	climate	387769.0	257216.5	644985.5
5	Thrive By Five	humanitarian_rights	379709.0	233141.0	612850.0
6	The Pharmacy Guild of Australia	cost_of_living	335291.5	0.0	335291.5
7	Australia for UNHCR	humanitarian_rights	332935.5	181999.0	514934.5
8	Smart Voting	political_advocacy	302671.5	0.0	302671.5
9	Australian Unions	political_advocacy	262517.0	280910.5	543427.5
10	Australian Education Union	climate	253022.5	0.0	253022.5
11	Advance Australia	political_advocacy	249974.5	34919.0	284893.5
12	It's Not A Race	political_advocacy	221677.5	0.0	221677.5
13	Australian Christian Lobby	mixed	185889.5	88367.0	274256.5
14	GetUp!	political_advocacy	182691.0	21147.0	203838.0
15	Plan International Australia	cost_of_living	181586.5	196615.0	378201.5
16	Chuffed.org	humanitarian_rights	171455.5	33299.5	204755.0
17	Médecins Sans Frontières Australia	humanitarian_rights	149037.0	157878.5	306915.5
18	ABC Friends	political_advocacy	146018.5	49.5	146068.0
19	Save the Children Australia	cost_of_living	127058.5	112923.5	239982.0

cumulative spend per byline. election-only ones elbow at election day, ongoing ones climb steady.

In [6]:
```
top20_bylines = top20['bylines'].tolist()

# Weekly aggregation per byline — cached on driver as pandas, reused by every chart
weekly = (v3
    .filter(col('bylines').isin(top20_bylines) & col('spend_mid').isNotNull())
    .withColumn('week', date_trunc('week', 'ad_creation_date'))
    .groupBy('week', 'bylines', 'group')
    .agg(spark_sum('spend_mid').alias('spend'))
    .orderBy('week')
    .toPandas())
```

```
# Pivot to byline × week (used by stacked/cumulative/heatmap charts below).
pivot = weekly.pivot_table(index='week', columns='bylines', values='spend', aggfunc
col_order = pivot.sum().sort_values(ascending=False).index.tolist()
pivot = pivot[col_order]
byline_group = dict(zip(top20['bylines'], top20['group']))

# Cumulative spend per byline — the elbow tells the story.
cumulative = pivot.cumsum()

fig, ax = plt.subplots(figsize=(14, 8))
for byline in cumulative.columns:
    grp = byline_group.get(byline, 'unknown')
    color = GROUP_COLORS.get(grp, '#777777')
    ax.plot(cumulative.index, cumulative[byline],
            color=color, alpha=0.85, linewidth=1.6)
    # End-of-line label
    end_val = cumulative[byline].iloc[-1]
    ax.text(cumulative.index[-1] + pd.Timedelta(days=2),
            end_val, ' ' + byline[:35],
            fontsize=7, va='center', color=color, alpha=0.95)

ax.axvline(ELECTION_DATE, linestyle='--', color='red', alpha=0.7, linewidth=1.5)
ax.text(ELECTION_DATE + pd.Timedelta(days=2), ax.get_ylim()[1] * 0.95,
        ' Election day', color='red', va='top')
ax.set_title('Cumulative spend by byline — election-only curves "elbow" at the elec
ax.set_xlabel('Week')
ax.set_ylabel('Cumulative spend ($)')
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



smooth the weekly series and compute persistence, centroid and EC off it, then assign each byline to a band.

In [7]:
```python
import numpy as np

HALF_WIDTH_WEEKS = 26
CENTROID_WINDOW  = 16
PERSIST_LOW      = 0.30
PERSIST_HIGH     = 0.80
SMOOTH_WINDOW    = 4

# collapse duplicate (bylines, week) rows that arise when a byline appears in
# more than one group, so the multi-index reindex below is unique.
weekly = (weekly
    .groupby(['bylines', 'week'], as_index=False)['spend'].sum())

# rebuild on a complete week index per byline so the rolling window sees
# zero-spend weeks instead of skipping them.
all_weeks = pd.date_range(weekly['week'].min(), weekly['week'].max(), freq='W-MON')
bylines_in_weekly = weekly['bylines'].unique()
full_index = pd.MultiIndex.from_product([bylines_in_weekly, all_weeks],
                                        names=['bylines', 'week'])
weekly = (weekly.set_index(['bylines', 'week'])
                .reindex(full_index, fill_value=0.0)
                .reset_index()
                .sort_values(['bylines', 'week']))

# re-attach group from top20.
weekly = weekly.merge(top20[['bylines', 'group']], on='bylines', how='left')

# 4-week centred rolling mean per byline, with a hard boundary at election day —
# pre and post are smoothed separately so a cliff-edged advertiser's final
# pre-election spike doesn't bleed forward into the post-election weeks.
weekly['is_post'] = weekly['week'] >= ELECTION_DATE
weekly['spend_smooth'] = (weekly.groupby(['bylines', 'is_post'])['spend']
    .transform(lambda s: s.rolling(window=SMOOTH_WINDOW, center=True, min_periods=1

# triangle weights for the EC index.
weekly['weeks_from_election'] = (weekly['week'] - ELECTION_DATE).dt.days / 7
weekly['weight'] = np.clip(
    1.0 - np.abs(weekly['weeks_from_election']) / HALF_WIDTH_WEEKS,
    a_min=0.0, a_max=None,
)
weekly['weighted_spend_smooth'] = weekly['weight'] * weekly['spend_smooth']

# recompute persistence, EC, centroid from the smoothed series.
def _summary(x):
    total = x['spend_smooth'].sum()
    if total <= 0:
        return pd.Series({'pre_spend_s': 0.0, 'post_spend_s': 0.0,
                          'ec_index': 0.0, 'centroid_weeks': 0.0})
    pre = x.loc[~x['is_post'], 'spend_smooth'].sum()
    post = x.loc[x['is_post'], 'spend_smooth'].sum()
    return pd.Series({
        'pre_spend_s':    pre,
        'post_spend_s':   post,
        'ec_index':       x['weighted_spend_smooth'].sum() / total,
        'centroid_weeks': (x['weeks_from_election'] * x['spend_smooth']).sum() / to
```

```
    })

smoothed = weekly.groupby('bylines').apply(_summary).reset_index()
smoothed['persistence'] = smoothed['post_spend_s'] / smoothed['pre_spend_s'].replac

for c in ('persistence', 'ec_index', 'centroid_weeks', 'band'):
    if c in top20.columns:
        top20 = top20.drop(columns=[c])
top20 = top20.merge(
    smoothed[['bylines', 'persistence', 'ec_index', 'centroid_weeks']],
    on='bylines', how='left',
)

def _classify(row):
    p = row['persistence'] if pd.notna(row['persistence']) else 0.0
    c = abs(row['centroid_weeks']) if pd.notna(row['centroid_weeks']) else 0.0
    if p < PERSIST_LOW and c > CENTROID_WINDOW:
        return 'off_campaign'
    if p < PERSIST_LOW:
        return 'election_only'
    if p > PERSIST_HIGH:
        return 'ongoing'
    return 'transitional'

top20['band'] = top20.apply(_classify, axis=1)
top20['band'] = pd.Categorical(
    top20['band'],
    categories=['election_only', 'transitional', 'ongoing', 'off_campaign'],
    ordered=True,
)

display_cols = ['bylines', 'group', 'pre_spend', 'post_spend',
                'persistence', 'ec_index', 'centroid_weeks', 'band']
top20.sort_values(['band', 'total_spend'], ascending=[True, False])[display_cols]
```

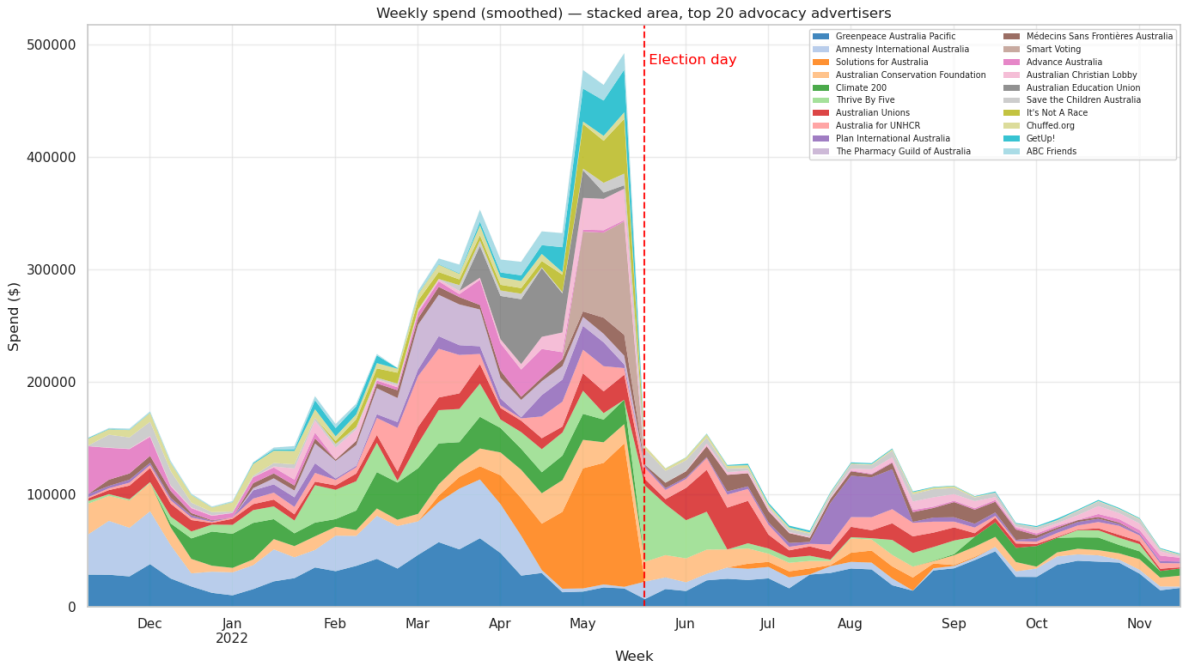
Out[7]:

	bylines	group	pre_spend	post_spend	persistence	ec_index	centroi
1	Amnesty International Australia	humanitarian_rights	706331.5	148487.5	0.211750	0.441121	-1(
2	Solutions for Australia	climate	588735.5	67405.0	0.125119	0.852070	-1
3	Climate 200	political_advocacy	514056.5	115618.0	0.222743	0.499911	-5
6	The Pharmacy Guild of Australia	cost_of_living	335291.5	0.0	0.000000	0.629485	-9
8	Smart Voting	political_advocacy	302671.5	0.0	0.000000	0.938735	-1
11	Advance Australia	political_advocacy	249974.5	34919.0	0.115904	0.374626	-12
10	Australian Education Union	climate	253022.5	0.0	0.000000	0.802521	-5
12	It's Not A Race	political_advocacy	221677.5	0.0	0.000000	0.828475	-4
16	Chuffed.org	humanitarian_rights	171455.5	33299.5	0.205657	0.471631	-1(
14	GetUp!	political_advocacy	182691.0	21147.0	0.116235	0.767506	-3
18	ABC Friends	political_advocacy	146018.5	49.5	0.000350	0.740500	-6
4	Australian Conservation Foundation	climate	387769.0	257216.5	0.648636	0.538804	-3
5	Thrive By Five	humanitarian_rights	379709.0	233141.0	0.731790	0.648939	-3
7	Australia for UNHCR	humanitarian_rights	332935.5	181999.0	0.556775	0.610156	-2
13	Australian Christian Lobby	mixed	185889.5	88367.0	0.493748	0.629040	(
0	Greenpeace Australia Pacific	climate	809723.0	747046.5	0.910608	0.466242	(
9	Australian Unions	political_advocacy	262517.0	280910.5	1.100284	0.635139	-1
15	Plan International Australia	cost_of_living	181586.5	196615.0	1.090892	0.598608	1

	bylines	group	pre_spend	post_spend	persistence	ec_index	centroi
17	Médecins Sans Frontières Australia	humanitarian_rights	149037.0	157878.5	1.123627	0.588786	(
19	Save the Children Australia	cost_of_living	127058.5	112923.5	0.992619	0.505566	-2

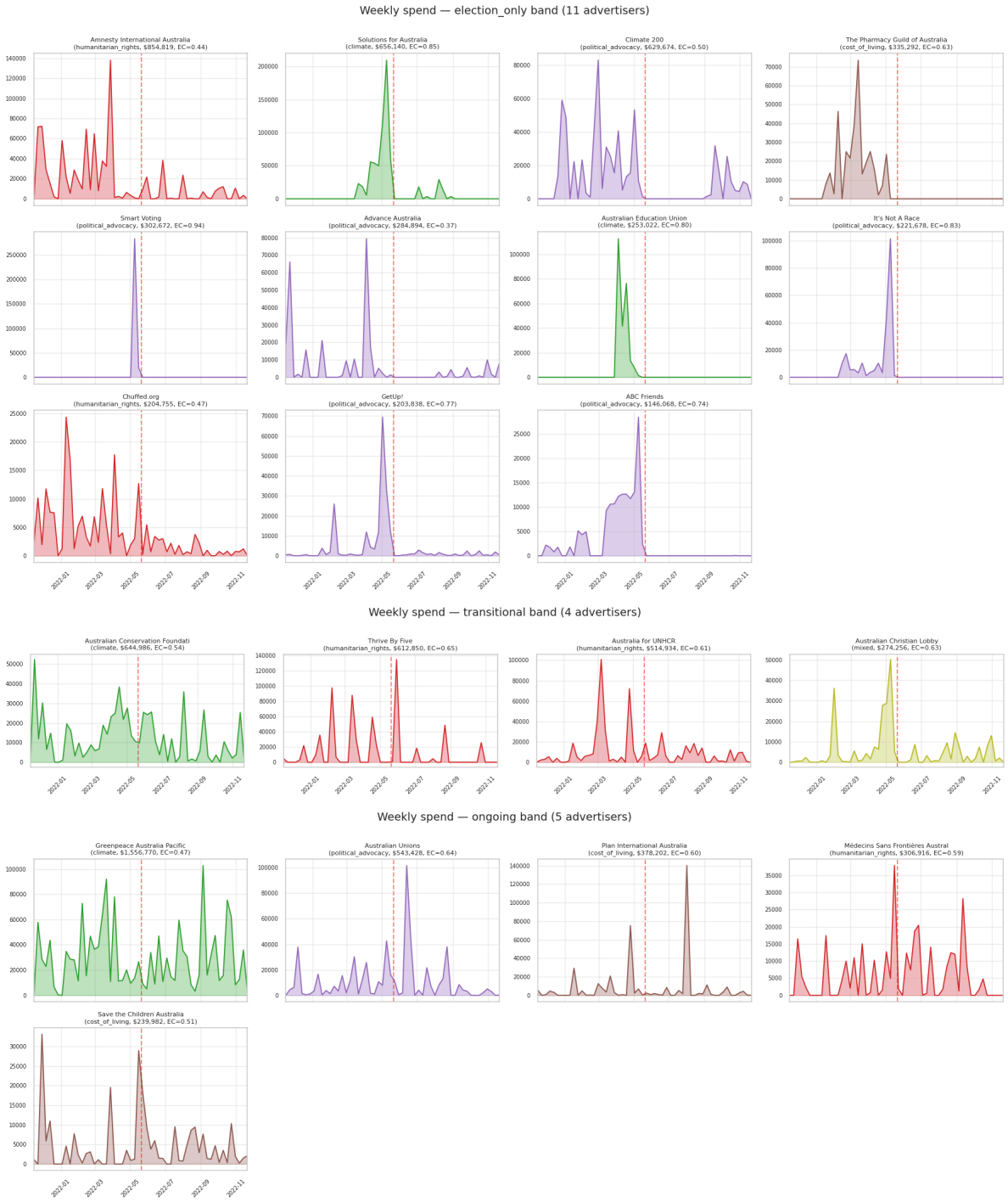
stacked area of weekly spend across the top 20.

In [8]:
```
pivot_smooth = (weekly.pivot_table(index='week', columns='bylines',
                                   values='spend_smooth', aggfunc='sum')
                      .fillna(0)
                      [pivot.columns])  # same byline order as the raw pivot

fig, ax = plt.subplots(figsize=(14, 8))
pivot_smooth.plot.area(ax=ax, alpha=0.85, linewidth=0, colormap='tab20')
ax.axvline(ELECTION_DATE, linestyle='--', color='red', alpha=0.9, linewidth=1.5)
ax.text(ELECTION_DATE + pd.Timedelta(days=2), ax.get_ylim()[1] * 0.95,
        ' Election day', color='red', va='top')
ax.set_title('Weekly spend (smoothed) — stacked area, top 20 advocacy advertisers')
ax.set_ylabel('Spend ($)')
ax.set_xlabel('Week')
ax.legend(loc='upper right', fontsize=7, ncol=2)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



small multiples, one panel per byline, grouped by band. shared x, independent y.

In [9]:
```
import math

# Full time window for sharex.
x_min = weekly['week'].min()
x_max = weekly['week'].max()

for band_name in ['election_only', 'transitional', 'ongoing', 'off_campaign']:
    band_data = top20[top20['band'] == band_name].sort_values('total_spend', ascend
    n = len(band_data)
    if n == 0:
        print(f'No advertisers in band "{band_name}".\n')
        continue

    ncols = min(4, n)
    nrows = math.ceil(n / ncols)

    fig, axes = plt.subplots(nrows, ncols, figsize=(18, 3.5 * nrows),
                             squeeze=False, sharex=True, sharey=False)
    axes = axes.flatten()

    for i, row in band_data.iterrows():
        ax = axes[i]
        byline = row['bylines']
        grp = row['group']
        color = GROUP_COLORS.get(grp, '#777777')
        sub = weekly[weekly['bylines'] == byline].sort_values('week')
        ax.plot(sub['week'], sub['spend'], color=color, linewidth=1.3)
        ax.fill_between(sub['week'], sub['spend'], color=color, alpha=0.3)
        ax.axvline(ELECTION_DATE, linestyle='--', color='red', alpha=0.5)
        ax.set_title(f'{byline[:32]}\n({grp}, ${row["total_spend"]:,.0f}, EC={row["
        ax.tick_params(axis='x', rotation=45, labelsize=7)
        ax.tick_params(axis='y', labelsize=7)
        ax.grid(alpha=0.3)
        ax.set_xlim(x_min, x_max)

    for j in range(n, len(axes)):
        axes[j].set_visible(False)

    fig.suptitle(f'Weekly spend — {band_name} band ({n} advertisers)', fontsize=14,
    plt.tight_layout()
    plt.show()
```



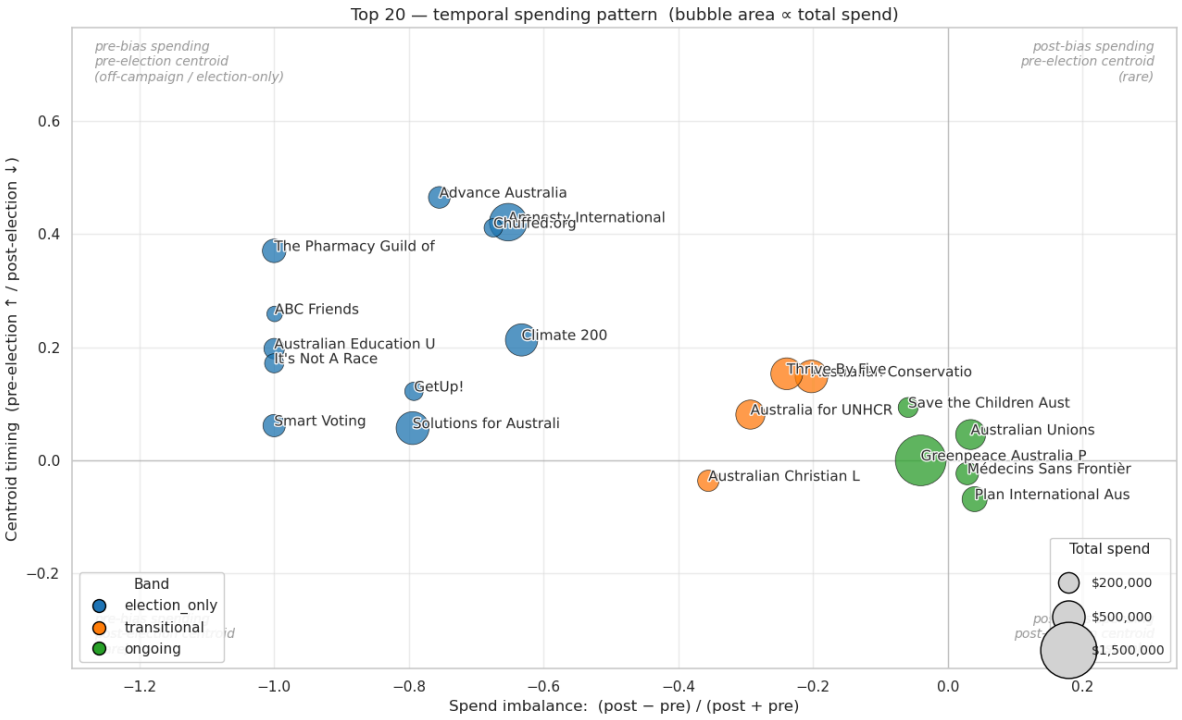

```
No advertisers in band "off_campaign".
```

quadrant scatter. x is spend imbalance (post − pre over total), y is centroid timing. bubble area is total spend, colour is band.

In [10]:
```
# temporal-pattern quadrant — spend imbalance (x) × centroid timing (y).

BAND_COLORS = {
    'election_only':    '#1f77b4',   # blue
    'transitional':     '#ff7f0e',   # orange
    'ongoing':          '#2ca02c',   # green
```

```
}

# axes — signed, normalised to ±1.
total = (top20['pre_spend'] + top20['post_spend']).replace(0, 1)
top20['x_imbalance'] = (top20['post_spend'] - top20['pre_spend']) / total
top20['y_centroid']  = (-top20['centroid_weeks'] / 26).clip(-1, 1)   # +ve = pre, -


def plot_quadrant(data, size_col, size_divisor, ref_sizes, size_legend_title,
                  size_label_fmt, chart_title):
    fig, ax = plt.subplots(figsize=(13, 8))

    ax.axhline(0, color='gray', linestyle='-', alpha=0.4, linewidth=1)
    ax.axvline(0, color='gray', linestyle='-', alpha=0.4, linewidth=1)

    sizes = data[size_col] / size_divisor
    colors = data['band'].astype(str).map(BAND_COLORS)
    ax.scatter(
        data['x_imbalance'], data['y_centroid'],
        s=sizes, c=colors, alpha=0.75,
        edgecolors='black', linewidths=0.5, zorder=3,
    )

    # labels — adjustText pushes them off the bubbles with leader lines.
    texts = [
        ax.text(row['x_imbalance'], row['y_centroid'], row['bylines'][:22],
                fontsize=11, alpha=0.95, zorder=5)
        for _, row in data.iterrows()
    ]

    try:
        from adjustText import adjust_text
        adjust_text(
            texts, ax=ax,
            arrowprops=dict(arrowstyle='-', color='gray', alpha=0.5, lw=0.6),
            expand_points=(2.0, 2.0),
            expand_text=(1.4, 1.6),
            force_points=(0.6, 0.8),
            force_text=(0.7, 0.9),
            lim=500,
            only_move={'points': 'xy', 'text': 'xy', 'objects': 'xy'},
        )
    except ImportError:
        from matplotlib import patheffects
        for t in texts:
            t.set_path_effects([patheffects.withStroke(linewidth=2.0, foreground='w

    # quadrant labels positioned in axes coordinates so they sit in the corners
    # regardless of the data range.
    ax.text(0.02, 0.98, 'pre-bias spending\npre-election centroid\n(off-campaign /
            transform=ax.transAxes, ha='left',  va='top',
            fontsize=10, alpha=0.45, fontstyle='italic')
    ax.text(0.98, 0.98, 'post-bias spending\npre-election centroid\n(rare)',
            transform=ax.transAxes, ha='right', va='top',
            fontsize=10, alpha=0.45, fontstyle='italic')
    ax.text(0.02, 0.02, 'pre-bias spending\npost-election centroid\n(rare)',
```

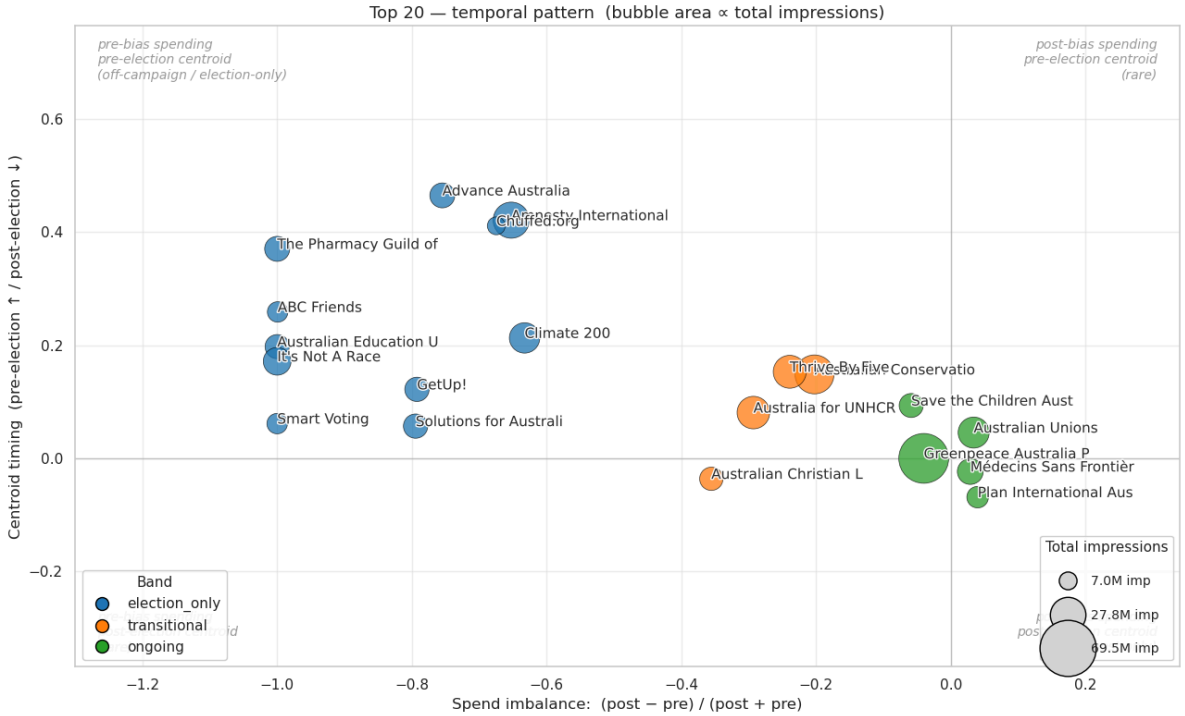
```
            transform=ax.transAxes, ha='left',  va='bottom',
            fontsize=10, alpha=0.45, fontstyle='italic')
    ax.text(0.98, 0.02, 'post-bias spending\npost-election centroid\n(post-only)',
            transform=ax.transAxes, ha='right', va='bottom',
            fontsize=10, alpha=0.45, fontstyle='italic')

    from matplotlib.lines import Line2D
    band_handles = [
        Line2D([0], [0], marker='o', linestyle='', color='w',
               markerfacecolor=c, markeredgecolor='black', markersize=10, label=b)
        for b, c in BAND_COLORS.items()
    ]
    # Line2D markersize is diameter in points → 2 * sqrt(area / pi).
    size_handles = [
        Line2D([0], [0], marker='o', linestyle='', color='w',
               markerfacecolor='lightgray', markeredgecolor='black',
               markersize=2 * np.sqrt((s / size_divisor) / np.pi),
               label=size_label_fmt(s))
        for s in ref_sizes
    ]
    band_legend = ax.legend(handles=band_handles, loc='lower left', fontsize=11,
                            framealpha=0.9, title='Band', title_fontsize=11)
    ax.add_artist(band_legend)
    ax.legend(handles=size_handles, loc='lower right', fontsize=10, framealpha=0.9,
              title=size_legend_title, title_fontsize=11, labelspacing=1.6)

    # Auto-fit axes to data with 10% padding on each side. Ensure 0 stays in
    # range so the quadrant cross-hair (axhline / axvline) remains visible.
    x_min, x_max = data['x_imbalance'].min(), data['x_imbalance'].max()
    y_min, y_max = data['y_centroid'].min(), data['y_centroid'].max()
    x_pad = max((x_max - x_min) * 0.10, 0.3)
    y_pad = max((y_max - y_min) * 0.10, 0.3)
    ax.set_xlim(min(x_min - x_pad, -0.05), max(x_max + x_pad, 0.05))
    ax.set_ylim(min(y_min - y_pad, -0.05), max(y_max + y_pad, 0.05))

    ax.set_xlabel('Spend imbalance:  (post − pre) / (post + pre)', fontsize=12)
    ax.set_ylabel('Centroid timing  (pre-election ↑ / post-election ↓)', fontsize=1
    ax.set_title(chart_title, fontsize=13)
    ax.tick_params(axis='both', labelsize=11)
    ax.grid(alpha=0.3, zorder=0)
    plt.tight_layout()
    plt.show()


plot_quadrant(
    top20,
    size_col='total_spend',
    size_divisor=1000,
    ref_sizes=[200_000, 500_000, 1_500_000],
    size_legend_title='Total spend',
    size_label_fmt=lambda s: f'${s:,.0f}',
    chart_title='Top 20 — temporal spending pattern  (bubble area ∝ total spend)',
)
```



same quadrant, bubble area is impressions instead of spend. comparison table at the end for spend rank vs impression rank.

In [11]:
```
# aggregate total impressions per top-20 byline from v3.
imp_pdf = (v3
    .filter(col('bylines').isin(top20_bylines) & col('impressions_mid').isNotNull())
    .groupBy('bylines')
    .agg(spark_sum('impressions_mid').alias('total_impressions'))
    .toPandas())

top20_imp = top20.merge(imp_pdf, on='bylines', how='left')
top20_imp['total_impressions'] = top20_imp['total_impressions'].fillna(0)

# scale so the biggest bubble is similar to the spend chart.
imp_divisor = top20_imp['total_impressions'].max() / 1500
imp_max = top20_imp['total_impressions'].max()

plot_quadrant(
    top20_imp,
    size_col='total_impressions',
    size_divisor=imp_divisor,
    ref_sizes=[imp_max * 0.1, imp_max * 0.4, imp_max],
    size_legend_title='Total impressions',
    size_label_fmt=lambda s: f'{s/1e6:,.1f}M imp',
    chart_title='Top 20 — temporal pattern  (bubble area ∝ total impressions)',
)

# spend rank vs impression rank.
cmp = top20_imp[['bylines', 'total_spend', 'total_impressions']].copy()
cmp['spend_rank'] = cmp['total_spend'].rank(ascending=False).astype(int)
cmp['imp_rank']   = cmp['total_impressions'].rank(ascending=False).astype(int)
cmp['cpm']        = cmp['total_spend'] / (cmp['total_impressions'] / 1000).replace(
```

```
cmp['rank_shift'] = cmp['spend_rank'] - cmp['imp_rank']
cmp.sort_values('rank_shift')[['bylines', 'spend_rank', 'imp_rank', 'rank_shift', '
```



Out[11]:

	bylines	spend_rank	imp_rank	rank_shift	cpm
2	Solutions for Australia	3	13	-10	39.626580
15	Plan International Australia	9	17	-8	29.110779
8	Smart Voting	12	18	-6	25.727588
3	Climate 200	5	7	-2	24.309106
13	Australian Christian Lobby	14	16	-2	18.043895
16	Chuffed.org	18	20	-2	22.744604
1	Amnesty International Australia	2	3	-1	23.373674
6	The Pharmacy Guild of Australia	10	10	0	19.130007
0	Greenpeace Australia Pacific	1	1	0	22.397573
10	Australian Education Union	15	14	1	15.478342
18	ABC Friends	20	19	1	12.459878
9	Australian Unions	7	6	1	20.120437
19	Save the Children Australia	16	15	1	14.871554
11	Advance Australia	13	11	2	16.366423
4	Australian Conservation Foundation	4	2	2	15.169744
5	Thrive By Five	6	4	2	20.135397
17	Médecins Sans Frontières Australia	11	9	2	16.458126
7	Australia for UNHCR	8	5	3	17.318131
14	GetUp!	19	12	7	12.306243
12	It's Not A Race	17	8	9	10.287720

roll up to bands. count, total spend, mean persistence and EC per band.

In [12]:
```
band_summary = top20.groupby('band', observed=True).agg(
    n_advertisers=('bylines', 'count'),
    pre_spend=('pre_spend', 'sum'),
    post_spend=('post_spend', 'sum'),
    total_spend=('total_spend', 'sum'),
    mean_persistence=('persistence', 'mean'),
    mean_ec=('ec_index', 'mean'),
).reset_index()

band_summary['total_share'] = band_summary['total_spend'] / band_summary['total_spe

band_summary
```

Out[12]:

	band	n_advertisers	pre_spend	post_spend	total_spend	mean_persistence	mean
0	election_only	11	3671926.0	420925.5	4092851.5	0.090705	0.667
1	transitional	4	1286303.0	760723.5	2047026.5	0.607737	0.606
2	ongoing	5	1529922.0	1495374.0	3025296.0	1.043606	0.558

the advocacy ecosystem splits into four patterns, with election_only the biggest by count and greenpeace dominating the ongoing band. shell only shows up as off_campaign once you combine persistence with centroid.

In [13]:
```
BAND_EXPORT_PATH = '../data/top20_bands.csv'

export_df = (top20[['bylines', 'band', 'group', 'pre_spend', 'post_spend',
                    'total_spend', 'persistence', 'centroid_weeks', 'ec_index']]
             .sort_values(['band', 'total_spend'], ascending=[True, False])
             .reset_index(drop=True))
export_df['band'] = export_df['band'].astype(str)

export_df.to_csv(BAND_EXPORT_PATH, index=False)
print(f'Wrote {BAND_EXPORT_PATH}')
display(export_df)
```

```
Wrote ../data/top20_bands.csv
```

	bylines	band	group	pre_spend	post_spend	total_spend	persis
0	Amnesty International Australia	election_only	humanitarian_rights	706331.5	148487.5	854819.0	0.2
1	Solutions for Australia	election_only	climate	588735.5	67405.0	656140.5	0.1
2	Climate 200	election_only	political_advocacy	514056.5	115618.0	629674.5	0.2
3	The Pharmacy Guild of Australia	election_only	cost_of_living	335291.5	0.0	335291.5	0.0
4	Smart Voting	election_only	political_advocacy	302671.5	0.0	302671.5	0.0
5	Advance Australia	election_only	political_advocacy	249974.5	34919.0	284893.5	0.1
6	Australian Education Union	election_only	climate	253022.5	0.0	253022.5	0.0
7	It's Not A Race	election_only	political_advocacy	221677.5	0.0	221677.5	0.0
8	Chuffed.org	election_only	humanitarian_rights	171455.5	33299.5	204755.0	0.2
9	GetUp!	election_only	political_advocacy	182691.0	21147.0	203838.0	0.1
10	ABC Friends	election_only	political_advocacy	146018.5	49.5	146068.0	0.0
11	Australian Conservation Foundation	transitional	climate	387769.0	257216.5	644985.5	0.6
12	Thrive By Five	transitional	humanitarian_rights	379709.0	233141.0	612850.0	0.7
13	Australia for UNHCR	transitional	humanitarian_rights	332935.5	181999.0	514934.5	0.5
14	Australian Christian Lobby	transitional	mixed	185889.5	88367.0	274256.5	0.4
15	Greenpeace Australia Pacific	ongoing	climate	809723.0	747046.5	1556769.5	0.9
16	Australian Unions	ongoing	political_advocacy	262517.0	280910.5	543427.5	1.1
17	Plan International Australia	ongoing	cost_of_living	181586.5	196615.0	378201.5	1.0

	bylines	band	group	pre_spend	post_spend	total_spend	persi
18	Médecins Sans Frontières Australia	ongoing	humanitarian_rights	149037.0	157878.5	306915.5	1.1
19	Save the Children Australia	ongoing	cost_of_living	127058.5	112923.5	239982.0	0.9

export top 20 bylines + band as csv for nb 06 to consume.

for each of the three main bands (election_only, transitional, ongoing), show every advertiser and their top 5 keywords from ad body + title text. regex tokenise, lowercase, drop stopwords.

In [14]:
```
from pyspark.sql.functions import (
    row_number, explode, lower, regexp_replace, split, length,
    concat_ws, array_join, collect_list,
)
from pyspark.sql.window import Window
from pyspark.ml.feature import StopWordsRemover

# english stopwords plus a few domain words that don't differentiate australian
# political advocacy bylines (everyone says 'australia', 'sign', etc).
STOP = set(StopWordsRemover.loadDefaultStopWords('english')) | {
    'australia', 'australian', 'australians', 'sign', 'petition', 'please',
    'today', 'help', 'support', 'make', 'get', 'one', 'us', 'go',
    'http', 'https', 'www', 'com', 'au',
}

top_keywords_by_band = {}
for band_name in ['election_only', 'transitional', 'ongoing']:
    band_bylines = top20.loc[top20['band'] == band_name, 'bylines'].tolist()
    if not band_bylines:
        top_keywords_by_band[band_name] = pd.DataFrame()
        continue

    # combine body + title arrays into one text blob per ad, tokenise.
    tokens = (v3
        .filter(col('bylines').isin(band_bylines))
        .withColumn('text', concat_ws(' ',
            array_join('creative_bodies', ' '),
            array_join('creative_link_titles', ' ')))
        .withColumn('text', regexp_replace(lower('text'), r'[^a-z\s]', ' '))
        .withColumn('token', explode(split(col('text'), r'\s+')))
        .filter(length('token') > 2)
        .filter(~col('token').isin(list(STOP))))

    counts = tokens.groupBy('bylines', 'token').count()
    w = Window.partitionBy('bylines').orderBy(desc('count'), 'token')
    top5 = (counts
```

```
        .withColumn('rn', row_number().over(w))
        .filter(col('rn') <= 5)
        .orderBy('bylines', 'rn')
        .groupBy('bylines')
        .agg(collect_list('token').alias('top_keywords'))
        .toPandas())

    # preserve band ordering by total spend.
    order = (top20.loc[top20['band'] == band_name]
                  .sort_values('total_spend', ascending=False)['bylines']
                  .tolist())
    top5['_order'] = top5['bylines'].map({b: i for i, b in enumerate(order)})
    top5 = top5.sort_values('_order').drop(columns='_order').reset_index(drop=True)
    top5['top_keywords'] = top5['top_keywords'].apply(lambda xs: ', '.join(xs))
    top_keywords_by_band[band_name] = top5

for band_name, df in top_keywords_by_band.items():
    print(f'\n=== {band_name} ({len(df)} advertisers) ===')
    display(df)
```

```
=== election_only (11 advertisers) ===
```

	bylines	top_keywords
0	Amnesty International Australia	people, quiz, take, rights, free
1	Solutions for Australia	energy, power, government, federal, jobs
2	Climate 200	climate, vote, independent, party, independents
3	The Pharmacy Guild of Australia	medicines, cost, prescription, visit, choose
4	Smart Voting	last, put, liberals, source, afford
5	Advance Australia	vaccine, freedom, fair, show, choice
6	Australian Education Union	morrison, government, vote, schools, tafe
7	It's Not A Race	auspol, climate, climateaction, change, right
8	Chuffed.org	people, aboriginal, chip, first, nations
9	GetUp!	vote, election, government, climate, morrison
10	ABC Friends	abc, save, fully, funded, independent

```
=== transitional (4 advertisers) ===
```

	bylines	top_keywords
0	Australian Conservation Foundation	protect, climb, climate, laws, nature
1	Thrive By Five	early, learning, educators, childcare, families
2	Australia for UNHCR	refugees, ukraine, donate, unhcr, winter
3	Australian Christian Lobby	truth, party, religious, schools, faith

```
=== ongoing (5 advertisers) ===
```

	bylines	top_keywords
0	Greenpeace Australia Pacific	woodside, gas, climate, tell, oceans
1	Australian Unions	union, free, morrison, join, joining
2	Plan International Australia	girls, add, fgm, name, every
3	Médecins Sans Frontières Australia	medical, people, care, teams, need
4	Save the Children Australia	children, child, end, protect, take

`06_election_content_classifier.ipynb`

logistic regression over ad text to score each ad's probability of being political content. labels come from the curated hashtag csvs from nb 04b — ads carrying a known-political hashtag are positives, ads carrying a known-non-political hashtag are negatives. hashtags themselves are stripped from the text before vectorising so the model has to learn from the surrounding prose, not the labels. then score every v3 advocacy ad and aggregate per advertiser per week to look for pivot patterns.

In [18]:
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, when, lit, lower, regexp_replace, concat_ws, array_join, broadcast,
    sum as spark_sum, count as spark_count, date_trunc, desc, abs as spark_abs,
    mean as spark_mean,
)
from pyspark.sql.window import Window
from pyspark.ml import Pipeline
from pyspark.ml.feature import (
    RegexTokenizer, StopWordsRemover, CountVectorizer, IDF,
)
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator, MulticlassClassifi
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style='whitegrid', context='notebook')

spark = SparkSession.builder \
    .appName('FB_API_election_content_clf') \
    .config('spark.sql.parquet.output.committer.class',
            'org.apache.parquet.hadoop.ParquetOutputCommitter') \
    .config('mapreduce.fileoutputcommitter.algorithm.version', '2') \
    .getOrCreate()

print('Master:', spark.sparkContext.master)
print('Spark version:', spark.version)
```

```
Master: yarn
Spark version: 3.5.0
```

In [19]:
```
V3_PATH = '/user/s3348393/main/preprocessing/v3/parquet'
POLITICAL_CSV     = '../data/hashtags_political.csv'
NON_POLITICAL_CSV = '../data/hashtags_non_political.csv'
BANDS_CSV         = '../data/all_bylines_bands.csv'   # used for downstream compari
ELECTION_DATE = pd.Timestamp('2022-05-21')
WINDOW_START = pd.Timestamp('2021-11-21')
WINDOW_END   = pd.Timestamp('2022-11-21')

# hot zone is only used downstream for FP/FN checks and pivot detection.
HOT_ZONE_WEEKS = 4
hot_start = (ELECTION_DATE - pd.Timedelta(weeks=HOT_ZONE_WEEKS)).date().isoformat()
hot_end   = (ELECTION_DATE + pd.Timedelta(weeks=HOT_ZONE_WEEKS)).date().isoformat()
```

load v3 advocacy ads, detect political and non-political hashtag presence per ad, then strip the hashtags from the text so the model learns from the surrounding prose, not the labels. also attach band labels from nb 05 for downstream comparison.

In [20]:
```
import re

v3 = spark.read.parquet(V3_PATH).filter(col('match_type').isNull())

# combine body + title arrays into one raw text blob, lowercase.
v3 = v3.withColumn('raw_text', lower(concat_ws(' ',
    array_join('creative_bodies', ' '),
    array_join('creative_link_titles', ' '))))
v3 = v3.filter(col('raw_text').isNotNull() & (col('raw_text') != ''))

# load curated hashtag lists.
political_hashtags     = set(pd.read_csv(POLITICAL_CSV)['hashtag'].str.lower())
non_political_hashtags = set(pd.read_csv(NON_POLITICAL_CSV)['hashtag'].str.lower())
print(f'Political hashtags loaded:     {len(political_hashtags)}')
print(f'Non-political hashtags loaded: {len(non_political_hashtags)}')

def _make_hashtag_regex(tags):
    if not tags:
        return r'(?!.*)'  # never matches
    return r'#(' + '|'.join(re.escape(t) for t in tags) + r')\b'

pol_re = _make_hashtag_regex(political_hashtags)
non_re = _make_hashtag_regex(non_political_hashtags)

v3 = v3.withColumn('has_political_hashtag',     col('raw_text').rlike(pol_re))
v3 = v3.withColumn('has_non_political_hashtag', col('raw_text').rlike(non_re))

# strip all hashtags from text before further processing — the model must learn
# from the prose, not from the hashtags themselves (which are the labels).
v3 = v3.withColumn('text', regexp_replace(col('raw_text'), r'#\w+', ' '))
v3 = v3.withColumn('text', regexp_replace(col('text'), r'[^a-z\s]', ' '))

# attach band labels for downstream comparison only.
try:
    bands_pdf = pd.read_csv(BANDS_CSV)[['bylines', 'band']]
    bands_sdf = spark.createDataFrame(bands_pdf)
    v3 = v3.join(broadcast(bands_sdf), 'bylines', 'left')
    have_bands = True
except FileNotFoundError:
    v3 = v3.withColumn('band', lit(None).cast('string'))
    have_bands = False
    print('Bands csv not found — band column will be null.')

v3 = v3.cache()
print(f'\nAdvocacy ads with text: {v3.count():,}')
print('\nHashtag-tag flags:')
v3.groupBy('has_political_hashtag', 'has_non_political_hashtag').count().show()
```

```
Political hashtags loaded:     210
Non-political hashtags loaded: 286
```

```
Advocacy ads with text: 94,358

Hashtag-tag flags:
+--------------------+------------------------+-----+
|has_political_hashtag|has_non_political_hashtag|count|
+--------------------+------------------------+-----+
|                true|                   false| 2599|
|                true|                    true| 1074|
|               false|                   false|85423|
|               false|                    true| 5262|
+--------------------+------------------------+-----+
```

build the labelled training set. positive = ad contains at least one political hashtag and no non-political hashtag. negative = ad contains at least one non-political hashtag and no political hashtag. ads with mixed signal (both kinds of hashtag) or no hashtag at all are unlabelled — they get scored later at inference.

In [21]:
```
labelled = v3.withColumn('label',
    when( col('has_political_hashtag') & ~col('has_non_political_hashtag'), lit(1.0
    .when(~col('has_political_hashtag') &  col('has_non_political_hashtag'), lit(0.
    .otherwise(lit(None))
)

train_full = labelled.filter(col('label').isNotNull()).cache()

print('Class balance:')
train_full.groupBy('label').count().show()

print('Distinct positive-class bylines:',
      train_full.filter(col('label') == 1.0).select('bylines').distinct().count())
print('Distinct negative-class bylines:',
      train_full.filter(col('label') == 0.0).select('bylines').distinct().count())

print('\nPositive class — top contributing bylines:')
train_full.filter(col('label') == 1.0).groupBy('bylines').count() \
    .orderBy(desc('count')).show(10, truncate=False)

print('Negative class — top contributing bylines:')
train_full.filter(col('label') == 0.0).groupBy('bylines').count() \
    .orderBy(desc('count')).show(10, truncate=False)
```

```
Class balance:
+-----+-----+
|label|count|
+-----+-----+
|  0.0| 5262|
|  1.0| 2599|
+-----+-----+

Distinct positive-class bylines: 259
Distinct negative-class bylines: 303

Positive class — top contributing bylines:
+----------------------------------------+-----+
|bylines                                 |count|
+----------------------------------------+-----+
|Australian Education Union              |255  |
|James Oliver Gibson                     |231  |
|It's Not A Race                         |131  |
|polipedia.com.au                        |81   |
|Craig Ondarchie MP                      |73   |
|First Tier Media                        |69   |
|Australian Education Union Federal Branch|64   |
|Cape York Institute                     |62   |
|Australian Parents for Climate Action   |50   |
|United Workers Union                    |43   |
+----------------------------------------+-----+
only showing top 10 rows

Negative class — top contributing bylines:
+--------------------------------------+-----+
|bylines                               |count|
+--------------------------------------+-----+
|Greenpeace Australia Pacific          |1723 |
|Festival of Dangerous Ideas           |629  |
|Private Media                         |340  |
|World Animal Protection Australia     |285  |
|National Youth Commission Australia   |128  |
|SBS Cantonese 廣東話節目              |125  |
|Special Broadcasting Service          |111  |
|The University of Melbourne           |91   |
|The Australian Institute of Architects|82   |
|Very Special Kids                     |77   |
+--------------------------------------+-----+
only showing top 10 rows
```

tokenise, drop stopwords, vectorise. uses spark mllib's pipeline so the same transform applies at inference time.

In [22]:
```
tokenizer = RegexTokenizer(inputCol='text', outputCol='tokens',
                           pattern=r'\s+', minTokenLength=4)

# election seeds are NOT in the stopword list — they're the obvious election
# vocabulary the model should be allowed to use as features. the seed list is
# only used in cell 4 as a positive-class filter to drop non-political bylines
```

```
# whose spend timing happens to look election-aligned.
stop_words = list(set(StopWordsRemover.loadDefaultStopWords('english')) | {
    'australia', 'australian', 'australians', 'sign', 'petition', 'please',
    'today', 'help', 'one', 'go', 'http', 'https', 'www', 'com',
    # calendar-month leakage — these correlate with the election window for
    # trivial temporal reasons, not content.
    'january', 'february', 'march', 'april', 'may', 'june',
    'july', 'august', 'september', 'october', 'november', 'december',
    'eofy',
})

remover = StopWordsRemover(inputCol='tokens', outputCol='tokens_clean',
                           stopWords=stop_words)

cv = CountVectorizer(inputCol='tokens_clean', outputCol='tf',
                     minDF=20, maxDF=0.3, vocabSize=5000)
idf = IDF(inputCol='tf', outputCol='features')
lr  = LogisticRegression(featuresCol='features', labelCol='label',
                         maxIter=50, regParam=0.01)

pipeline = Pipeline(stages=[tokenizer, remover, cv, idf, lr])
```

train/test split 80/20, fit, evaluate.

In [23]:
```
train, test = train_full.randomSplit([0.8, 0.2], seed=42)
train = train.cache()
test = test.cache()

print(f'train: {train.count():,}')
print(f'test:  {test.count():,}')

model = pipeline.fit(train)
pred = model.transform(test).cache()
```

```
train: 6,353
test:  1,508
```

auc and confusion-matrix metrics on the held-out test set.

In [24]:
```
auc_eval = BinaryClassificationEvaluator(labelCol='label', metricName='areaUnderROC
auc = auc_eval.evaluate(pred)
print(f'AUC: {auc:.4f}')

for metric in ['accuracy', 'weightedPrecision', 'weightedRecall', 'f1']:
    e = MulticlassClassificationEvaluator(labelCol='label',
                                          predictionCol='prediction',
                                          metricName=metric)
    print(f'{metric}: {e.evaluate(pred):.4f}')

print('\nConfusion matrix (label × prediction):')
pred.groupBy('label', 'prediction').count().orderBy('label', 'prediction').show()
```

```
AUC: 0.9738
accuracy: 0.9350
weightedPrecision: 0.9348
weightedRecall: 0.9350
f1: 0.9348

Confusion matrix (label × prediction):
+-----+----------+-----+
|label|prediction|count|
+-----+----------+-----+
|  0.0|       0.0|  966|
|  0.0|       1.0|   44|
|  1.0|       0.0|   54|
|  1.0|       1.0|  444|
+-----+----------+-----+
```

leave-one-advertiser-out validation. for each election_only advertiser, hold their ads out, retrain on the rest, and score the held-out advertiser's pre-election ads. if precision stays high across all hold-outs, the model is learning content not advertiser identity. if it collapses for any one, the model was leaning on that advertiser's vocabulary.

In [25]:
```python
# leave-one-advertiser-out across the bylines contributing the most positive
# examples. with hashtag-based labels the positive class draws from dozens of
# distinct bylines, so loo over the top contributors is a good generalisation
# check — if holding one big positive-class byline out tanks the model, the
# model was leaning on their style.
TOP_N_BYLINES_TO_LOO = 15

positive_bylines_pdf = (train_full.filter(col('label') == 1.0)
                        .groupBy('bylines').count()
                        .orderBy(desc('count'))
                        .limit(TOP_N_BYLINES_TO_LOO)
                        .toPandas())
loo_bylines = positive_bylines_pdf['bylines'].tolist()
print(f'LOO over top {len(loo_bylines)} positive-class bylines.')

loo_results = []
for held_out in loo_bylines:
    train_loo = train_full.filter(col('bylines') != held_out)
    held_ads  = train_full.filter((col('bylines') == held_out) &
                                  (col('label') == 1.0))
    n_held = held_ads.count()
    if n_held == 0:
        continue

    m = pipeline.fit(train_loo)
    p = m.transform(held_ads).select('prediction').toPandas()
    held_recall = (p['prediction'] == 1.0).mean()
    loo_results.append({
        'held_out': held_out,
        'n_ads_held_out': n_held,
        'recall_on_held_out': held_recall,
    })
```

```
loo_df = pd.DataFrame(loo_results).sort_values('recall_on_held_out')
display(loo_df)

unweighted_mean = loo_df['recall_on_held_out'].mean()
min_recall      = loo_df['recall_on_held_out'].min()
weighted_mean   = ((loo_df['recall_on_held_out'] * loo_df['n_ads_held_out']).sum()
                   loo_df['n_ads_held_out'].sum())

print(f'\nUnweighted mean recall:  {unweighted_mean:.3f}')
print(f'Ad-weighted mean recall: {weighted_mean:.3f}')
print(f'Min recall (any byline): {min_recall:.3f}')
```

```
LOO over top 15 positive-class bylines.
```

	held_out	n_ads_held_out	recall_on_held_out
8	Australian Parents for Climate Action	50	0.000000
3	polipedia.com.au	81	0.086420
2	It's Not A Race	131	0.229008
14	Brad Battin MP	32	0.375000
10	The Climate Council	42	0.476190
7	Cape York Institute	62	0.516129
1	James Oliver Gibson	231	0.545455
11	Bill Tilley MP	38	0.605263
4	Craig Ondarchie MP	73	0.630137
9	United Workers Union	43	0.790698
13	Mark Swivel Team	34	0.882353
0	Australian Education Union	255	0.901961
6	Australian Education Union Federal Branch	64	1.000000
5	First Tier Media	69	1.000000
12	ABC Friends	36	1.000000

```
Unweighted mean recall:  0.603
Ad-weighted mean recall: 0.612
Min recall (any byline): 0.000
```

apply the model to every v3 advocacy ad, extracting the probability of class 1 (election-content).

In [26]:
```
from pyspark.sql.functions import udf
from pyspark.sql.types import DoubleType

extract_prob = udf(lambda v: float(v[1]), DoubleType())

scored = (model.transform(v3)
    .withColumn('election_prob', extract_prob('probability'))
```

```
    .select('id', 'bylines', 'page_name', 'ad_creation_date',
            'spend_mid', 'text', 'election_prob')
    .cache())

print(f'Scored ads: {scored.count():,}')
```

```
[Stage 2217:=========================================>             (3 + 1) / 4]
Scored ads: 94,358
```

false-positive check. take ads with the highest election_prob but from advertisers we'd intuitively expect to score low (i.e. ongoing humanitarian / environmental advertisers, far away from election day). if the model is well-calibrated, these should genuinely contain election-themed content. if not, the model is over-firing.

In [27]:
```
# ads from outside the hot zone with very high election_prob
fp_check = (scored
    .filter((col('ad_creation_date') < lit(hot_start)) |
            (col('ad_creation_date') > lit(hot_end)))
    .filter(col('election_prob') > 0.85)
    .orderBy(desc('election_prob'))
    .limit(30)
    .toPandas())

fp_check['text_preview'] = fp_check['text'].str[:120]
print('High-scoring ads from outside hot zone (potential false positives):')
display(fp_check[['bylines', 'ad_creation_date', 'election_prob', 'text_preview']])
```

```
High-scoring ads from outside hot zone (potential false positives):
```

	bylines	ad_creation_date	election_prob	text_preview
0	Health Care Providers Association	2022-08-12	1.0	what if you could get funded by the government...
1	Ananda Ranga Lalbahadur Sunkanpally	2022-11-06	1.0	i am your best choice i am the most...
2	Ananda Ranga Lalbahadur Sunkanpally	2022-10-07	1.0	with the respect and dignity i fly high \n\n...
3	Health Care Providers Association	2022-08-12	1.0	are you an accountant or a book keeper \nby be...
4	Ananda Ranga Lalbahadur Sunkanpally	2022-10-25	1.0	i am your best choice i am the most...
5	Ananda Ranga Lalbahadur Sunkanpally	2022-10-11	1.0	i am your best choice \ni request you to vo...
6	Steven B Hughes	2022-09-22	1.0	why did the chicken cross the road because co...
7	Ananda Ranga Lalbahadur Sunkanpally	2022-09-25	1.0	i request you to vote for me sunkanpally a...
8	Ananda Ranga Lalbahadur Sunkanpally	2022-10-10	1.0	to all beloved north ward council voters city...
9	Ananda Ranga Lalbahadur Sunkanpally	2022-10-25	1.0	i am your best choice i am the most...
10	Health Care Providers Association	2022-08-12	1.0	are you a registered nurse do you want to bec...
11	Ananda Ranga Lalbahadur Sunkanpally	2022-10-13	1.0	i am your best choice \n my body m...
12	Ananda Ranga Lalbahadur Sunkanpally	2022-10-11	1.0	my body mind and soul is dedicated to acti...
13	Ananda Ranga Lalbahadur Sunkanpally	2022-10-21	1.0	i am your best choice \n my body m...
14	Health Care Providers Association	2022-08-21	1.0	are you a registered nurse or do you know one ...
15	Ananda Ranga Lalbahadur Sunkanpally	2022-10-04	1.0	with the respect and dignity i fly high \n\n...
16	Steven B Hughes	2022-09-14	1.0	why did the chicken cross the road because co...
17	Canberra Weekly	2022-09-15	1.0	canberra grammar school sports camps offer a t...
18	Health Care Providers Association	2022-08-22	1.0	what if you could start a business that helps ...

	bylines	ad_creation_date	election_prob	text_preview
19	Save The Preston Market Action Group	2022-11-11	1.0	important election information for preston...
20	Independent Cowper Pty LTD	2022-04-09	1.0	why politicians create anxiety \n\nare you afr...
21	Australian Christian Lobby	2022-10-27	1.0	there is a war waging in victoria against peop...
22	Australian Christian Lobby	2022-10-27	1.0	the victorian government needs people to hold ...
23	Chris Crabbe	2022-10-17	1.0	your ballot paper will be arriving this week ...
24	Chris Crabbe	2022-10-17	1.0	your ballot paper will be arriving this week ...
25	Steven B Hughes	2022-03-31	1.0	frankston residents your rates are far too hi...
26	Health Care Providers Association	2022-08-21	1.0	are you an accountant or a bookkeeper \nby bec...
27	Tracie M Lund	2022-10-27	1.0	frequently asked questions \n\n why are you st...
28	Tracie M Lund	2022-10-27	1.0	frequently asked questions \n\n why are you st...
29	Australian Christian Lobby	2022-10-27	1.0	schools have been given the green light to tra...

aggregate per advertiser. for each byline compute total spend and spend-weighted mean election_prob. high mean = advertiser's portfolio is dominated by election content. low mean = mostly non-election content.

In [28]:
```
from pyspark.sql.functions import sum as spark_sum, avg

per_advertiser = (scored
    .filter(col('bylines').isNotNull() & col('spend_mid').isNotNull())
    .withColumn('weighted_prob', col('election_prob') * col('spend_mid'))
    .groupBy('bylines')
    .agg(
        spark_sum('spend_mid').alias('total_spend'),
        spark_sum('weighted_prob').alias('weighted_prob_sum'),
        spark_count('*').alias('n_ads'),
    )
    .withColumn('mean_election_prob_weighted',
                col('weighted_prob_sum') / col('total_spend'))
    .orderBy(desc('total_spend'))
    .toPandas())
```

```
top30 = per_advertiser.head(30).reset_index(drop=True)
display(top30[['bylines', 'total_spend', 'n_ads', 'mean_election_prob_weighted']])
```

	bylines	total_spend	n_ads	mean_election_prob_weighted
0	Greenpeace Australia Pacific	1556769.5	9661	0.118278
1	Amnesty International Australia	854819.0	2262	0.283277
2	Solutions for Australia	656140.5	1819	0.425051
3	Australian Conservation Foundation	644985.5	2229	0.354141
4	Climate 200	629674.5	1351	0.775033
5	Thrive By Five	612850.0	1100	0.727414
6	Australian Unions	543427.5	545	0.395147
7	Australia for UNHCR	514934.5	1331	0.239935
8	Future Super	402754.0	292	0.507733
9	Plan International Australia	378201.5	397	0.188576
10	The Pharmacy Guild of Australia	335291.5	1017	0.525762
11	Médecins Sans Frontières Australia	306915.5	469	0.205338
12	Smart Voting	302671.5	57	0.805251
13	Victorian Electoral Commission	290254.0	192	0.612487
14	Advance Australia	284893.5	613	0.785366
15	Our Watch	276536.0	28	0.343796
16	Australian Christian Lobby	274256.5	187	0.628516
17	Bank Australia	263669.5	461	0.500682
18	Oxfam Australia	263180.5	2139	0.343558
19	Australian Education Union	253022.5	255	0.979735
20	Save the Children Australia	239982.0	1036	0.346452
21	It's Not A Race	221677.5	445	0.578251
22	Monash University	217402.0	96	0.339137
23	Koffels Solicitors and Barristers	212719.0	62	0.217181
24	Chuffed.org	204755.0	290	0.365659
25	GetUp!	203838.0	424	0.737956
26	Health Care Providers Association	202760.5	79	0.719673
27	Destiny Rescue	201487.0	326	0.809879
28	Indigenous Law Centre, UNSW	187144.0	12	0.295269
29	United Workers Union	179795.0	510	0.766178

compute a single content classifier metric per advertiser: the mean election_prob across ads created in the pre-election window (4 weeks before polling day). exported as data/content_classifier_per_advertiser.csv for nb 07 (combined PAC ranking).

In [29]:
```
from pyspark.sql.functions import mean as spark_mean

advertiser_pre = (scored
    .filter(col('bylines').isNotNull())
    .filter((col('ad_creation_date') >= lit(hot_start)) &
            (col('ad_creation_date') <  lit(ELECTION_DATE.date().isoformat())))
    .groupBy('bylines')
    .agg(spark_mean('election_prob').alias('mean_prob_pre'),
         spark_count('*').alias('n_ads_pre'))
    .toPandas())

content_export = advertiser_pre.merge(
    per_advertiser[['bylines', 'total_spend', 'n_ads', 'mean_election_prob_weighted
    on='bylines', how='outer')

CONTENT_SCORES_CSV = '../data/content_classifier_per_advertiser.csv'
content_export.to_csv(CONTENT_SCORES_CSV, index=False)
print(f'Wrote {CONTENT_SCORES_CSV}  ({len(content_export):,} advertisers)')

# Preview --- top 20 by pre-election content score
display(content_export.sort_values('mean_prob_pre', ascending=False).head(20))
```

```
Wrote ../data/content_classifier_per_advertiser.csv  (2,439 advertisers)
```

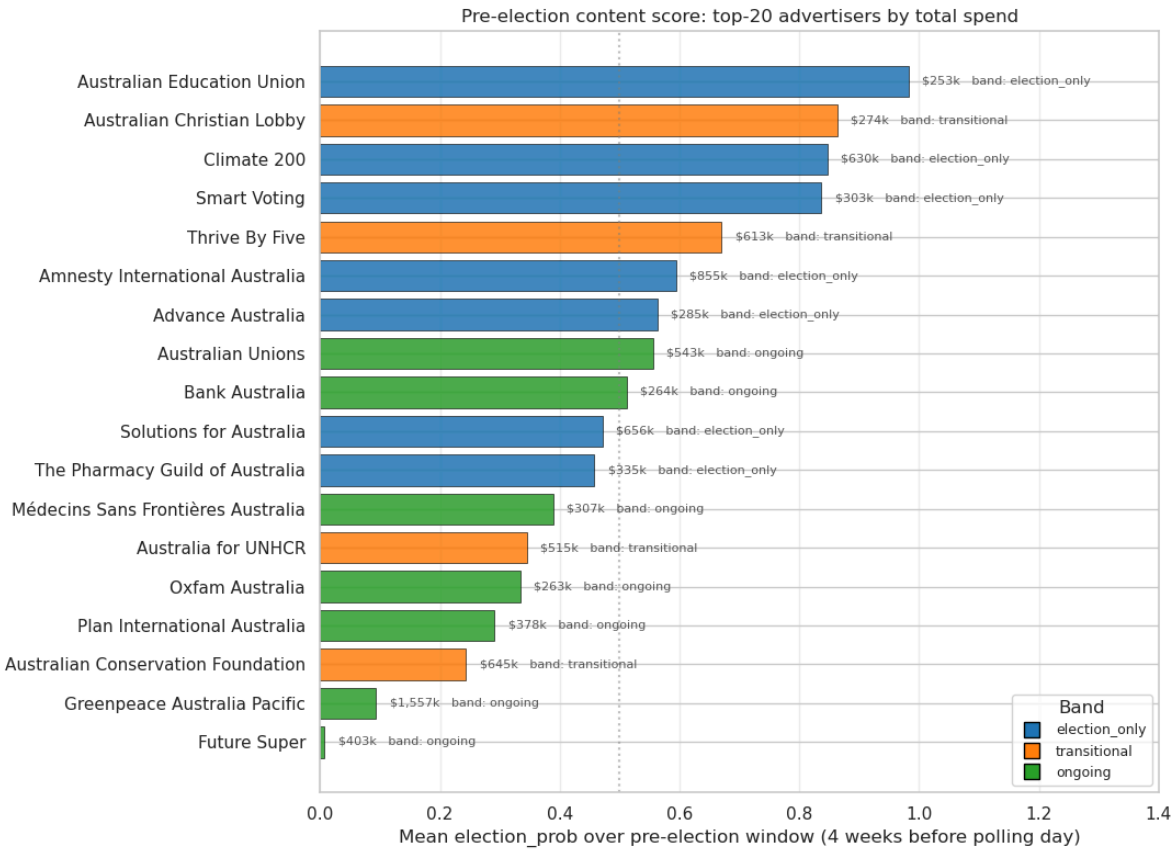
	bylines	mean_prob_pre	n_ads_pre	total_spend	n_ads	mean_election_prob_weighte
638	The Executive Council of Australian Jewry (ECAJ)	1.000000	2.0	546.5	7	0.37848
392	Sustainable Australia Party	0.999999	3.0	3535.5	29	0.68389
486	Andrew John Bartlett	0.999998	1.0	49.5	1	0.99999
273	Doctors of	0.999990	1.0	1096.0	8	0.87231
103	Project Planet Australia Inc	0.999988	9.0	445.5	9	0.99998
667	David Andrew Fulton	0.999971	2.0	247.5	5	0.41276
79	Alison Champion	0.999944	1.0	1644.5	11	0.42896
126	Anthony Peck	0.999923	1.0	49.5	1	0.99992
402	Mitchell Cameron Griffin	0.999894	2.0	148.5	3	0.73598
495	Mornington Peninsula Climate Action Network	0.999870	3.0	248.5	3	0.99992
262	Teresa Maree Lane	0.999817	1.0	247.5	5	0.68780
462	Women With Disabilities Australia (WWDA)	0.999533	1.0	148.5	3	0.35253
661	BREAZE Inc.	0.999442	1.0	199.0	2	0.25220

	bylines	mean_prob_pre	n_ads_pre	total_spend	n_ads	mean_election_prob_weighte
411	Australian Youth Climate Coalition	0.999436	45.0	42160.5	179	0.86303
474	Wayne R Farnham	0.999206	1.0	3820.5	59	0.82491
335	Seed Indigenous Youth Climate Network	0.999118	1.0	348.5	3	0.86268
586	Daniel D O'Brien	0.999036	1.0	695.0	10	0.71832
345	The Climate Reality Project - Australia and Pa...	0.998667	1.0	598.0	4	0.73030
215	Epping Civic Trust	0.998419	3.0	544.5	11	0.37091
188	John Kennedy MP	0.998402	2.0	5030.5	39	0.45679

pre-election content score bar chart for the top-20 spenders --- sorted descending so the most election-content-heavy advertiser is at the top.

In [30]:
```
import numpy as np
from matplotlib.lines import Line2D

BAND_COLORS = {
    'election_only': '#1f77b4',
    'transitional':  '#ff7f0e',
    'ongoing':       '#2ca02c',
    'off_campaign':  '#d62728',
}

top20 = content_export.sort_values('total_spend', ascending=False).head(20).copy()
if have_bands:
    top20 = top20.merge(bands_pdf, on='bylines', how='left')
else:
    top20['band'] = '?'

top20 = (top20
    .dropna(subset=['mean_prob_pre'])
    .sort_values('mean_prob_pre', ascending=True)
    .reset_index(drop=True))
```

```
colours = top20['band'].map(BAND_COLORS).fillna('#888888')

fig, ax = plt.subplots(figsize=(11, 8))
y_pos = np.arange(len(top20))

ax.barh(y_pos, top20['mean_prob_pre'], color=colours,
        edgecolor='black', linewidth=0.5, alpha=0.85)

for i, (_, row) in enumerate(top20.iterrows()):
    ax.text(row['mean_prob_pre'] + 0.01, i,
            f'  ${row["total_spend"]/1000:,.0f}k   band: {row["band"]}',
            va='center', fontsize=8, alpha=0.75)

ax.axvline(0.5, color='gray', linestyle=':', alpha=0.5)
ax.set_yticks(y_pos)
ax.set_yticklabels([b[:35] for b in top20['bylines']])
ax.set_xlim(0, 1.4)
ax.set_xlabel('Mean election_prob over pre-election window (4 weeks before polling
ax.set_title('Pre-election content score: top-20 advertisers by total spend')

present_bands = [b for b in BAND_COLORS if b in top20['band'].values]
if len(present_bands) > 1:
    band_handles = [
        Line2D([0], [0], marker='s', linestyle='', color='w',
               markerfacecolor=BAND_COLORS[b], markeredgecolor='black',
               markersize=10, label=b)
        for b in present_bands
    ]
    ax.legend(handles=band_handles, loc='lower right', fontsize=9, title='Band')

ax.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()
```



the pre-election content score is now in data/content_classifier_per_advertiser.csv. nb 07 joins it with the temporal classifier (nb 05) and produces the combined PAC ranking.

07_top_ads_per_advertiser.ipynb

combined PAC analysis. joins the temporal classifier (nb 05, all_bylines_bands.csv) and the content classifier (nb 06, content_classifier_per_advertiser.csv) per-advertiser, computes a combined PAC-likeness score, and produces two tables for the report:

1. top 20 advertisers by total spend, with their three metrics (band / EC index, pre-election content score) and the combined PAC score.
2. top 10 PACs (highest combined score), with their top 3 ads by spend each.

the combined score is `pac_score = ec_index * mean_prob_pre` --- a multiplicative AND composite that is high only when both temporal concentration and pre-election content density are high together.

In [1]:
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, expr, coalesce, substring, first,
    sum as spark_sum, count as spark_count,
    row_number, desc,
)
from pyspark.sql.window import Window
import pandas as pd
import numpy as np

spark = SparkSession.builder \
    .appName('FB_API_combined_pacs') \
    .config('spark.sql.parquet.output.committer.class',
            'org.apache.parquet.hadoop.ParquetOutputCommitter') \
    .config('mapreduce.fileoutputcommitter.algorithm.version', '2') \
    .getOrCreate()

print('Master:', spark.sparkContext.master)
print('Spark version:', spark.version)
```

```
Master: yarn
Spark version: 3.5.0
```

```
26/05/18 07:48:38 WARN SparkSession: Using an existing Spark session; only runtime S
QL configurations will take effect.
```

In [2]:
```
V3_PATH        = '/user/s3348393/main/preprocessing/v3/parquet'
BANDS_CSV      = '../data/all_bylines_bands.csv'
CONTENT_CSV    = '../data/content_classifier_per_advertiser.csv'

TOP_20_OUT_CSV     = '../data/top_20_with_metrics.csv'
TOP_PACS_OUT_CSV   = '../data/top_10_pacs_with_top_ads.csv'

TOP_N_SPENDERS    = 20

# A PAC must clear both classifier thresholds (top-right quadrant of the
# pac-likeness space). Top 10 PACs are then ranked by total spend.
EC_INDEX_MIN      = 0.5
MEAN_PROB_PRE_MIN = 0.5
TOP_N_PACS        = 10
TOP_N_ADS_PER_PAC = 3
```

load temporal and content scores per advertiser, join them, compute the combined PAC score.

In [3]:
```
bands_df   = pd.read_csv(BANDS_CSV)
content_df = pd.read_csv(CONTENT_CSV)

print(f'temporal classifier rows: {len(bands_df):,}')
print(f'content classifier rows:  {len(content_df):,}')

# Inner-join on bylines --- only advertisers that survived both classifiers
metrics = (bands_df[['bylines', 'band', 'persistence', 'ec_index']]
    .merge(
        content_df[['bylines', 'mean_prob_pre',
                    'mean_election_prob_weighted', 'total_spend']],
        on='bylines', how='inner'
    ))

# Combined PAC-likeness score: strict-AND multiplicative composite
metrics['pac_score'] = metrics['ec_index'] * metrics['mean_prob_pre']

print(f'\njoined metrics: {len(metrics):,} advertisers')
print(f'pac_score range: {metrics["pac_score"].min():.3f} - {metrics["pac_score"].m
```

```
temporal classifier rows: 567
content classifier rows:  2,439

joined metrics: 567 advertisers
pac_score range: 0.001 - 0.914
```

table 1: top 20 advertisers by total spend, with their three metrics and the combined PAC score.

In [4]:
```
top20_metrics = (metrics
    .sort_values('total_spend', ascending=False)
    .head(TOP_N_SPENDERS)
    .reset_index(drop=True))

display_cols = ['bylines', 'total_spend', 'band',
```

```
                'ec_index', 'mean_prob_pre', 'pac_score']

print(f'Top {TOP_N_SPENDERS} advertisers by total spend:')
display(top20_metrics[display_cols])

top20_metrics[display_cols].to_csv(TOP_20_OUT_CSV, index=False)
print(f'\nWrote {TOP_20_OUT_CSV}')
```

```
Top 20 advertisers by total spend:
```

	bylines	total_spend	band	ec_index	mean_prob_pre	pac_score
0	Greenpeace Australia Pacific	1556769.5	ongoing	0.466242	0.093489	0.043589
1	Amnesty International Australia	854819.0	election_only	0.441121	0.594555	0.262271
2	Solutions for Australia	656140.5	election_only	0.852070	0.471664	0.401890
3	Australian Conservation Foundation	644985.5	transitional	0.538804	0.243394	0.131142
4	Climate 200	629674.5	election_only	0.499911	0.847031	0.423440
5	Thrive By Five	612850.0	transitional	0.648939	0.670353	0.435018
6	Australian Unions	543427.5	ongoing	0.635139	0.556150	0.353233
7	Australia for UNHCR	514934.5	transitional	0.610156	0.345543	0.210835
8	Future Super	402754.0	ongoing	0.488685	0.006898	0.003371
9	Plan International Australia	378201.5	ongoing	0.598608	0.290985	0.174186
10	The Pharmacy Guild of Australia	335291.5	election_only	0.629485	0.457915	0.288251
11	Médecins Sans Frontières Australia	306915.5	ongoing	0.588786	0.388875	0.228964
12	Smart Voting	302671.5	election_only	0.938735	0.836677	0.785418
13	Victorian Electoral Commission	290254.0	election_only	0.409471	NaN	NaN
14	Advance Australia	284893.5	election_only	0.374626	0.563391	0.211061
15	Our Watch	276536.0	ongoing	0.145009	NaN	NaN
16	Australian Christian Lobby	274256.5	transitional	0.629040	0.863279	0.543037
17	Bank Australia	263669.5	ongoing	0.567246	0.511342	0.290057
18	Oxfam Australia	263180.5	ongoing	0.553953	0.334564	0.185333
19	Australian Education Union	253022.5	election_only	0.802521	0.982732	0.788663

```
Wrote ../data/top_20_with_metrics.csv
```

table 2: top 10 PACs by combined score, with each PAC's top 3 ads by spend.

In [5]:
```
# Define a PAC as ec_index > 0.5 AND mean_prob_pre > 0.5 (the top-right
# quadrant of the PAC-likeness scatter) AND band != 'ongoing'. The
# 'ongoing' band by definition means post-election spend matches
# pre-election spend, which contradicts the PAC definition --- so we
```

```
# exclude it even when the content classifier scores high (this catches
# year-round advocates whose vocabulary overlaps with electoral
# language, e.g. unions, charities).
pacs = (metrics
    .query('ec_index > @EC_INDEX_MIN '
           'and mean_prob_pre > @MEAN_PROB_PRE_MIN '
           "and band != 'ongoing'")
    .sort_values('total_spend', ascending=False)
    .reset_index(drop=True))

print(f'{len(pacs)} advertisers meet the PAC definition '
      f'(ec_index > {EC_INDEX_MIN}, mean_prob_pre > {MEAN_PROB_PRE_MIN}, '
      f"band != 'ongoing').")

top_pacs = pacs.head(TOP_N_PACS).reset_index(drop=True)
top_pac_bylines = top_pacs['bylines'].tolist()

print(f'\nTop {min(TOP_N_PACS, len(pacs))} PACs by spend:')
display(top_pacs[['bylines', 'total_spend', 'band', 'ec_index',
                  'mean_prob_pre', 'pac_score']])

# Pull each PAC's top 3 ads from v3 (by total spend per unique creative title)
def first_non_empty(col_name):
    return expr(f"filter({col_name}, x -> x is not null and length(x) > 0)[0]")

ad_title = coalesce(
    first_non_empty('creative_link_titles'),
    substring(first_non_empty('creative_bodies'), 1, 80),
)

v3 = spark.read.parquet(V3_PATH).filter(col('match_type').isNull())

ad_spend = (v3
    .filter(col('bylines').isin(top_pac_bylines) & col('spend_mid').isNotNull())
    .withColumn('ad_title', ad_title)
    .filter(col('ad_title').isNotNull())
    .groupBy('bylines', 'ad_title')
    .agg(
        spark_sum('spend_mid').alias('total_spend'),
        spark_count('*').alias('n_runs'),
        first('ad_snapshot_url').alias('example_url'),
    ))

w = Window.partitionBy('bylines').orderBy(desc('total_spend'))
top_pac_ads = (ad_spend
    .withColumn('rank', row_number().over(w))
    .filter(col('rank') <= TOP_N_ADS_PER_PAC)
    .orderBy('bylines', 'rank')
    .toPandas())

# Preserve PAC ranking order
top_pac_ads['_byline_order'] = top_pac_ads['bylines'].map(
    {b: i for i, b in enumerate(top_pac_bylines)})
top_pac_ads = (top_pac_ads
    .sort_values(['_byline_order', 'rank'])
    .drop(columns='_byline_order')
```

```
    .reset_index(drop=True))

print(f'\n{len(top_pac_ads)} rows: top {TOP_N_ADS_PER_PAC} ads for each of the top
```

```
97 advertisers meet the PAC definition (ec_index > 0.5, mean_prob_pre > 0.5, band !=
'ongoing').

Top 10 PACs by spend:
```

	bylines	total_spend	band	ec_index	mean_prob_pre	pac_score
0	Thrive By Five	612850.0	transitional	0.648939	0.670353	0.435018
1	Smart Voting	302671.5	election_only	0.938735	0.836677	0.785418
2	Australian Christian Lobby	274256.5	transitional	0.629040	0.863279	0.543037
3	Australian Education Union	253022.5	election_only	0.802521	0.982732	0.788663
4	It's Not A Race	221677.5	election_only	0.828475	0.530003	0.439094
5	GetUp!	203838.0	election_only	0.767506	0.776768	0.596174
6	ABC Friends	146068.0	election_only	0.740500	0.794177	0.588088
7	Australian Automobile Association	128001.5	election_only	0.699492	0.870148	0.608662
8	TAB	93490.0	election_only	0.863349	0.767769	0.662852
9	Cancer Council SA	81084.0	transitional	0.511264	0.604354	0.308984

```
26/05/18 07:48:54 WARN YarnScheduler: Initial job has not accepted any resources; ch
eck your cluster UI to ensure that workers are registered and have sufficient resour
ces
26/05/18 07:49:09 WARN YarnScheduler: Initial job has not accepted any resources; ch
eck your cluster UI to ensure that workers are registered and have sufficient resour
ces
```

```
30 rows: top 3 ads for each of the top 10 PACs.
```

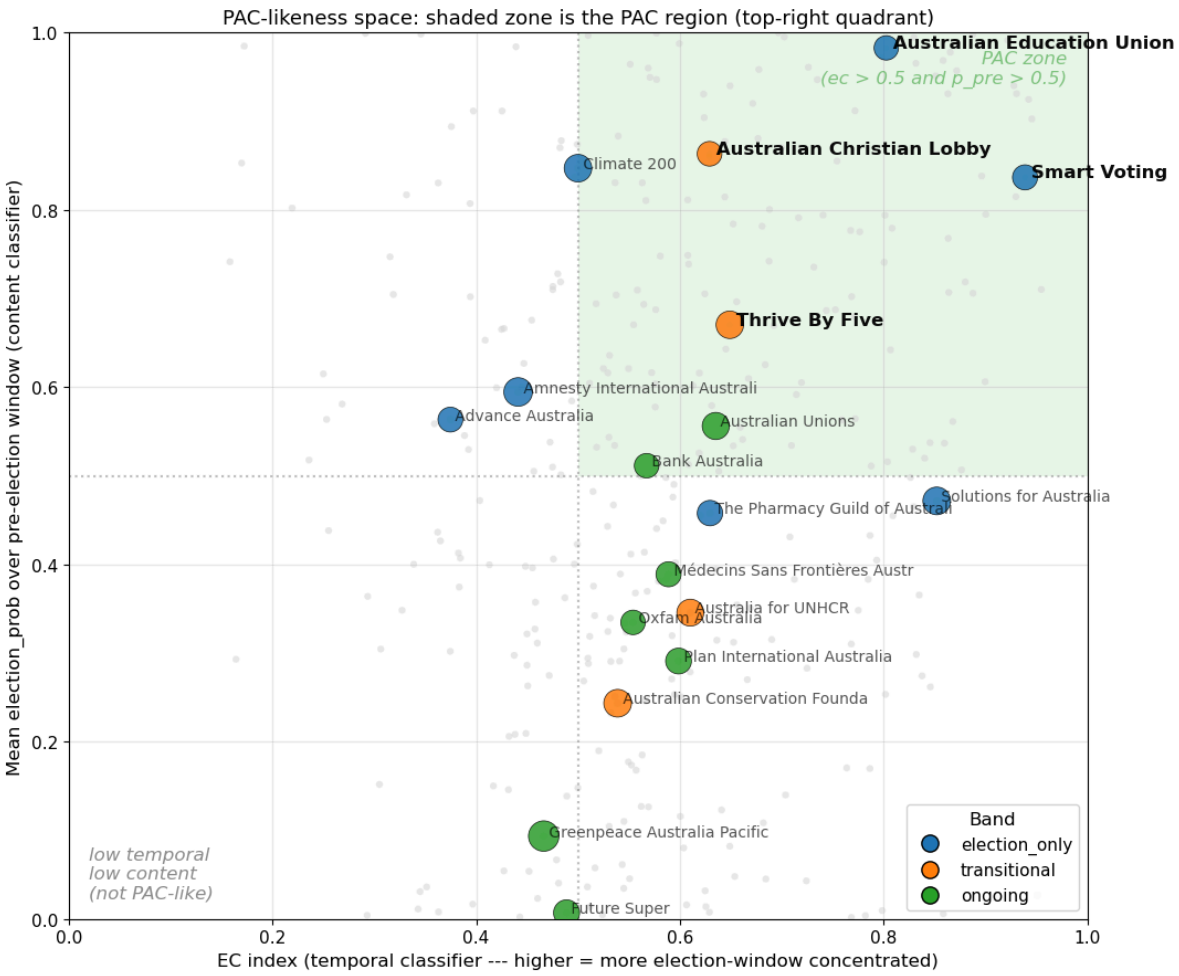
visualisations. (1) a PAC-likeness scatter showing where every advertiser sits in temporal-classifier $\times$ content-classifier space, with the top 20 spenders highlighted and the top 10 PACs in bold. (2) a horizontal bar chart of the top 10 PACs ranked by combined PAC score.

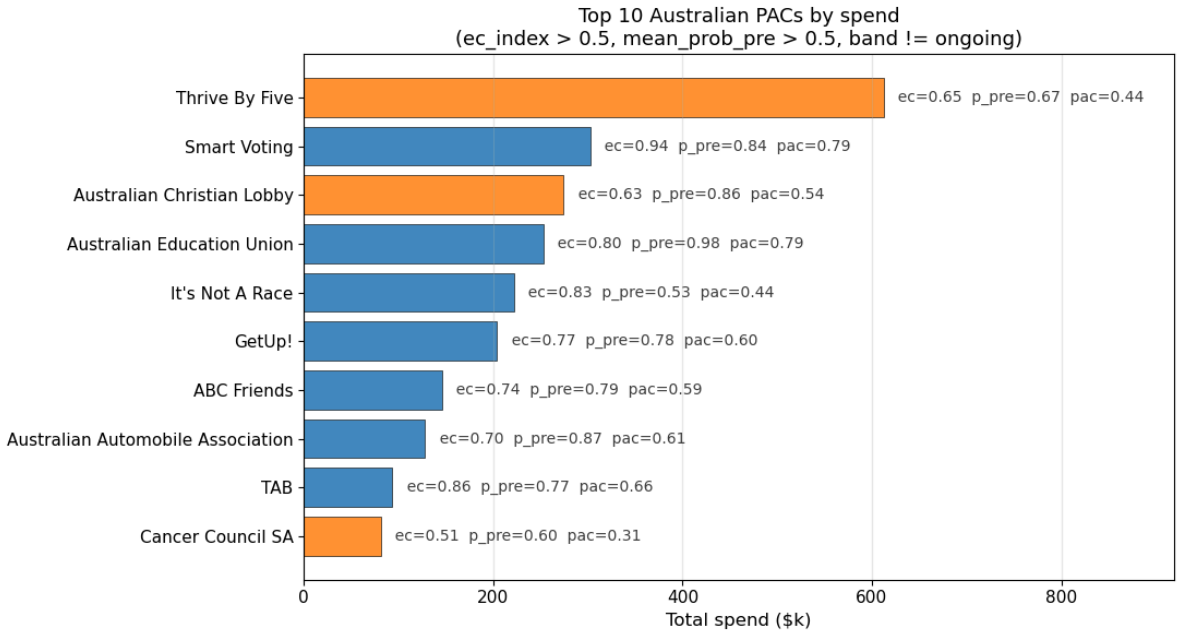
In [8]:
```
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.patches import Rectangle

BAND_COLORS = {
    'election_only': '#1f77b4',
    'transitional':  '#ff7f0e',
    'ongoing':       '#2ca02c',
}

# === Viz 1: PAC-likeness scatter (temporal x content) ===
```

```
# Background: all advertisers as small grey dots.
# Foreground: top 20 spenders sized by log spend, coloured by band.
# Top-right quadrant (ec_index > 0.5, mean_prob_pre > 0.5) shaded as PAC zone.
# PACs identified by the threshold filter labelled in bold.

fig, ax = plt.subplots(figsize=(11, 9))

# Shade the PAC quadrant
ax.add_patch(Rectangle((EC_INDEX_MIN, MEAN_PROB_PRE_MIN),
                       1 - EC_INDEX_MIN, 1 - MEAN_PROB_PRE_MIN,
                       facecolor='#2ca02c', alpha=0.10,
                       edgecolor='none', zorder=0))

ax.scatter(metrics['ec_index'], metrics['mean_prob_pre'],
           s=20, c='lightgray', alpha=0.5, edgecolor='none', zorder=1)

top20_colours = top20_metrics['band'].map(BAND_COLORS).fillna('#888888')
spend_sizes = (np.log10(top20_metrics['total_spend'].fillna(1)) - 4).clip(lower=0.5

ax.scatter(top20_metrics['ec_index'], top20_metrics['mean_prob_pre'],
           s=spend_sizes, c=top20_colours, alpha=0.85,
           edgecolor='black', linewidth=0.5, zorder=3)

top_pac_set = set(top_pacs['bylines'])
texts = []
for _, row in top20_metrics.iterrows():
    is_pac = row['bylines'] in top_pac_set
    texts.append(ax.text(
        row['ec_index'], row['mean_prob_pre'],
        ' ' + row['bylines'][:30],
        fontsize=12 if is_pac else 10,
        fontweight='bold' if is_pac else 'normal',
        alpha=0.95 if is_pac else 0.65,
        zorder=5 if is_pac else 4,
    ))

try:
    from adjustText import adjust_text
    adjust_text(texts, ax=ax,
                arrowprops=dict(arrowstyle='-', color='gray', alpha=0.5, lw=0.6))
except ImportError:
    pass

ax.axhline(MEAN_PROB_PRE_MIN, color='gray', linestyle=':', alpha=0.5)
ax.axvline(EC_INDEX_MIN, color='gray', linestyle=':', alpha=0.5)

ax.text(0.98, 0.98, 'PAC zone\n(ec > 0.5 and p_pre > 0.5)',
        transform=ax.transAxes, ha='right', va='top',
        fontsize=12, alpha=0.55, fontstyle='italic', color='#2ca02c')
ax.text(0.02, 0.02, 'low temporal\nlow content\n(not PAC-like)',
        transform=ax.transAxes, ha='left', va='bottom',
        fontsize=12, alpha=0.45, fontstyle='italic')

band_handles = [
    Line2D([0], [0], marker='o', linestyle='', color='w',
           markerfacecolor=c, markeredgecolor='black', markersize=11, label=b)
```

```
    for b, c in BAND_COLORS.items()
]
ax.legend(handles=band_handles, loc='lower right', fontsize=11,
          title='Band', title_fontsize=12)

ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.set_xlabel('EC index (temporal classifier --- higher = more election-window conc
              fontsize=12)
ax.set_ylabel('Mean election_prob over pre-election window (content classifier)',
              fontsize=12)
ax.set_title('PAC-likeness space: shaded zone is the PAC region (top-right quadrant
             fontsize=13)
ax.tick_params(axis='both', labelsize=11)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()


# === Viz 2: Top PACs ranked by spend (bar length = total spend) ===

fig, ax = plt.subplots(figsize=(11, 6))

top_pacs_sorted = top_pacs.sort_values('total_spend', ascending=True).reset_index(d
y_pos = np.arange(len(top_pacs_sorted))
colours = top_pacs_sorted['band'].map(BAND_COLORS).fillna('#888888')

spend_k = top_pacs_sorted['total_spend'] / 1000
ax.barh(y_pos, spend_k, color=colours,
        edgecolor='black', linewidth=0.5, alpha=0.85)

for i, (_, row) in enumerate(top_pacs_sorted.iterrows()):
    ax.text((row['total_spend'] / 1000) + 5, i,
            f'  ec={row["ec_index"]:.2f}  p_pre={row["mean_prob_pre"]:.2f}  pac={ro
            va='center', fontsize=10, alpha=0.75)

ax.set_yticks(y_pos)
ax.set_yticklabels([b[:35] for b in top_pacs_sorted['bylines']], fontsize=11)
ax.set_xlim(0, spend_k.max() * 1.5)
ax.set_xlabel('Total spend ($k)', fontsize=12)
ax.set_title(f'Top {len(top_pacs)} Australian PACs by spend\n(ec_index > {EC_INDEX_
             fontsize=13)
ax.tick_params(axis='x', labelsize=11)

ax.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()
```

```
posx and posy should be finite values
posx and posy should be finite values
posx and posy should be finite values
posx and posy should be finite values
```





per-PAC display and export.

In [7]:
```
for byline in top_pac_bylines:
    sub = top_pac_ads[top_pac_ads['bylines'] == byline]
    pac_row = top_pacs[top_pacs['bylines'] == byline].iloc[0]
```

```
    print(f'\n=== {byline}  (pac_score={pac_row["pac_score"]:.3f}, '
          f'${pac_row["total_spend"]:,.0f} total, band={pac_row["band"]}) ===')
    if sub.empty:
        print('(no ads matched filter)')
        continue
    display(sub[['rank', 'ad_title', 'total_spend', 'n_runs']])

# Join PAC-level scores onto the ad-level rows for the export
top_pac_ads_export = top_pac_ads.merge(
    top_pacs[['bylines', 'pac_score', 'band']],
    on='bylines', how='left')

out_cols = ['bylines', 'pac_score', 'band', 'rank', 'ad_title',
            'total_spend', 'n_runs', 'example_url']
import re
top_pac_ads_export['example_url'] = top_pac_ads_export['example_url'].apply(
    lambda u: re.sub(r'(?<=[?&])access_token=[^&]*&?', '', u).rstrip('?&') if pd.no
)
top_pac_ads_export[out_cols].to_csv(TOP_PACS_OUT_CSV, index=False)
print(f'\nWrote {TOP_PACS_OUT_CSV}')
```

```
=== Thrive By Five  (pac_score=0.435, $612,850 total, band=transitional) ===
```

	rank	ad_title	total_spend	n_runs
0	1	Let's make early learning and childcare fair f...	128700.5	399
1	2	Country kids shouldn’t be left behind.	106430.5	39
2	3	Families should be able to choose what works f...	98973.5	53

```
=== Smart Voting  (pac_score=0.785, $302,672 total, band=election_only) ===
```

	rank	ad_title	total_spend	n_runs
3	1	Since the Coalition took power, prices have sk...	119245.5	9
4	2	We can’t afford three more years of economic b...	27993.0	14
5	3	Scott Morrison promised he’d create a national...	19696.5	7

```
=== Australian Christian Lobby  (pac_score=0.543, $274,256 total, band=transitional)
===
```

	rank	ad_title	total_spend	n_runs
6	1	Sign the petition to defend Christian schools	33748.5	3
7	2	{{product.name}}	20497.5	5
8	3	Secure your tickets today!	13494.0	12

```
=== Australian Education Union  (pac_score=0.789, $253,022 total, band=election_onl
y) ===
```

	rank	ad_title	total_spend	n_runs
9	1	Properly fund public schools	37498.5	3
10	2	Australia is in the midst of a skills crisis	23497.5	5
11	3	High quality education for every child	21247.0	6

```
=== It's Not A Race  (pac_score=0.439, $221,678 total, band=election_only) ===
```

	rank	ad_title	total_spend	n_runs
12	1	Government should be about setting the right p...	27337.0	26
13	2	We've changed!' says coalition full of climate...	17542.5	15
14	3	When it comes to #ClimateAction, Dave Sharma '...	13048.5	3

```
=== GetUp!  (pac_score=0.596, $203,838 total, band=election_only) ===
```

	rank	ad_title	total_spend	n_runs
15	1	Want climate leadership? Vote for it!	18629.5	41
16	2	Federal Election 2022	12498.5	3
17	3	We make the future. Join us this federal elect...	11498.5	3

```
=== ABC Friends  (pac_score=0.588, $146,068 total, band=election_only) ===
```

	rank	ad_title	total_spend	n_runs
18	1	Save our ABC	56131.5	437
19	2	Save Our ABC	35896.0	208
20	3	Kerry O'Brien stars in call-to-arms video just...	18246.5	7

```
=== Australian Automobile Association  (pac_score=0.609, $128,002 total, band=electi
on_only) ===
```

	rank	ad_title	total_spend	n_runs
21	1	This Election Demand Better	92086.5	1127
22	2	Australian Automobile Association	17034.5	231
23	3	Australian Automobile	9645.0	10

```
=== TAB  (pac_score=0.663, $93,490 total, band=election_only) ===
```

	rank	ad_title	total_spend	n_runs
24	1	Australian Election Markets	59093.5	13
25	2	Australian Election Markets	32498.5	3
26	3	Democracy snags are good, but Bunnings snags a...	649.5	1

```
=== Cancer Council SA  (pac_score=0.309, $81,084 total, band=transitional) ===
```

	rank	ad_title	total_spend	n_runs
27	1	Choose water instead	8947.5	5
28	2	Don't Miss Your Chance To Get A FREE Host Kit!	4797.0	6
29	3	Make a hairy sacrifice	4197.0	6

```
Wrote ../data/top_10_pacs_with_top_ads.csv
```